NEA Programming Project

Jayden Shah - [redacted]

Contents

Analysis **4**
Description of the problem 4
Computational Methods 4
Problem Recognition 4
Decomposition 4
Abstraction 5
Algorithms 5
Stakeholders 5
Interview 6
Interview Plan 6
Interview Transcript 6
Interview Analysis 7
Research 8
247 Backgammon 8
Backgammon js 9
Kahoot 10
Features of the solution 10
Limitations of the solution 12
Software and Hardware Requirements 13
Server 13
Client 13
Solution Requirements 14
Success Criteria 15
Design **17**
Decomposition 17
Menu 17
Networking 18
Game Logic 19
Rendering 19
Program Structure 20
Client Program Structure 21
Server Program Structure 21
Algorithms, Variables and Iterative Testing 23
Menu Module 23
Networking Module 27
Game Logic Module 34
Rendering Module 40
Usability Design 44
Stakeholder Comments 45
Changes 45
Post Development Testing 46
Function and Robustness Test Plan 46

Usability Test Plan....47
Development....49
Setting Up the Web Server....49
Iteration 1....50
Iteration 2....53
Validation....55
Review....55
Main Menu....56
Iteration 1....56
Iteration 2....65
Validation....74
Review....75
Rendering the Game....75
Iteration 1....76
Iteration 2....79
Validation....83
Review....83
Server-side Room Handling....84
Iteration 1....84
Validation....93
Review....93
Backgammon Gameplay....95
Iteration 1....95
Iteration 2....110
Iteration 3....119
Validation....121
Review....121
Maintainability....123
Post Development Testing....125
Function and Robustness Testing....125
Usability Testing....126
Stakeholder Feedback....129
Evaluation....130
Success Criteria....130
Unmet criteria....132
Usability features....133
Unmet Criteria....134
Maintenance....135
Corrective Maintenance....135
Adaptive Maintenance....135
Perfective Maintenance....135
Maintenance Features....136
Limitations....137
Improvements....138
Programming Techniques....139

Analysis

Description of the problem

Backgammon is one of the most popular tabletop games in the world. Traditionally, it is played over a physical board between two people. However, when I looked for a quick way to play with my friend remotely, I realised that the number of solutions to playing it online is quite limited and the few that exist aren't that good, whether it be due to intrusive ads, lack of customizability or unintuitive controls. This is unfortunate since the benefits of playing the game online make it much more accessible, customisable and cheaper.

As a basic outline, I intend to create a web-based version of backgammon that can be played remotely between two people. It will use a room-based system, so you'll need to enter a private room code to connect to a specific backgammon game against your friend. As it is a computational solution, I will need to develop a functional networking system and a web-based client program. Hopefully, this will make backgammon much more accessible and encourage people, who've never played it before, to start playing.

Computational Methods

This problem requires computational methods to solve. This is because the solution critically depends on computers for the functionality of both the client and the server. The client will use a web browser to take in mouse inputs and display to a screen, which depends on a computer in order to run. The server will run on software that uses sockets, ports and a network in order to function, which requires a computer. As a result, it is absolutely necessary to use computational methods to find and implement a solution. While backgammon is a physical game that can be played without a computer, this avoids the crucial problem that you cannot play the game remotely using a physical board.

Problem Recognition

The overall goal of the program is to allow two people to play backgammon together. In order to accomplish this, I first need to create a server system that can handle requests on moves, simulate the game logic and then respond with information about the game state to all of the clients connected to that specific game. Once I do that, I need to create a web-based client-side system that can send and receive information from the server, take in mouse inputs and display the game accordingly.

Decomposition

As a complex problem, it can be broken down into a number of sub-problems. These include the following:

- Take in mouse input from the client system to determine the action.

- Send data to the server, which processes the action and updates the game state.
- Receive data from the server that tells the client how to update itself.
- Display the backgammon game and relevant information in a GUI.

This process will make developing the program a lot easier as I can now focus on individual parts of the program and get each part to work before combining them. Furthermore, I can always divide each part further if I feel like the task is too complex.

Abstraction

Displaying and animating image assets is demanding for lower-end computers or older browsers, especially when you have to display many of them simultaneously (such as giving each piece an image asset). By using simple shapes to represent the board and the pieces, you can optimise the game's performance and make it easier to develop.

When it comes to multiplayer games, simulating game logic on the client-side can be a problem as this leads to many instances of the same game on browsers that use a slow language like Javascript. Plus, it creates difficulties trying to figure out what happens when two identical game states desync and makes it easier to cheat (since they can change their local game state and make themselves win). By simplifying everything and only processing game logic once per board (on the server), you speed up the program, make it easier to develop and decrease the risks of cheating.

Algorithms

Backgammon is a game that contains a random element (the dice) and a completely fixed, deterministic set of rules to figure out the outcome of the game. The dice can be implemented using a pseudo-random number generator. And the fixed set of rules can be implemented using different algorithms, as they are essentially a list of instructions to solve a task. For example, getting mouse input, determining whether a move is legal, determining whether someone has won and sending and receiving requests to the server can all be implemented using algorithms.

Stakeholders

The clients of this software include a wide range of backgammon fans or enthusiasts. As a result, the stakeholders will represent young people looking for an accessible way to get into backgammon. While my solution involves providing a means to play the game online, it does not involve producing a guide to teach people how to play. Therefore, my stakeholders will represent people that have played maybe 1 or 2 games before physically and know the rules of the game.

I feel like an appropriate environment for picking stakeholders that are easily accessible during the development of the solution and have played the game before would be my school. Therefore, my stakeholder will be [redacted]. He is a student who has played the

game before and has organised a local backgammon club at school. He will use the solution in the event that there is a shortage of physical backgammon boards. In such a case, he will use the software to allow everyone to get a chance to play, allowing him to grow his club. This will be appropriate since, at my school, it is likely for every student to have a device, like a Chromebook, on them.

Interview

Interview Plan

For my interview, I will ask a range of questions in order to gauge his background on the subject, his opinion on the proposed solution, any other requirements they would like to add/prefer and how he and other clients will use the solution upon completion. While I may ask him some additional questions during the interview depending on his responses, the main questions I intend to ask [redacted] (representing a student and backgammon enthusiast) are as follows:

- Can you explain your previous experience with the game?
- Have you ever played Backgammon online?
- If so, what was your experience with the software like?
- Is there anything you would add or fix?
- How do you imagine people using the proposed solution?
- Do you think a code-based room system is suitable?
- Is there anything else you want to add?

The first and second questions will establish his background on the subject of backgammon, both in-person and online. The third and fourth questions will establish his opinions on existing solutions and aspects to improve upon. The fifth and sixth questions establish his opinions on the proposed solution, such as how and when people will use it and whether he is ok with a code-based room system. The interview concludes by asking him if he has anything to add.

Interview Transcript

The following is a brief, slightly-edited transcript of the interview which omits filler words, stutters and summarises the key points of the discussion.

Can you explain your previous experience with the game?

> “Since I was very young, I often played backgammon with my grandfather. He taught me how to play. I mostly play with him and I play with my siblings and my parents a bit. Took a bit of a break, but recently got back into it. Overall, I’ve been playing it for many years.”

Have you ever played Backgammon online?

> “Yes I have. Only recently. But I’ve been using 247 Backgammon.”

What was your experience with the software like?

> “It’s decent. Not much to complain about with it. It has good online multiplayer capabilities and features like the doubling dice.”

Is there anything you would add or fix?

> “Not much, maybe graphics but they’re not too important. The important thing is the fact that it's blocked on school wifi and the fact that you can’t run the servers locally.”

How do you imagine people using the proposed solution?

> “I would use it to play online with my friends. Also, it would be very useful to help run Backgammon club, which is my new club which I’ve started very recently, because the school doesn’t have any physical boards. I only have so many physical boards. If I want to make the club bigger, I need an online solution that isn’t blocked on the school wifi. Preferably, I’d prefer something web-based since it would be easiest. You can access it anywhere, from any device, with little to no setup. Also, the majority of players use Chromebooks as their school device. It’s very difficult to download and run stuff on Chromebooks.”

Do you think a code-based room system is suitable?

> “I think it’s suitable. In the club, people would only play with other people nearby or in the same room. So, that system works perfectly.”

Is there anything else you want to add?

> “The doubling dice, maybe. Also, if you’re playing in the same room multiple times, maybe there can be a score tracker to keep track of how many games were won by each player. Also, an account system isn’t too important because it doesn’t really matter how many times you’ve won on one account. It matters how many times you’ve won against one opponent. I don’t mind whether you add an account system.”

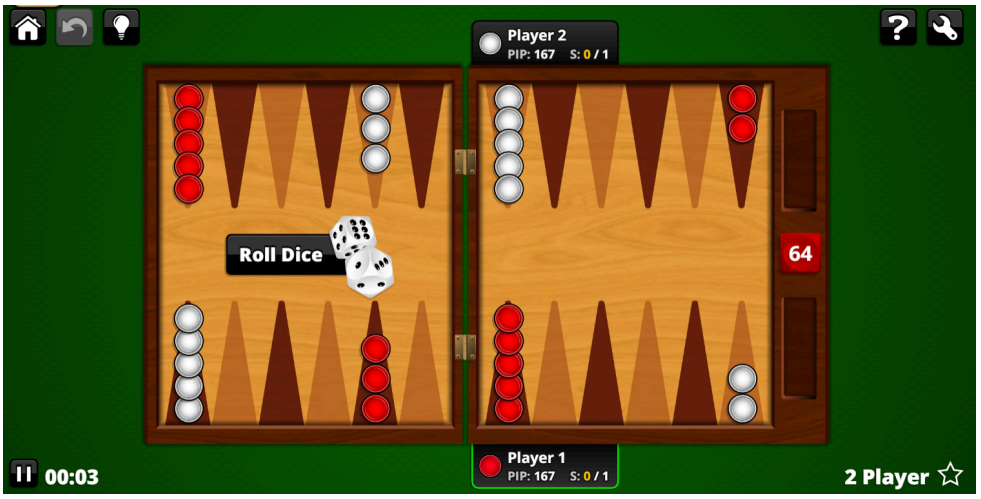
Interview Analysis

The majority of students will struggle to download and run software on their device. Therefore, it is essential that the software is web-based and, at least for the client, is written in a language like Javascript. Also, the software must be relatively simple to use but, given [redacted] has a lot of experience, instructions for playing Backgammon won’t be needed as he can teach anyone new to the game. While I initially thought about abstracting the graphics, the software wouldn’t look good and the majority of devices, including Chromebooks, won’t struggle to render image-based graphics. Thus, the software will use image-based graphics.

I was worried about creating an account-based system since it's quite difficult and time-consuming to implement. However, [redacted] doesn't deem it to be too important, so there's no need to implement it. While I was worrying about the cost of server hosting and a domain, running it locally on a capable machine at school will avoid these costs and avoid the solution being blocked. Finally, I can attempt to implement additional features such as the doubling dice and a score tracker.

Research

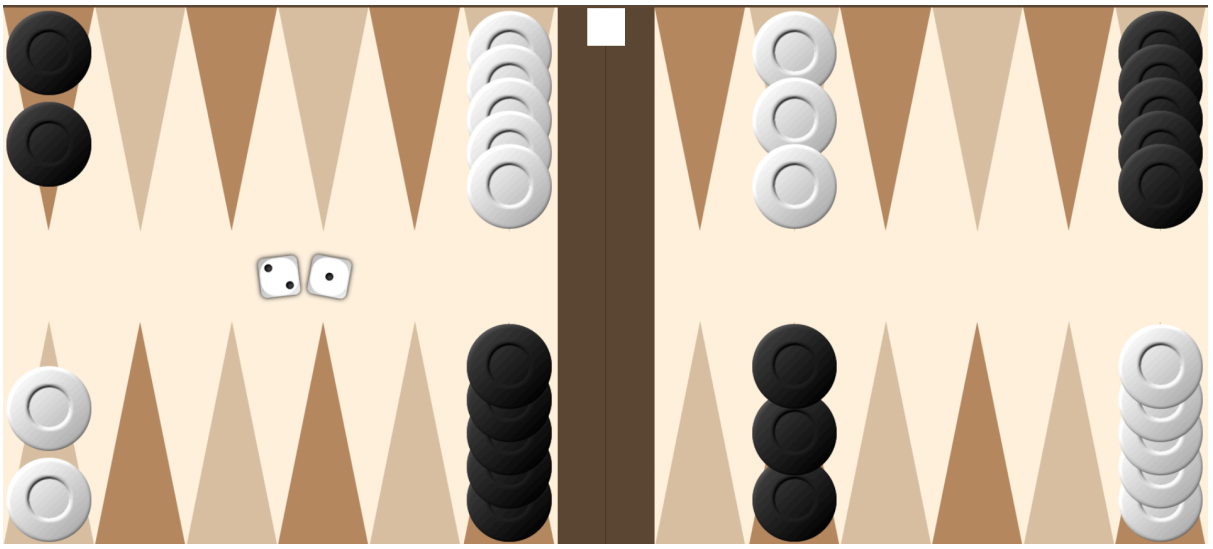
247 Backgammon



247 Backgammon is a web-based backgammon game. Upon going on the website, you are prompted to play online 2-player, local 2-player and against a computer. When you select online 2-player, you will be given a code to share with your friend, who will then join you remotely on their device. Then, using mouse controls, players can interact with multiple elements such as clicking the roll dice button and clicking the pieces. Notably, upon clicking the pieces, each legal move will be clearly visible and highlighted in green. Unfortunately, the game does not include drag and drop functionality, which I found slightly unnatural and inconvenient. However, the UI of the game is very simple, with intuitive icons indicating each button's function, such as returning home, pausing the game, and opening the settings.

On 247 Backgammon, it is incredibly simple to create and join backgammon games remotely. This is something I'd like to implement as it will greatly improve the user experience. Furthermore, once you roll your dice, the game will highlight every legal move that you have. This is especially useful to newer players of the game and, as a result, something that I'd like to implement. The UI is intuitive and I'd like to implement some features, such as the settings, pausing and returning to the main menu, as it improves the functionality of the system. Finally, the game does not support dragging and dropping pieces. I find this unnatural and inconvenient as it does not imitate real life, where you grab and move pieces physically. This is a desirable feature that I'd like to implement.

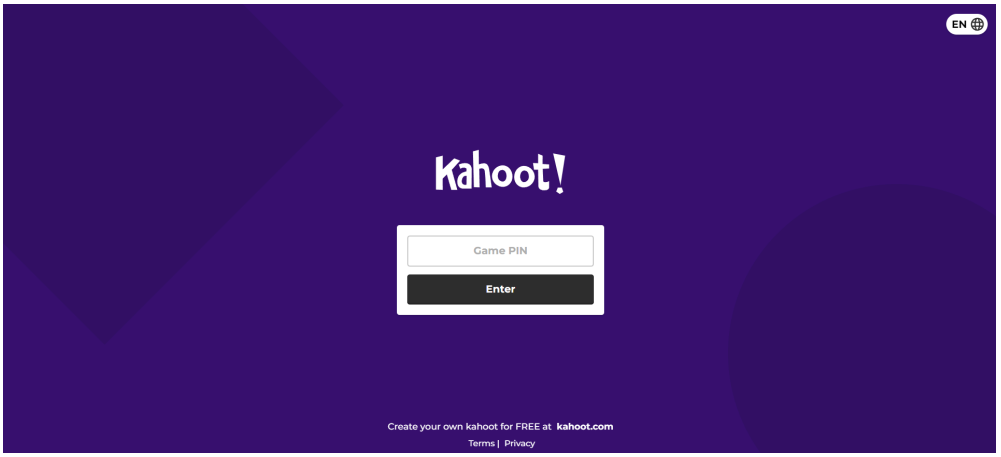
Backgammon js



Backgammon js is an open-source Backgammon game written in Javascript. Unlike other solutions, this solution has not been deployed and isn't accessed using a website. Instead, you clone the GitHub repository, install NodeJS and run the server locally. People on the same network as you can connect via a local IP address. Unlike other solutions, probably because it's open source, this solution seems to be rather underdeveloped with unintuitive controls. There is no code-based room system: you get paired randomly and can't choose your opponent. The UI is unclear, with little instructions as to whose turn it is, which piece colour you are, whether you're meant to roll or play etc. Furthermore, you move pieces by clicking on them. Where they move to is unclear and involves a lot of fiddling in order to move the piece where you want it to go. Finally, basic functionality like the resign button and the menu button aren't implemented and, if they are, I can't figure out how to use them. This solution shows us how many small things we take for granted as a user when using a solution.

This is the only solution which runs a server locally, which is what I want to implement based on the analysis of the interview conducted. However, it is still unintuitive to run, with the requirement of a dependency (NodeJS) in order to run. As a result, I would ideally like to make my server application an executable file in order to simplify the server setup process for the user. Furthermore, unlike this solution, I'd like to implement a code-based room system as, due to the requirements of the solution, it requires the ability for players to play against specific opponents. When it comes to moving the pieces, the same concepts apply from the previous research conducted on 247 Backgammon. Finally, I'd like to implement core functionality like a menu button in order for the user to properly navigate the solution.

Kahoot



Kahoot is a web-based application that allows you to organise an online quiz competition with your friends by giving you points depending on how long it took you to answer the question, if you answered correctly. In order to create an online quiz, you first need an account. Then, you search up a topic on the website and, hopefully, someone's made a multiple-choice quiz on the topic. Then, you select the quiz, configure some settings like the language, order of questions and answers, whether to remove some questions from the quiz etc. and, finally, you click on the create button. It'll give you a room code and, once people type in the room code in a very intuitive and simple web GUI as shown in the image above, they enter their name and join the room. Once enough people have joined, you click the start button and the quiz competition starts.

While Kahoot has almost nothing to do with the game of Backgammon, its code-based room system is very similar to the system which I want to implement in my solution. First of all, it is incredibly simple to join and use. It's web-based, so you don't have to download anything, and it's incredibly easy to use as the UI is very intuitive and minimalistic. As a result, it is something I'd like to implement as it will improve the experience of the user. Furthermore, it is incredibly easy to create a quiz and configure the settings. This is because the UI is intuitive as it's partially designed for classrooms and teachers that may not be very familiar with computers in general. This is something that I'd like to implement as it would widen the range of people that could use the software, allowing more people to get into Backgammon. Unfortunately, Kahoot requires you to create an account to use their service, which can be annoying and discourage some people. As a result, I would like to omit an account system to avoid these problems and also account systems are difficult and time-consuming to implement.

Features of the solution

Using data gathered from the interview and analysed from pre-existing games, I can come up with a list of essential features that my solution must have and a list of desirable features that would be nice for my solution to have.

Feature	Description
Start menu	A menu that the user can interact with upon loading the website. They should be able to enter their name and either create a room or enter a room code and join a room. This is an essential feature as there must be a way to determine what the user wants to do.
Settings	A menu that the user can interact with in order to change any settings they want, such as game audio volume, turning on or off animations or changing the graphics quality of the game. This is a desirable feature as it is not needed to use the solution.
Login system	A system that allows the users to login into their account. From there, they can track and record their games and customise their profile. This is a desirable feature as it isn't needed to use the solution and it's unlikely to be implemented due to time and knowledge constraints.
Network system	A system that allows the users to create backgammon rooms or join existing rooms using codes. From there, the system allows the clients and server to communicate to create an online backgammon experience. This is an essential feature as there must be a way for users to connect to the server and play remotely.
Displaying the board	A way to display the board and all of the pieces on the board. This should display to the screen and also handle events such as when there are too many pieces on a single point. This is an essential feature as there must be a way for the user to see the current state of the board in order to play the game.
Dice rolling	A way for users to, on their turn, roll a set of two dice. The dice should be displayed on the screen and a button should appear allowing the user to roll the dice on their turn. This is an essential feature as the dice and an element of randomness are a key part to the game of Backgammon and must be implemented for a successful backgammon implementation. However, sound effects and animations are desirable as they improve user experience but aren't required for the solution.
Menu button	A button that appears on your screen during gameplay. Upon clicking, it gives you access to many buttons that change the settings, allow you to leave the room, resign etc. I consider this a desirable feature as it is not required to play the game or use my solution, but it will improve customizability and functionality of my solution.
Game board state	There should be a system to track the state of the game board. This allows the program to figure out whether a certain move is legal and allows the server to send and the client to update their local game state. This is an essential feature as it is required to properly

	implement the game of backgammon to know what to display, whether a move is legal etc.
Timer	There should be a timer to track how long a player has to make a move. Especially when it comes to multiplayer turn-based games, imposing a time limit on how long your move takes will limit the time length of the game and also allows the server to determine whether you have disconnected or not (if you don't answer in time, the system will kick you and mark you as disconnected). This is a desirable feature as it isn't required to play backgammon remotely.

Limitations of the solution

Limitation	Description	Justification
Game Assets	These assets determine that look and feel of the game as you play it. It includes the textures of the buttons, the cards, the dice and the board.	I won't be able to implement good-looking, aesthetically-pleasing assets as, in order to do this, I need to be competent in graphic design and software like Photoshop, which I am not. However, I can implement basic-looking self-made assets or assets I download off the internet.
Computer Opponents	Computer opponents would allow for a single-player experience of the game, allowing you to play with the computer if you have no available friends.	I won't be able to implement computer opponents as, in order to do this, I need to know a variety of AI algorithms and the fundamentals of machine learning, which is a lot of technical knowledge which I don't have.
Account System	An account system would allow users to login to the system, creating a way to track their progress, their stats and record their games.	I won't be able to implement an account system as, in order to do this, I need to know how to use SQL databases and learn about securing databases (since I'm probably handling personal data). This is knowledge which I don't have.
Network Connectivity	Network connectivity lays out the fundamental backbone for an online multiplayer system as it allows a means for the client and server to communicate remotely.	I won't be able to implement this at a professional level of quality sufficient for public deployment as, in order to do this, I need access to very powerful server hardware to make network connectivity as smooth as possible. This is hardware I don't have access

		to. However, I can run the server locally on my laptop, which will be enough for my stakeholder's needs of playing remotely with few people at his local school club.

Software and Hardware Requirements

My solution will consist of server-side code (written in Python and compiled into an executable file) and client-side code (written in Javascript and run on a web browser). As the different parts of the code will be run by different people, I have decided to define software and hardware requirements for both the client and the server, grouped in two sections.

Server

Software:

- Windows 7, 10 or 11 (64-bit) - the operating systems that can run a .exe file efficiently since I'm compiling my python code into a .exe file. It must be a 64-bit system since I'm compiling using a 64-bit python interpreter.

Hardware:

- 1.6Ghz processor or faster - to ensure the server runs smoothly.
- 1GB of RAM or more - to ensure the server can handle enough clients.
- 100MB of storage or more - to ensure the server has enough disk space to store the software.
- Standard peripherals (mouse, keyboard, monitor) - to make keyboard and mouse inputs to navigate the OS and download the server files and set them up.

Client

Software:

- Windows, Mac or Linux - the main operating systems supported by most modern browsers.
- Any modern browser that supports Javascript ES6 - to run the client code in the browser i.e. Chrome, Edge, Firefox, Opera etc.

Hardware:

- 1.3Ghz processor or faster - to ensure the browser runs smoothly.
- 4GB of RAM or more - to ensure the browser runs smoothly.
- 100MB of storage or more - to ensure the client has enough disk space to store the browser.
- Standard peripherals (mouse, keyboard, monitor) - to make keyboard and mouse inputs to use the browser by typing in IP addresses, room code, clicking pieces etc.
- Check browser system requirements for specific details - these suggested requirements are based on a Chrome browser.

Solution Requirements

#	Requirement	Explanation
1.1	A menu screen should be displayed.	Allows the user to choose what they want to do with the solution.
1.2	The system should be able to take keyboard input from the user.	Allows the user to type in input and interact with the solution using key presses.
1.3	The system should be able to take mouse input from the user.	Allows the user to use mouse input, specifically clicking and drag and drop, to interact with the solution.
2.1	The user should be able to enter a room code.	If the user can't enter a room code, they won't be able to choose a specific game of backgammon to play.
2.2	The user should be able to enter a username.	In order for your opponent to know your username, you must first send your username to the server.
2.3	The user should be able to create a room.	If the user can't create rooms, no one will be able to join rooms and remote backgammon games won't be possible.
2.4	The user should be able to load into a backgammon room.	If this isn't possible, it won't be possible to play backgammon remotely as neither opponent can load into the game.
2.5	The user should be able to roll the dice on their turn.	To play backgammon, there must be an element of randomness, traditionally using two dice.
2.6	The user should be able to move pieces.	To play backgammon, the player must be able to move pieces legally based on the rules of the game and the dice number that they rolled.
2.7	Game logic should happen on the server.	In order to prevent cheating, game logic should be held on the server to prevent the user from modifying the game logic locally to make themselves win.
2.8	The user should have the option to rematch the opponent at the end of the game.	It would be unintuitive if a user wanted to play the same opponent repeatedly but had to keep creating new rooms to do so.
3.1	The user should be able to view the backgammon board.	To play backgammon, the viewer must be able to see the board.

3.2	The user should be able to hear some music or sound effects.	To improve the user experience and the atmosphere of the game.
3.3	The graphics/audio should be customisable in a settings menu.	To ensure the user can modify the game to run smoothly on their machine or customise the game to their preference.
3.4	The user should be able to run the game at 15 fps minimum, preferably 30 fps.	To ensure the user has a smooth and responsive experience when playing the game.

Success Criteria

#	Testing	Desirable	Acceptable	Failure
1.1	The stakeholder will join the server IP address on their browser and check if they load into and can navigate the menu.	A nice, minimalistic menu that provides all features and options.	A simple menu that provides the barebones features (create game, join game).	No menu is loaded.
1.2	The program should log all keyboard input to the console.	Keyboard inputs are registered, logged and perform appropriate actions.		No keyboard input is registered.
1.3	The program should log all mouse input to the console.	Mouse inputs are registered, logged and perform appropriate actions.		No mouse input is registered.
2.1	The stakeholder will click on the room code input and type in a code.	Only numbers are taken in as input and displayed.	Any key input is taken in as input and displayed.	No input is taken in and nothing is displayed.
2.2	The stakeholder will click on the username input and type in a username.	Any key input is taken in as input with a fixed character limit and displayed.	Any key input is taken in as input and displayed.	No input is taken in and nothing is displayed.
2.3	The stakeholder will click on the button to create a room.	A valid room code is displayed and a room is created		Nothing gets displayed and nothing is created.
2.4	The program should log when a successful room	A successful room connection is made, the connection is logged		Nothing connects

	connection is made.	and the user is remotely connected to the room.		successfully.
2.5	The stakeholder, on their turn, should click on the roll button and check that dice are rolled.	The dice are rolled with an animation upon clicking the button.	The dice are rolled upon clicking the button.	Nothing happens when clicking the roll button.
2.6	The stakeholder should move their pieces many times (throughout a game) by clicking and drag and dropping.	The pieces move from one point to another with a smooth animation.	The pieces move from one point to another.	Nothing happens when moving the pieces.
2.7	Every update to the server-side board should be validated, logged then sent back to all clients in the room.	Updates are successfully validated, logged and sent back to all clients.		No server-side updates happen.
2.8	Upon the completion of a game, the stakeholder should click the rematch button.	A rematch request is successfully sent to the opponent and, if they accept, the backgammon board is reset.		Nothing happens when trying to rematch.
3.1	Thorough testing to ensure that there are no visual bugs and the board is properly displayed.	The board is properly displayed with no visual bugs throughout the game.		The board isn't displayed.
3.2	Thorough testing to ensure that audio can be clearly heard without any bugs.	The audio can be clearly heard with no bugs throughout the game.		Audio isn't working.
3.3	The stakeholder can customise the graphics/audio in a menu.	Customisation works and changes take immediate effect.		The customisation doesn't work.
3.4	The frame rate is continuously logged throughout using the solution on a lower-end device.	The game comfortably runs above 30 fps.	The game runs at around 15 fps.	The game runs below 15 fps.

Design

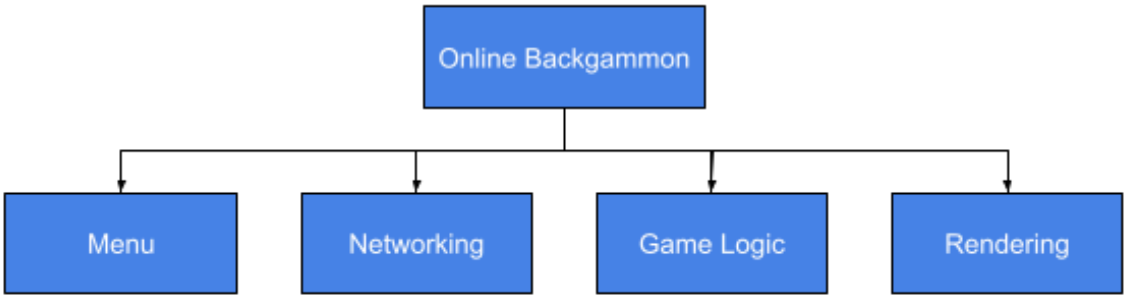
Decomposition

The overall problem that I'm trying to solve is the remote play of Backgammon. This can be represented as:

Connect to a game remotely → Handle Backgammon game logic

This is a very simplistic idea of the process and can be further analysed and broken down using decomposition. As the problem is very big, I have decided to break it down into further submodules as this process will make the development of the solution much easier in the future. The submodules that I will implement are:

- Menu - to do with the GUI of the game.
- Networking - to do with the multiplayer aspect of the game.
- Game logic - to do with the mechanics of the game.
- Rendering - to do with the textures that make up the game and how they're displayed.

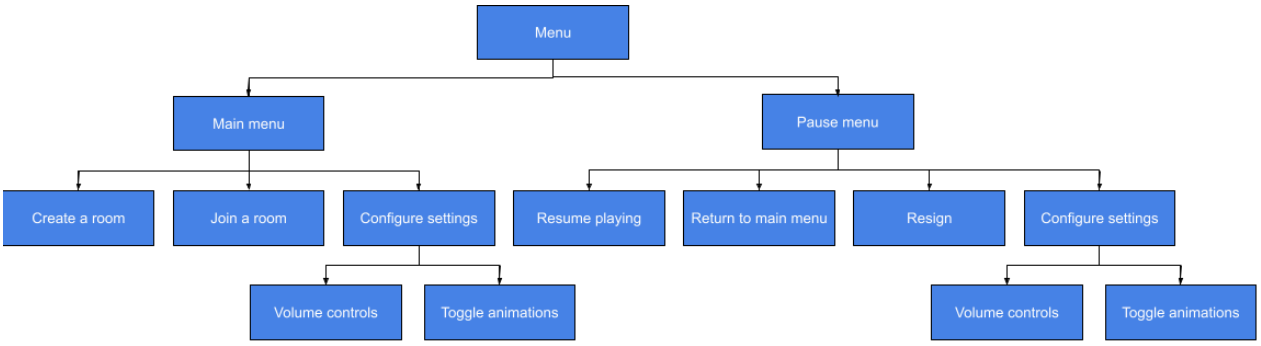


Below, I have broken down the main submodules down further and created top down modular designs for each submodule. Any text in blue is used to justify the existence of each submodule and why I broke down the problem in this way.

Menu

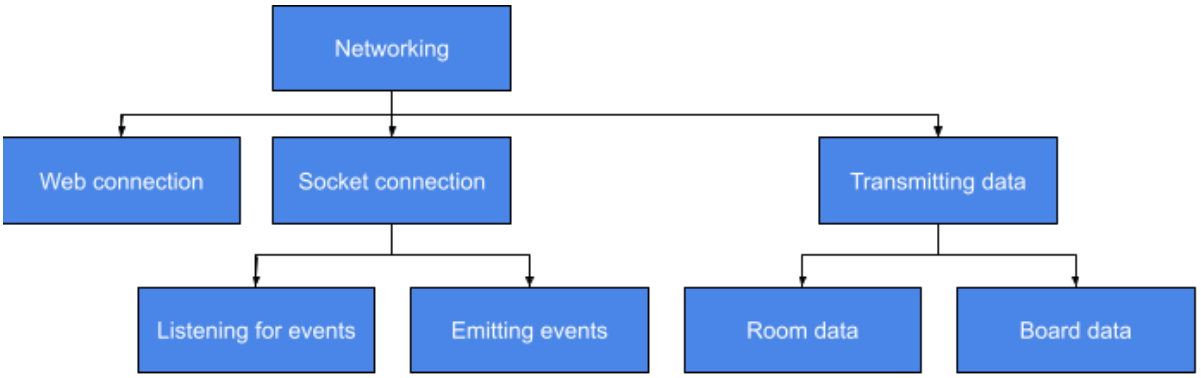
- The main menu is the menu that opens initially and must allow the player to…
  - Create a room
  - Join a room
  - This provides a way to choose who the user plays against remotely
  - Configure their settings
    - Volume settings
    - Enable/disable animations

    - Makes the program more customizable
- The pause menu is the menu that appears during the game and must allow the player to…
    - Configure their settings
        - Volume settings
        - Enable/disable animations
    - Makes the program more customizable
    - Resume playing
    - Return to the main menu
    - Resign
    - Key functionality of a lot of strategic online games
- The menus are required to allow the user to use the program and navigate through the different options and perform different actions depending on the situation.



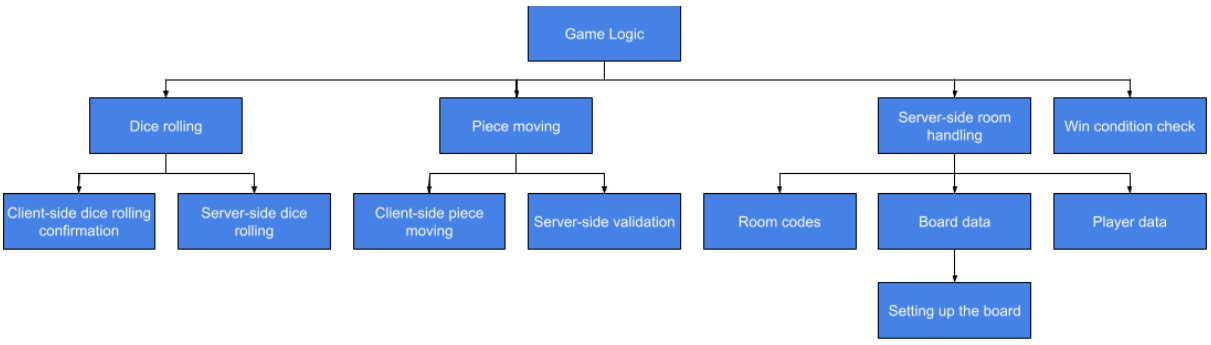
Networking

- Web connection
- Allows the user to access the program remotely from their browser
- Socket connection
    - Listening for events
    - Emitting events
- Allows quick event transmission between the server and client
- Allows the client to receive data and update without refreshing the web page
- Transmitting data
    - Room data
    - Board data
- Ensures that room data and board data can be sent between client and server, allowing for remote play



Game Logic

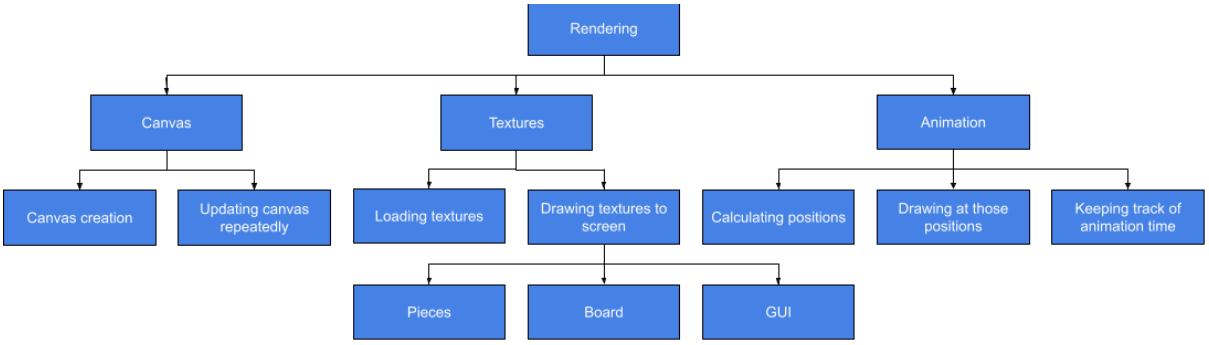
- Dice rolling
    - Client-side dice rolling confirmation
    - Server-side dice rolling
- Essential part of playing the game
- Piece moving
    - Client-side piece moving
    - Server-side validation
- Essential part of playing the game
- Server-side room handling
    - Room codes
    - Board data
        - Setting up the board
    - Player data
- The server must be able to handle multiple instances of Backgammon at the same time. This includes handling multiple room codes, multiple boards and multiple players simultaneously.
- Win condition check
- Allows the game to have a winner and loser
- Also allows the game to end



Rendering

- Canvas

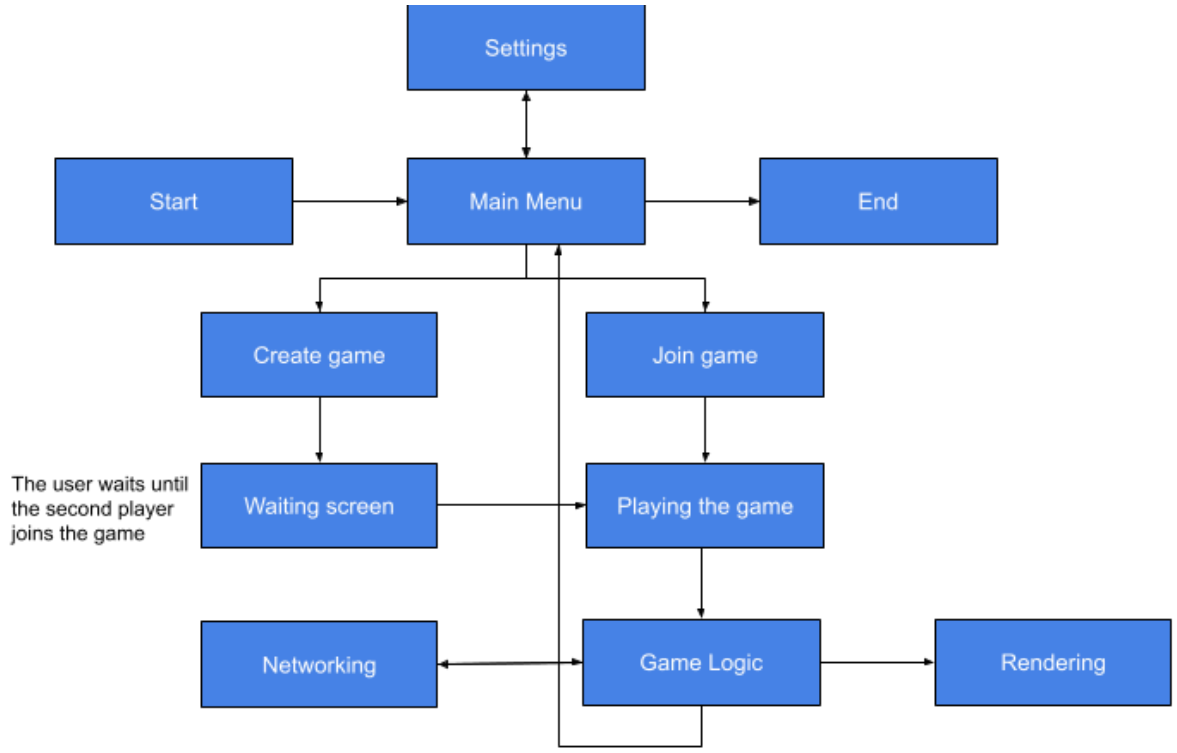
    - Canvas creation
    - Updating canvas repeatedly
- Provides a way to display a GUI in pure Javascript
- Textures
    - Loading textures
    - Drawing textures on the screen
        - Pieces
        - Board
        - GUI
- Required so the user can see the menu and the game on the screen
- Animation
    - Calculating positions
    - Drawing at those positions
    - Keeping track of animation time
- Nice to have to make the game feel more fluid and smoother



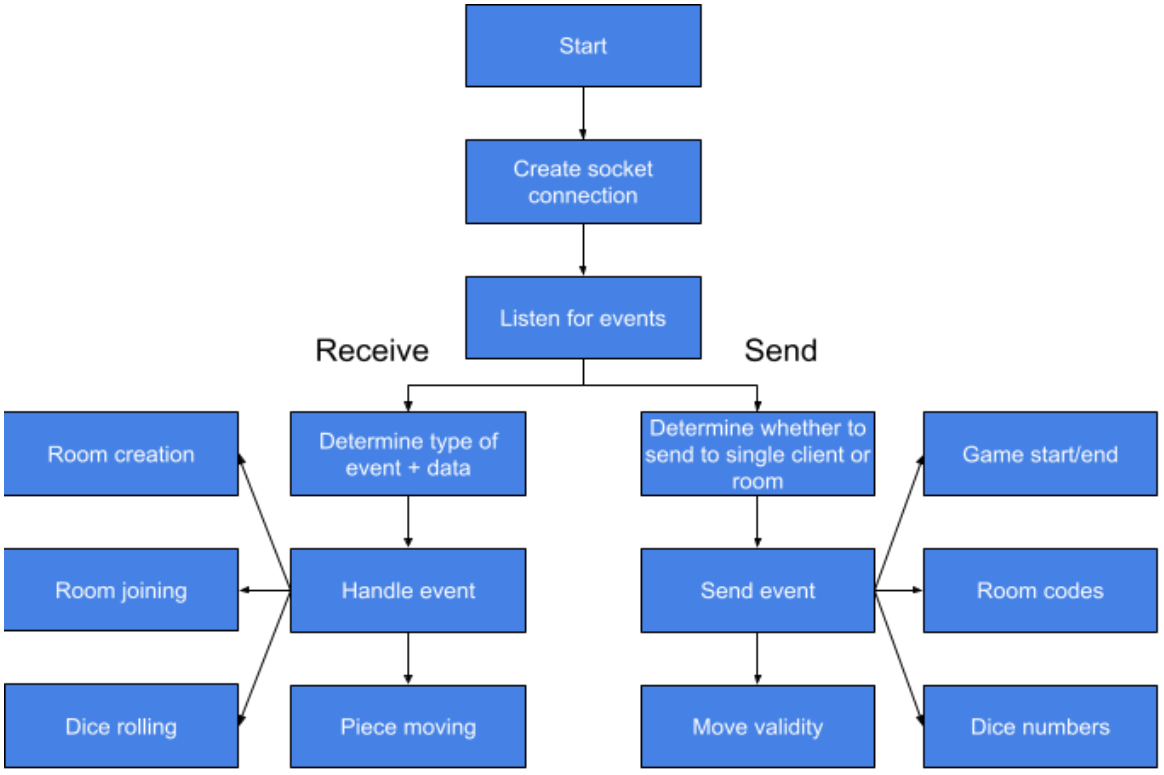
Program Structure

In order to make the development process much easier, I have decided to plan out how each of the modules and some submodules will interact with each other. Since I am designing a multiplayer game with both client code and server code, I have decided to create two diagrams, one for the client's program structure and another for the server's program structure.

Client Program Structure



Server Program Structure

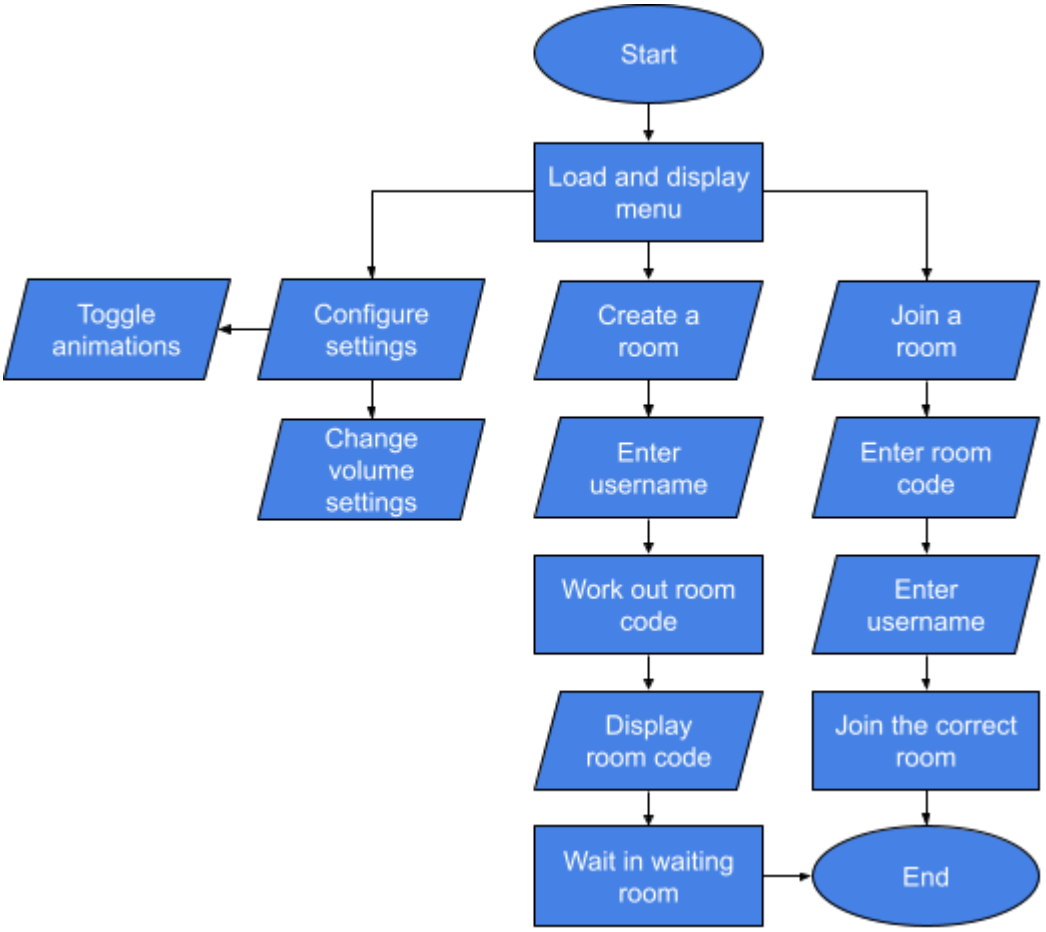


Algorithms, Variables and Iterative Testing

Menu Module

Main Menu

Flowchart



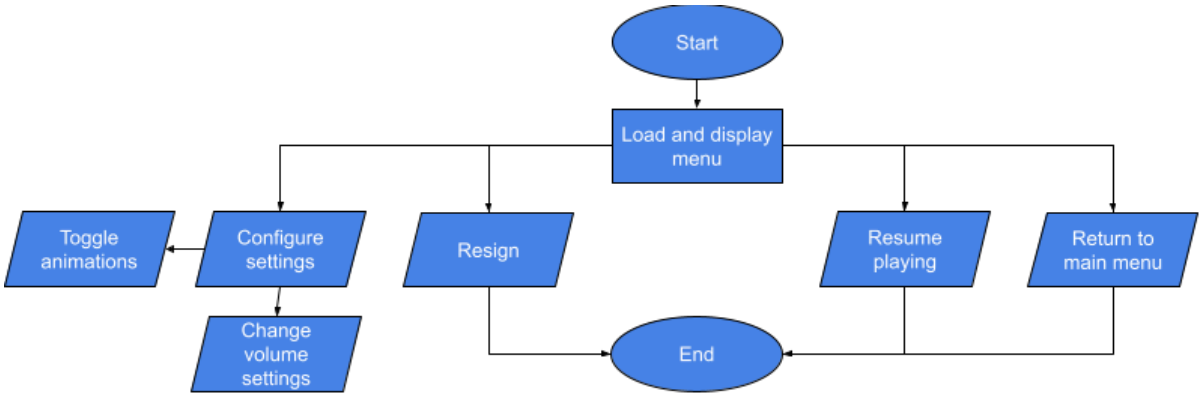
Pseudocode

```
display(mainMenu)
choice = USERINPUT
IF (choice == roomCreation) THEN
      username = USERINPUT
      roomCode = getRoomCode()
      display(roomCode)
      waitInRoom()
ELIF (choice == roomJoin) THEN
      roomCode = USERINPUT
```

```
      username = USERINPUT
      joinRoom(roomCode)
ELIF (choice == configureSettings) THEN
      settingsChoice = USERINPUT
      IF (settingsChoice == animation) THEN
            toggleAnimation = USERINPUT
      ELIF (settingsChoice == volume) THEN
            volume = USERINPUT
      ENDIF
ENDIF
```

Pause Menu

Flowchart



Pseudocode

```
display(pauseMenu)
choice = USERINPUT
IF (choice == resumePlaying) THEN
      resumePlaying()
ELIF (choice == returnToMainMenu) THEN
      returnToMainMenu()
ELIF (choice == resign) THEN
      resign()
ELIF (choice == configureSettings) THEN
      settingsChoice = USERINPUT
      IF (settingsChoice == animation) THEN
            toggleAnimation = USERINPUT
      ELIF (settingsChoice == volume) THEN
            volume = USERINPUT
      ENDIF
ENDIF
```

Justification

A menu system is required (Req 1.1) to allow the user to navigate the program, select what they do and fully utilise the solution.

On the main menu, the create room button is essential to creating a Backgammon game (Req 2.3) that someone else can join remotely, which is essential in playing the game remotely. It must enable them to enter their username (Req 2.2) (to change how the other player sees your name) and then display a room code to allow another person to join. The join room button is also essential to joining a Backgammon game (Req 2.4) in order to play the game remotely. It allows them to choose who they play against by entering a room code (Req 2.1). It also enables them to enter their username (Req 2.2). Note that a quit button on the menu is not essential and is not a feature of this solution as the user can just close the tab on their web browser.

On the pause menu, the resume playing button is essential to allow the user to exit the pause menu and go back to playing the game (Req 3.1). Returning to the main menu is also essential as it provides the user a way to leave the room and the Backgammon game if they ever want to stop playing or start a new game. Resigning is an desirable feature as it provides the user a way to intentionally forfeit and end the game, which is a common feature for most 2 player strategy games. However, it is not required to implement a fully-functioning solution.

When configuring settings, toggling animation is desirable (Req 3.3) as it only improves the performance of the solution on lower-end devices. Volume control (Req 3.3) is also desirable as it only improves the auditory experience of the user in an easy-to-use way.

Data Structures

Name	Type	Description	Justification
username	String	The name that is displayed to the opponent.	The opponent needs to know who they're playing.
roomCode	Integer	Determines which room the server puts the user in.	The server needs to know which room to put the user in.
toggleAnimation	Boolean	A toggle that either enables or disables animations.	Lets the user change how the pieces move depending on their preference.
volume	Integer	Volume control for SFX.	Allows the user to customise the volume to their preference.

Validation

Validation is not required for volume or toggleAnimation as these values are changed via buttons or sliders. However, it will be required for the username, to ensure only alphanumeric characters are entered, and the roomCode, to ensure only numbers are entered. The alphanumeric validation is required as the server or a user browser may not understand non-alphanumeric characters and could crash or not render those characters. Number validation is required as room codes generated by the server are always going to be integers and allowing the user to enter other characters may lead to errors.

Validation Algorithms

```
// USERNAME VALIDATION
username = “”
WHILE (username == “” OR usernameCharacters NOT IN
alphanumericCharacters) DO
      username = USERINPUT
ENDWHILE

// ROOM CODE VALIDATION
roomCode = “”
WHILE (roomCode == “” OR roomCodeCharacters NOT IN
numericCharacters OR roomCodeCharacters < 100000 OR
roomCodeCharacters > 999999) DO
      roomCode = USERINPUT
ENDWHILE
```

Iterative testing

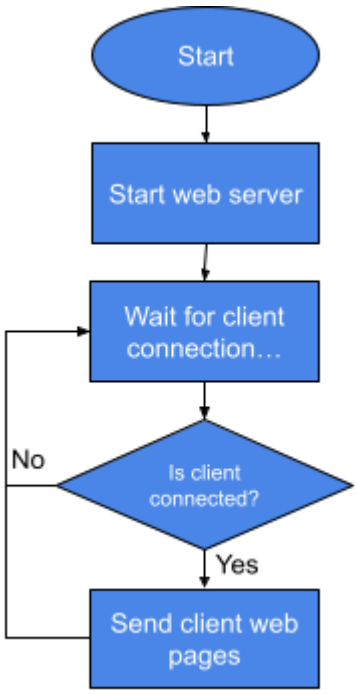
Function	Justification	Req. tested	Input	Expected result
Volume control	Making sure the audio sliders so the user can change the volume.	1.3, 3.3	Sliding to 0%, 50% and 100%.	The volume should be mute, then 50% then 100%.
Toggling animation	Making sure the animation toggle switch turns on and off the animation.	1.3, 3.2, 3.3	Toggling animation off and on.	Animations should be disabled then enabled.
Creating a room	Making sure a room can be created, which creates a joinable Backgammon game.	1.3, 2.3	Clicking the “create a room” button and typing in a username	A room should be created and a room code should be displayed.
Joining a room	Making sure a room	1.3, 2.4	Clicking the	The player

	can be joined, which allows players to join the same room and play.		“join a room” button, typing in a room code and a username	should load into the correct room and a game should start.
Entering a username	Making sure only alphanumeric characters can be typed, which prevents any errors.	1.2, 2.2	Typing in a username with a mix of letters, numbers and symbols.	Only letters and numbers should be recognised by the program and entered.
Entering a room code	Making sure only numbers can be typed, which prevents any errors.	1.2, 2.1	Typing in a room code with a mix of letters, numbers and symbols.	Only numbers should be recognised by the program and entered.

Networking Module

Web Server

Flowchart

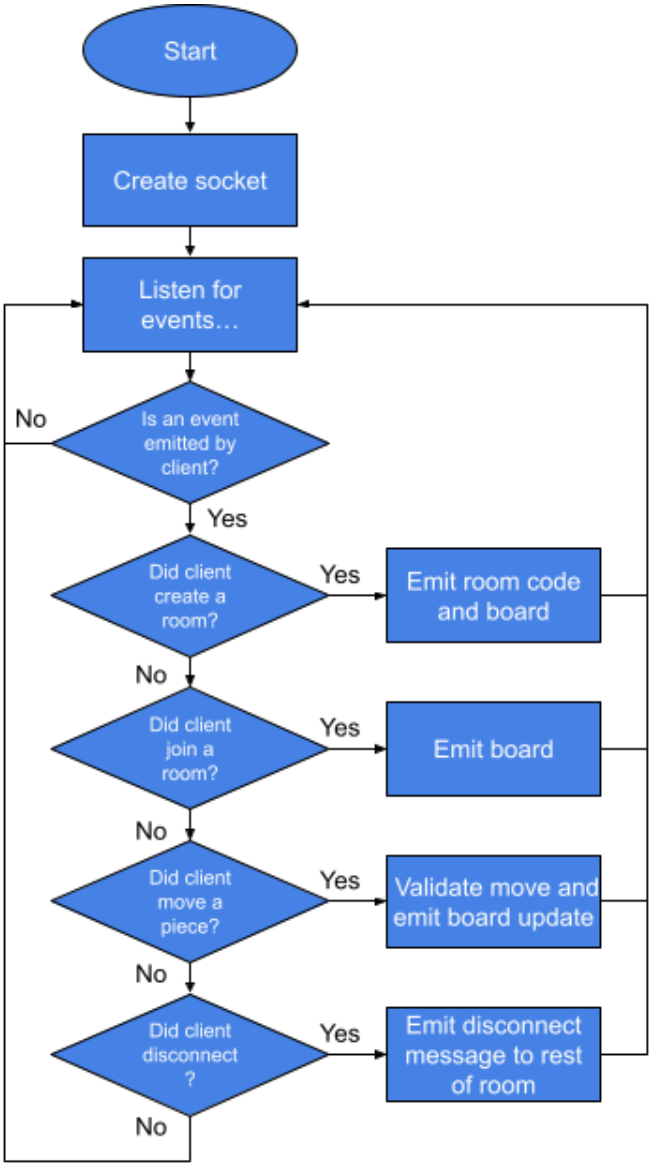


Pseudocode

```
server = Server()
server.run(host=0.0.0.0, port=80)
WHILE True DO
      waitForConnection()
      IF (connected == True) THEN
            sendWebPages()
      ENDIF
ENDWHILE
```

Server-side socket connection

Flowchart

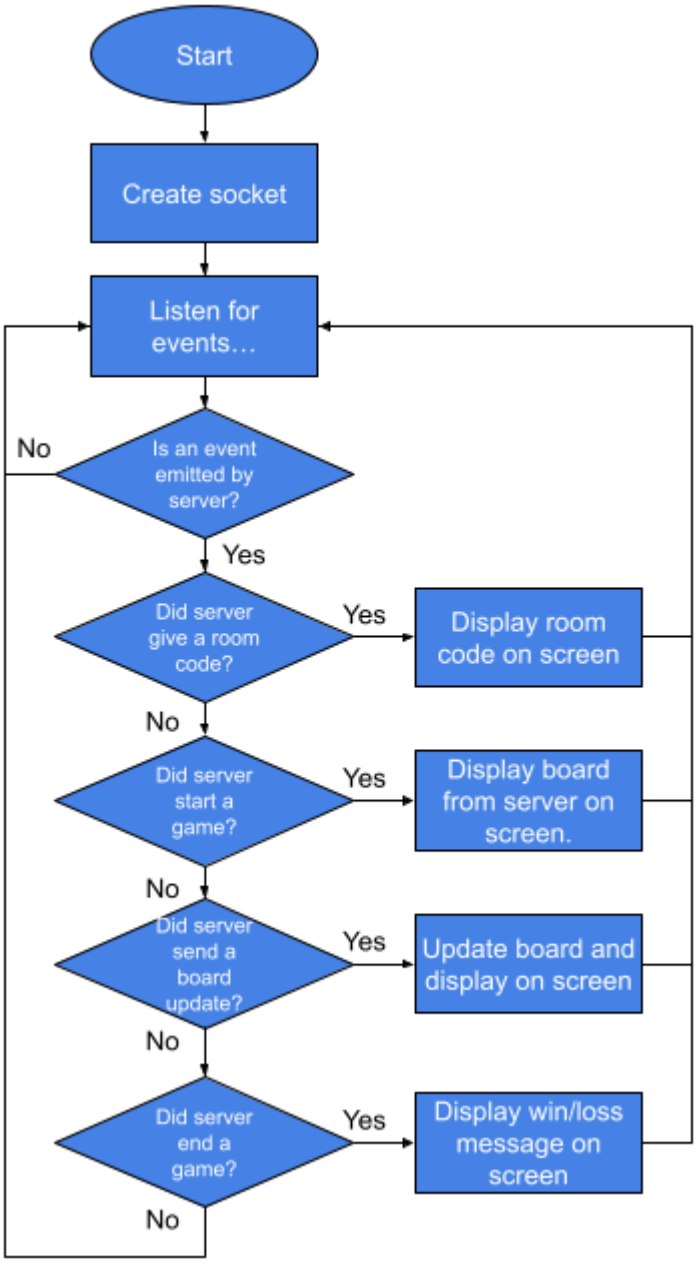


Pseudocode

```
socket = Socket()
socket.run(host=0.0.0.0, port=80)
WHILE True DO
      listenForEvents()
      IF (eventExists == True) THEN
            IF (eventType == roomCreation) THEN
                  roomCode = generateRoomCode()
                  board = setUpBoard()
                  emit(roomCode, board)
            ELIF (eventType == roomJoin) THEN
                  board = getBoard()
                  emit(board)
            ELIF (eventType == pieceMove) THEN
                  move = getMove()
                  IF (validate(move) == True) THEN
                        updateBoard(move)
                        board = getBoard()
                        emitAll(board)
                  ENDIF
            ELIF (eventType == disconnect) THEN
                  emitAll(disconnectMessage)
            ENDIF
      ENDIF
ENDWHILE
```

Client-side socket connection

Flowchart



Pseudocode

```
socket = Socket()
socket.connect()
WHILE True DO
      listenForEvents()
      IF (eventExists == True) THEN
```

```
            IF (eventRoomCode exists) THEN
                  display(eventRoomCode)
            ELIF (eventType == start) THEN
                  display(eventBoard)
            ELIF (eventType == boardUpdate) THEN
                  display(eventBoard)
            ELIF (eventType == end) THEN
                  IF (eventWin == True) THEN
                        display(winMessage)
                  ELSE
                        display(loseMessage)
                  ENDIF
            ENDIF
      ENDIF
ENDWHILE
```

Do note that, due to the fact that I am using Flask, a Python module, to create a web server, it has inbuilt functions for starting a web server, starting a socket, listening and emitting events etc. Therefore, I do not need to create additional algorithms for these events.

Justification

A web server is required to allow the user to connect and use the solution in their web browser. This allows them to see the menu and the board on their web browser. (Req 1.1, 3.1). Furthermore, sending web pages to the client is required to receive the client-side Javascript code and the HTML of the menu from the server, allowing them to use the solution.

A server-side socket connection is required to allow the server to listen for events from the client and emit events to the client. This allows the server to handle many aspects of remote Backgammon, including creating a room, joining a room and handling game logic on the server (Req 2.3, 2.4, 2.7) which is necessary for remote Backgammon play.

A client-side socket connection is required to allow the client to listen for events from the server and emit events to the server. This allows the client to inform the server when they roll the dice and make a move (Req 2.5, 2.6) (which is part of the game of Backgammon) as well as listen for when a game starts, ends or updates. This allows the client to join, leave and display an updated Backgammon board (Req 2.4, 3.1) which is necessary to use the solution.

Data Structures

Name	Type	Description	Justification
host	String	The IP that the server and socket is hosted on.	The server and socket needs an IP to run on so

			clients can connect to them via their IP.
port	Integer	The port the server and socket runs on.	The server and socket needs a port in order to run as it needs a port to send and receive signals from.
eventType	String	A name for the type of event that a client/server listens to..	Each event must have a type so the client/server knows how to deal and handle it, depending on what it is.
eventData	Array	Stores the transmitted data that comes from listening to events.	Each event must have a data variable so that data can be transmitted between the client and the server i.e. board data, room data, player data etc.

Validation

Validation is not required for any of these data structures as these are all controlled by the program and not inputted from the player. Therefore, there won't be a need to validate the data given everything is implemented correctly.

Iterative testing

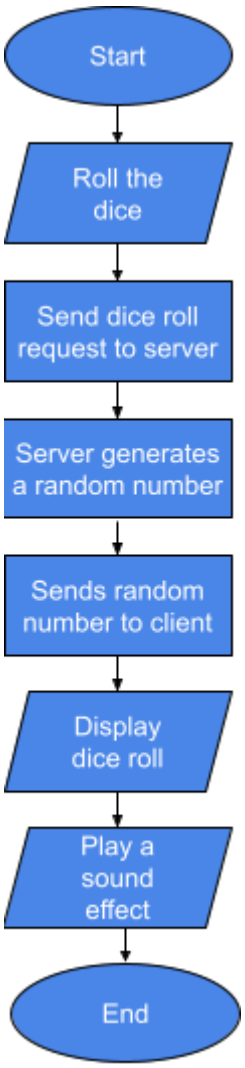
Function	Justification	Req. tested	Input	Expected result
Web server	Making sure the web server is running and the client can connect to it and receive web pages, thus viewing the game.	1.1, 3.1	Enter the server IP in the web browser and connect to the server.	A menu should be displayed on the web browser.
Emitting an event via the socket	Making sure the socket is capable of emitting events and talking remotely to the client/server.	2.7	Click on the "create a room" button.	A room should be created if the event was emitted properly.
Listening for an event via the socket	Making sure the socket is capable of listening for events broadcast remotely from the client/server.	2.7	Clicking the "create a room" button.	A room code should be displayed if the client listened to the roomCode

				event.

Game Logic Module

Dice rolling

Flowchart



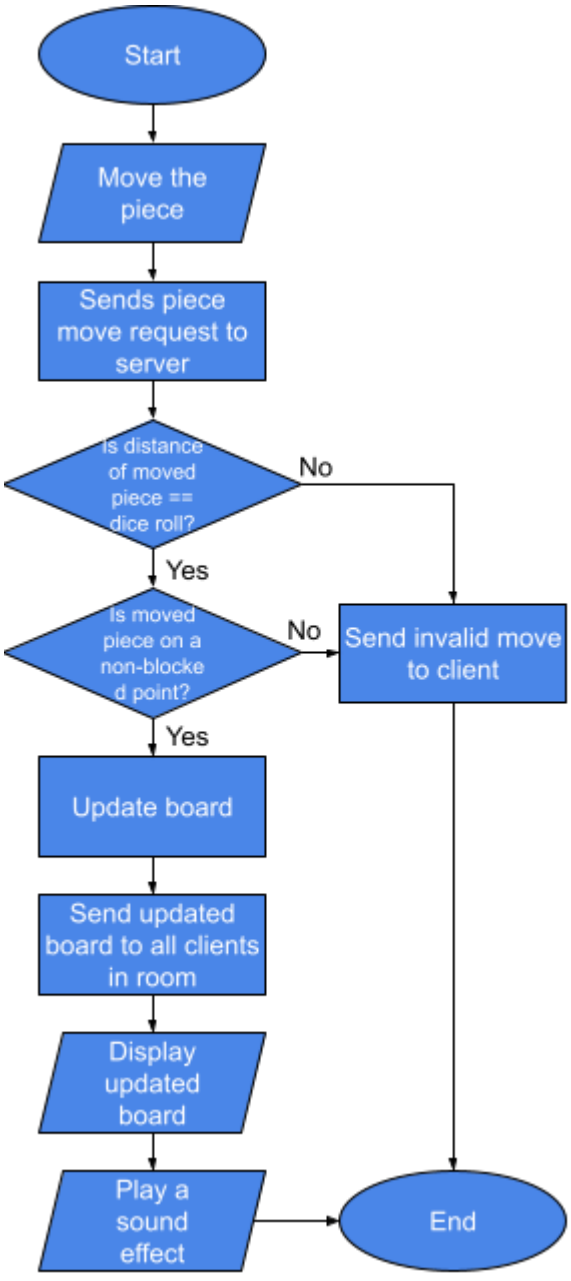
Pseudocode

```
// CLIENT SIDE
IF (action == diceRoll) THEN
      sendDiceRollRequest()
      // SERVER SIDE
      dice = RANDOM(1, 6)
```

```
      sendToClient(dice)
      // CLIENT SIDE
      display(dice)
      playsfx(diceRoll)
ENDIF
```

Piece moving

Flowchart

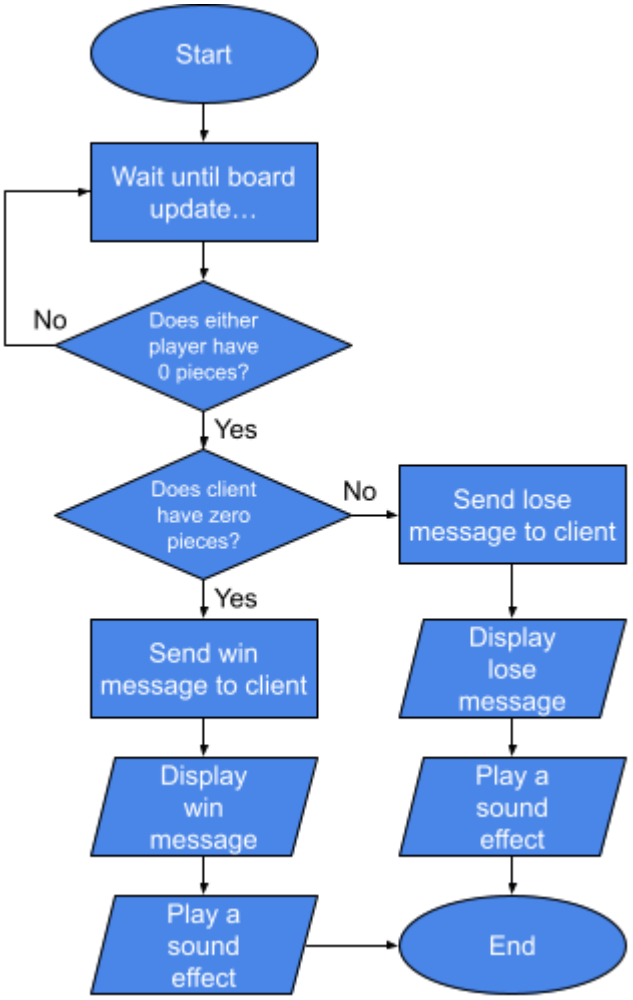


Pseudocode

```
// CLIENT SIDE
IF (action == pieceMove) THEN
      sendPieceMoveRequest()
      // SERVER SIDE
      IF (distance == dice) THEN
            IF (piecesOnPoint <= 2) THEN
                  updatedBoard = updateBoard()
                  sendToClient(updatedBoard)
                  // CLIENT SIDE
                  display(updatedBoard)
                  playsfx(pieceMove)
            ENDIF
      ENDIF
ENDIF
```

Win condition check

Flowchart



Pseudocode

```
// SERVER SIDE
WHILE True DO
      waitForBoardUpdate()
      IF (player1Pieces == 0 OR player2Pieces == 0) THEN
            IF (clientPieces == 0) THEN
                  sendWinMessage()
                  // CLIENT SIDE
                  display(winMessage)
                  playsfx(win)
            // SERVER SIDE
            ELSE
                  sendLoseMessage()
```

```
                // CLIENT SIDE
                display(loseMessage)
                playsfx(lose)
            ENDIF
        ENDIF
ENDWHILE
```

Due to the fact that I'm coding in Javascript, it offers an inbuilt method to listen to key and mouse presses. As a result, I don't need to directly code these algorithms to meet (Req 1.2, 1.3).

Justification

The dice rolling algorithm (initialised by the mouse (Req 1.3) is required (Req 2.5) as rolling dice is needed to play Backgammon in real life and remotely. This allows the player to determine how many spaces to move their pieces. The server-side validation in the dice rolling algorithm is required (Req 2.7), if it wasn't implemented, the client could locally modify the dice variable and cheat. When rolling the dice, a sound effect should be played (Req 3.2) to improve the experience of the game.

The piece moving algorithm (initialised by the mouse (Req 1.3)) is required (Req 2.6) as piece moving is needed to play the game remotely. The server-side validation is required (Req 2.7) as it prevents the player from locally making an illegal move and cheating. When moving the pieces, a sound effect should be played (Req 3.2) to improve the experience of the game.

The win condition check is required as, without it, the game would never end. This is a required function to play the game remotely. The check is nearly all on the server for server-side validation (Req 2.7) as it prevents the player from locally making them win and cheating. After winning, the players should have the option to rematch (Req 2.8) to improve the functionality of the program and make it easier to start new games.

Data Structures

Name	Type	Description	Justification
diceNumber	integer	The number on the dice when it`s been rolled.	A dice number is required for the outcome of dice rolling, an essential part of the game.
pieceMoveDistance	Integer	The distance the piece moved.	Required for the server to check if it's equal to the dice roll to check if the move is valid or not.
piecesOnPoint	Integer	The number of existing	Required so the point a

		pieces on the point a piece moves to.	piece moves to is valid.
board	Array	A list containing all of the pieces on the board and their positions.	Required so the server can create and update it to send to all clients. Allows for piece moving.
clientPieces	Integer	The number of pieces the client has left on the board.	Required for the server to check if the client has won the game or not.

Validation

Validation is not required for any of these data structures as these are all controlled by the program and not inputted from the player. Therefore, there won't be a need to validate the data given everything is implemented correctly.

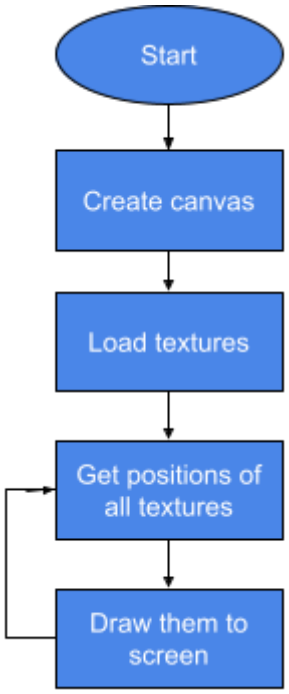
Iterative testing

Function	Justification	Req. tested	Input	Expected result
Dice Rolling	Making sure the user can roll the dice in order to play the game.	1.3, 2.5, 2.7	Click on the "roll dice" button on your turn.	Two dice should display a number between 2 and 12.
Piece moving	Making sure the user can move a piece in order to play the game.	1.3, 2.6, 2.7	Click on a piece then click on the point to move it to.	If it's a valid move, the piece should move. Otherwise, the piece shouldn't move.
Win condition checking	Making sure the server is capable of recognising when a player has won or not in order to end the game.	2.7, 2.8	Win the game by getting rid of all of your pieces.	A win message should be displayed and the option to rematch should be present.

Rendering Module

Drawing textures

Flowchart

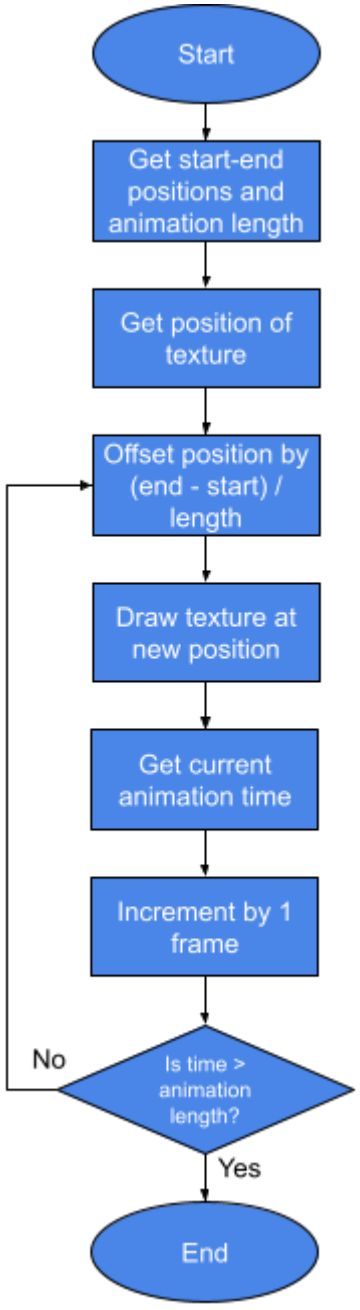


Pseudocode

```
canvas = createCanvas()
textures = loadTextures(canvas)
WHILE True DO
      positions = getTexturePositions()
      FOR texture, position IN textures, positions DO
            display(texture, position)
      ENDFOR
ENDWHILE
```

Animation

Flowchart



Pseudocode

```
texture = getTexture()
startPosition = getStartPosition()
endPosition = getEndPosition()
length = getAnimationLength()
```

```
time = getAnimationTime()
WHILE time < length DO
      position = getPosition()
      position += (endPosition - startPosition) / length
      display(texture, position)
      time += 1
ENDWHILE
```

Due to the fact that I'm coding in Javascript, I have access to an inbuilt function that creates a canvas and repeatedly updates it. I also have access to an inbuilt function that loads image textures and plays sound effects. As a result, I can fulfil requirements (Req 1.1, 3.1, 3.2) without having to directly code these algorithms.

Justification

Drawing textures to the screen is required for the user to view and use the solution (Req 1.1, 3.1). The textures should be redrawn continuously at 30fps (Req 3.4) in order to create a visually smooth gameplay experience. The program should have the ability to play sound effects (Req 3.2) to enhance the user experience.

Animation is desirable as it helps improve the visual aspect of the program. It would justify the setting of "toggle animation" (Req 3.3) and improve everything that is drawn to the screen. For example, the piece movement and dice rolling would be more visually pleasing to look at (Req 2.6, 2.5). Finally, animation justifies the frame rate of running at 30fps (Req 3.4) as, without animation, running a high frame rate would be a waste of performance.

Data Structures

Name	Type	Description	Justification
canvas	Class	A class which displays the objects drawn onto it.	Required to create any GUI element in pure Javascript.
textures	Array	A list of image textures, including the board and piece textures.	Required so the program knows what to display on screen.
positions	Array	A list of positions of image textures.	Required so the program knows where to display the textures on screen.
fps	Integer	An integer that dictates the target fps of the game.	Required so the program knows how frequently it runs the game loop and updates the screen.
animationLength	Integer	The number of frames an animation lasts.	Required to know how fast an object is animated

			between two points.

Validation

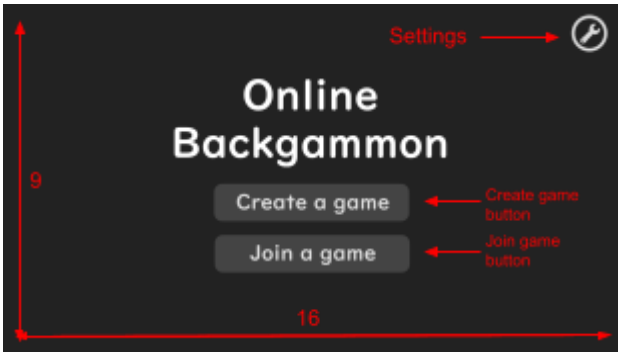
Validation is not required for any of these data structures as these are all controlled by the program and not inputted from the player. Therefore, there won't be a need to validate the data given everything is implemented correctly.

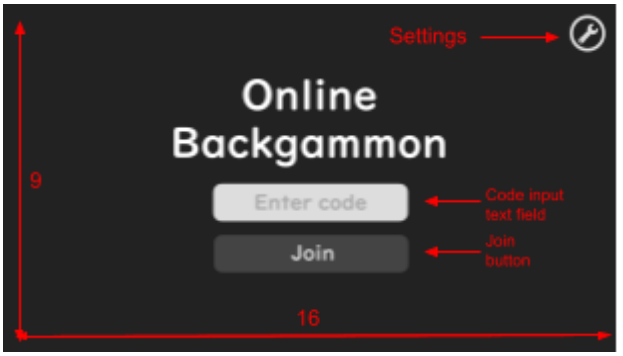
Iterative testing

Function	Justification	Req. tested	Input	Expected result
Displaying a GUI	Making sure the user can view a visually smooth GUI at all times regardless of what they're doing to interact with elements like the game and the screen.	1.1, 3.1, 3.4	Opening the program.	A menu should be displayed and should be able to be navigated.
Playing SFX	Making sure SFX can be played when performing actions to improve user experience.	3.2	Move a piece during gameplay.	A piece move sound effect should be played.
Animation	Desirable to make sure piece movements and other aspects are fluid and smooth during gameplay.	2.5, 2.6, 3.4	Move a piece during gameplay.	The piece should move fluidly between two points.

Usability Design

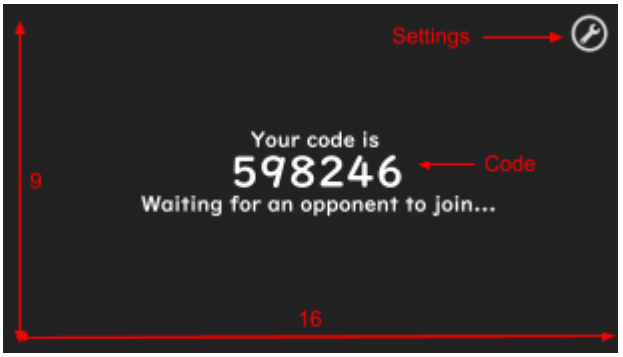
Main Menu / Menu to enter a code





Settings Menu / Waiting Room





Element	Design Details	Justification
Background	Solid grey colour with hex code #222222. 16:9 aspect ratio with dimensions 1920x1080.	A solid colour will not distract the user of the important parts of the UI. A dark colour won't strain the user's eyes, especially in dark environments.
Settings	Circle outline with colour hex code #DDDDDD. Wrench icon in the middle. Settings button is always in the top right, with a padding of 32 pixels.	Keeping the settings icon in the top right keeps consistency through menus. The colour provides a bright contrast with the background, making it stand out.
Title/Room code	Solid white colour with hex code #FFFFFF. 96px font size with font UD Digi	Bright colour provides the most contrast with the background and it's the

	Kyokasho.	biggest text on the screen. Makes it stand out.
Button	Rounded rectangle with dimensions 600x120. Colour hex code #444444. Text inside button has 48px font size with font UD Digi Kyokasho with colour hex code #DDDDDD.	Rounded corners provide a more modern look and make it feel more like a button than a box. Text has the same font as the title for consistency yet isn't as bright or big since it's less important.
Input text field	Rounded rectangle with dimensions 600x120. Colour hex code #DDDDDD. Text inside button has 48px font size with font UD Digi Kyokasho with colour hex code #AAAAAA.	Rounded corners provide a more modern look. Text has the same font as the title for consistency yet isn't as big since it's less important.
Toggle button	Very rounded rectangle with white circle inside. Rounded rectangle has either green colour with hex code #5CDB59 or red colour with hex code #D82915.	Green conveys on, red conveys off. The state of the toggle button is visually very clear.
Slider	Very thin rounded rectangle with dimension 850x10 with colour hex code #DDDDDD. Black circle on the slider with colour hex code #222222.	Very long width to provide lots of distance to slide and adjust settings. Contrasting colours make the slider stand out from the background, and the knob stand out from the slider.

Stakeholder Comments

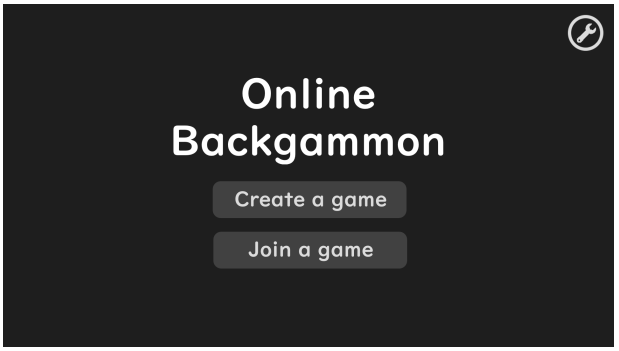
- There doesn't seem to be a way to exit the settings.
- There doesn't seem to be a way to go back, when entering a code.
- Maybe try adding a box around the information in the waiting room.

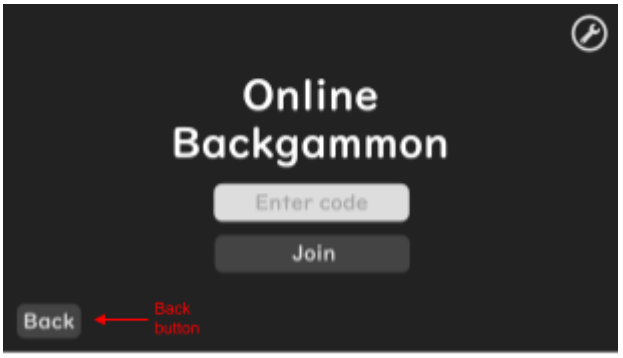
Changes

- Added a close button in the top right, allowing the user to exit the settings modal.
- Added a back button in the bottom right of all menus, except the starting menu and waiting screen (since, on the waiting screen, it is likely too late to go back without causing server-side issues).

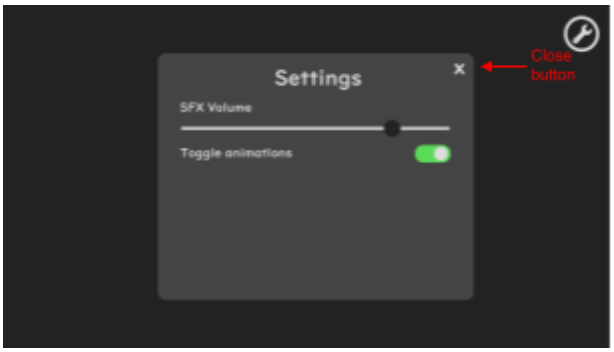
- Added a box around the information in the waiting room, improving theming consistency and making the text stand out more. Has hex colour code #444444 with dimensions 1300x470.

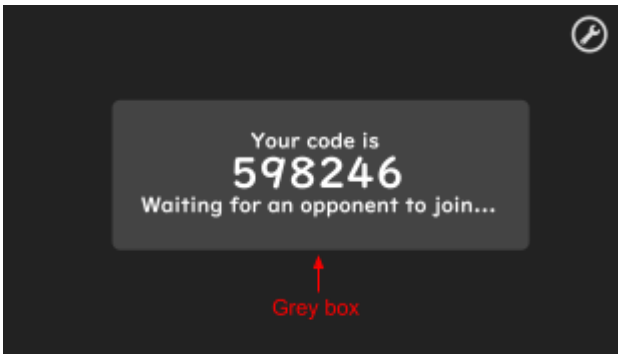
Main Menu / Menu to enter a code





Settings Menu / Waiting Room





Post Development Testing

Function and Robustness Test Plan

Function	Justification	Req. Tested	Input	Expected Result
Volume control	Ensures that the slider correctly changes the volume.	1.3, 3.3	Boundary - set volume to 0% then to 100%.	Volume should be mute then increases to max volume.
Toggle animations	Ensures that the toggle switch correctly toggles on and off animations.	1.3, 3.3	Boundary - turn animations off then on.	Animations should be disabled at first and then enabled.

Entering a username	Ensures only alphanumeric characters can be typed, which prevents any errors.	1.2, 2.2	Erroneous - enter letters, numbers and symbols.	Numbers and letters should be recognised and entered, symbols should be ignored.
Entering a room code	Ensures only numbers can be typed, which prevents any errors	1.2, 2.1	Erroneous - enter letters, numbers and symbols.	Numbers should be recognised and entered, letters and symbols should be ignored.

Usability Test Plan

Function	Justification	Req. Tested	Input	Expected Result
Menu (creating and joining a game)	Ensures that an initial menu is loaded and displayed, allowing the end user to easily navigate it on their own and create a game or join a game.	1.1-1.3, 2.1-2.4, 2.7	The user navigates the menu and creates or joins a game on their own.	The user successfully creates or joins a game.
Menu (pause)	Ensures that a pause menu is loaded and displayed during gameplay, allowing the end user to navigate it on their own and return to main menu, resign, rematch or resume playing.	1.1, 1.3, 2.8	The user navigates the menu and resigns, rematches, resumes playing or returns to main menu on their own.	The user successfully resigns, rematches, resumes playing or returns to main menu.
Menu (settings)	Ensures that a settings menu is loaded and displayed, allowing the end user to navigate it on their own and control the SFX volume or toggle animations.	1.1, 1.3, 3.3	The user navigates the menu and changes the settings on their own.	The user successfully changes the settings on their own.
Game logic (rolling a dice)	Ensures that the user can roll a dice on the board during gameplay on their own and ensures an appropriate sound effect is played.	1.3, 2.5, 2.7, 3.1, 3.2	The user rolls a dice on the board during gameplay on their own.	The user successfully rolls a dice and a sound effect successfully plays.

Game logic (moving a piece)	Ensures that the user can move a piece on the board during gameplay on their own and ensures an appropriate sound effect is played.	1.3, 2.6, 2.7, 3.1, 3.2	The user moves a piece on the board during gameplay on their own.	The user successfully moves a piece and a sound effect successfully plays.
Game logic (win condition)	Ensures that the user can win during gameplay when they have no pieces remaining and ensures an appropriate sound effect is played.	2.7, 3.2	The user gets rid of all of their pieces during gameplay and wins.	The program successfully detects that the user has won and a sound effect successfully plays.
Rendering (running game smoothly)	Ensures that the program can run smoothly at 30fps over the course of the game, regardless of mouse inputs or the game state.	3.1, 3.4	The user plays the game using their mouse.	The program successfully runs smoothly at 30fps regardless of mouse inputs or the game state.

Development

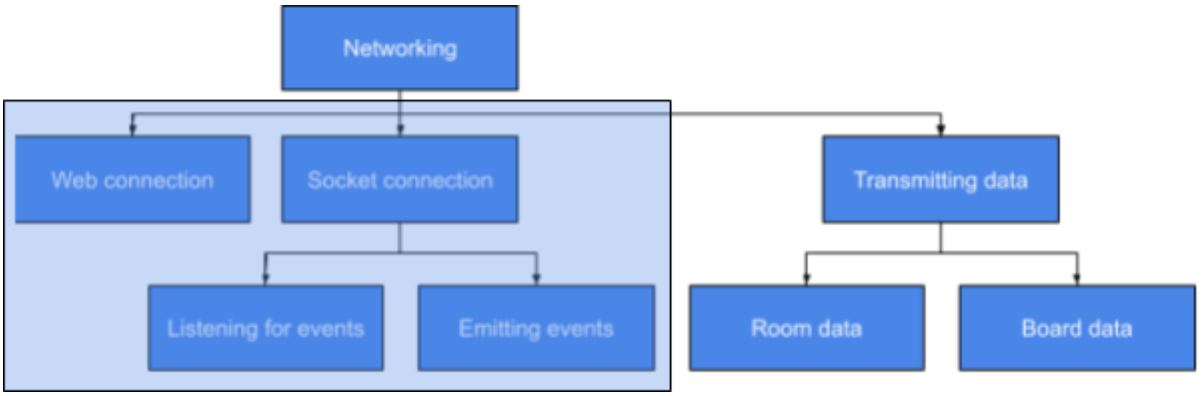
As this is a web-based multiplayer game, I need to code both the frontend of the game, which the user can interact with via a web browser, and the backend, which allows users to join the web server and play multiplayer games. To code the frontend, as this is a web-based project, I plan on using Javascript to display the Backgammon game, take in mouse inputs and send events to the backend. To code the backend, I plan on using Flask (a Python web framework) to run a web server, handle incoming requests and manage the game logic of every multiplayer game being played simultaneously. I have chosen this because Flask is relatively simple (compared to other Python framework alternatives like Django) and I am relatively comfortable coding in Python compared to other languages.

Please note that when developing the solution, I used external assets and tutorials to implement features that I did not know how to implement on my own. References to these tutorials and assets will be listed in the section below the code section of the development.

Further note that, despite the fact that this is a web based solution, I will not be including any CSS files. This is because these CSS files can get very long and add nothing to the functionality of the solution. They only change the styling of the web page. However, I will be showing UI output, so you can see the effect of the CSS.

Setting Up the Web Server

I need to set up a web server to allow me to make my solution web-based and to provide a way for the client and server to communicate via sockets. As this is mostly setup and I don't have a game or GUI to test it, I will have to code some rudimentary UI to test that the client and server can communicate and send messages. This section will tackle the decomposed modules shown below.



Iteration 1

The aim of this iteration is to set up a simple web server with some socket functionality that can send and receive information from the client. The aim is also to set up a basic UI that will allow me to test this feature.

Code

main.py

```python
from flask import Flask, Blueprint, render_template
from flask_socketio import SocketIO, emit

main = Blueprint("main", __name__)

# Serves a web page
@main.route("/")
def index():
    return render_template("index.html")

socketio = SocketIO()

# Listens for when a client connects
@socketio.on("connect")
def handle_connect():
    print("Client connected!")

# Listens for when a client sends a message
@socketio.on("send")
def handle_send(data):
    print(f"Message received: {data}")
    emit("receive", f"The server has received your message, which is {data}")

# Web server config
app = Flask(__name__)
app.config["SECRET_KEY"] = "248f9ce1-93d1-484f-8ec3-3b2ae097b6d8"
app.config["DEBUG"] = True

app.register_blueprint(main)

# Connects web server and socket
socketio.init_app(app)
socketio.run(app, host="0.0.0.0", port=80)
```

main.py serves HTML web pages, when someone goes on the website, making it a functional web server. It also listens for messages from the client. If this is the case, it will print out what it received and emit a message back to the client.

I need a web server to make my solution web based. To do this, I use libraries such as flask and flask_socketio as they allow me to use procedures within them that allow me to run a web server, create a socket connection and receive and send data using sockets. I use procedures when sending web pages and receiving information from the client as they are reusable. Thus, the server can run the same piece of code every time a client connects to the server or every time the client sends information to the server.

index.html

```
<!DOCTYPE html>
<head>
    <title>Online Backgammon</title>
</head>
<body>
    <!-- Title -->
    <h1>Online Backgammon</h1>

    <!-- Message sending UI -->
    Send message to server:
    <input type="text" id="send_text">
    <input type="submit" id="send"><br>

    <!-- Message receiving UI -->
    Message received from server:
    <span id="receive"></span>

    <!-- Importing Javascript files -->
    <script src="https://cdn.socket.io/4.7.5/socket.io.min.js"
    integrity="sha384-2huaZvOR9iDzHqslqwpR87isEmrfxqyWOF7hr7BY6KG0+hVKLoEXMPUJw3ynWuhO"
    crossorigin="anonymous"></script>
    <script src="static/game.js"></script>
</body>
</html>
```

index.html is just a base HTML template file with a UI that allows me to send messages to the server via a textbox and receive messages from the server via text. It also imports the game.js javascript and external code that allows me to set up a socket connection.

I use HTML because I need to code web pages for the web server, making the solution usable. I use script tags because that is the defined way to import javascript code (either my own or the external socket code) and run it when a user connects to the web server. Furthermore, I use input tags as that sets up the UI that the user can use to send and receive messages from the web server.

game.js

```
const socket = io();

// Listens for when the server sends a message
socket.on("receive", (data) => {
    document.getElementById("receive").textContent = data;
});

// Procedure that runs when the user clicks on the send message button
document.getElementById("send").addEventListener("click", () => {
    data = document.getElementById("send_text").value;
    socket.emit("send", data);
    document.getElementById("send_text").value = "";
});
```

game.js is a Javascript file that creates a socket variable and connects it to the server. It also updates the UI when a message is received from the server and, when the submit button is clicked, it sends whatever's in the textbox to the server.

A Javascript file is required to manage logic and functionality on the web page, such as networking logic. Storing the socket object in a variable is useful as I get to use the predefined methods in the object, such as the methods "on" and "emit" for receiving and sending messages from the server. I also use a procedure-based approach as, similar to main.py, reusable chunks of code allow the client to run the same piece of code every time the user clicks on the button or every time the client receives a message.

Testing

Test	Req. tested	Expected result	Result	Remedial action
Web server running	1.1, 3.1	A web page should be served when going on the website.		None
Sending a message to the server	2.7	The message should appear on the server console, showing that the server received it.		None
Receiving a message from the server	2.7	The message should appear on the client UI, showing that the client received it.		None
Joining a room	2.4	A message should be printed on the server console, showing that the client joined a room.		Needs to be implemented
Sending a message to multiple clients in the same room	2.7	A message should appear on the client UI of all clients connected to a specific room, showing that the message was successfully sent to all clients.		Needs to be implemented

Iteration 2

While I have a functional web server capable of sending and receiving messages via sockets, unfortunately this won't be enough for the final solution. My final solution is room-based, which means that the server will have to send a message to a room (consisting

of 2 clients) and not a single client. I will have to implement this functionality. Thus, the aim of the 2nd iteration is to code a web server where clients can join rooms and the server can send and receive information to and from rooms.

Code

main.py

```
# Listens for when a client sends a message
@socketio.on("send")
def handle_send(data):
    client_rooms = [room for room in rooms() if room != request.sid]
    print(f"Message received: {data}")

    if len(client_rooms) == 0:
        emit("receive", f"The server has received your message, which is {data}")
    else:
        emit("receive", f"The server has received your message, which is {data}", to=client_rooms[0])

# Listens for when a client joins a room
@socketio.on("join")
def handle_join(data):
    client_rooms = [room for room in rooms() if room != request.sid]
    for room in client_rooms:
        leave_room(room)

    join_room(data)
    print(f"A client has joined room {data}")
```

main.py now listens for when a client joins a room. When this happens, the server disconnects the client from all other rooms it's part of, connects the client to the room and logs this action. Furthermore, when the server sends a message to the clients, if the client is part of a room, the server sends the message to everyone in the room. Otherwise, it sends the message to the client only.

These extra socket procedures are required to manage multiple connections to the web server. The server must disconnect the client from all other rooms first because it would cause logic errors if a client was part of multiple rooms. Furthermore, allowing the server to send a message to everyone in the client's room would cause an error if the client is not already part of a room. Thus, the server must check first if the client is in a room before deciding who to send the message to.

index.html

```
<!-- Room joining UI -->
Join room:
<input type="text" id="room_text">
<input type="submit" id="room"><br>
```

index.html now provides a way in the UI to join a room.

game.js

```
// Procedure that runs when the user clicks on the join room button
document.getElementById("room").addEventListener("click", () => {
    data = document.getElementById("room_text").value;
    socket.emit("join", data);
});
```

game.js now checks when the join room button is clicked. When this happens, it sends a request to the server to join the room and provides the name of the room the client wants to join.

This is required to add functionality to the join room button. Both index.html and game.js provide a way for the user to enter a room code, join a room and send a join room request to the server.

Testing

Test	Req. tested	Expected result	Result	Remedial action
Joining a room	2.4	A message should be printed on the server console, showing that the client joined a room.		None
Sending a message to multiple clients in the same room	2.7	A message should appear on the client UI of all clients connected to a specific room, showing that the message was successfully sent to all clients.		None

Validation

This is a module with procedures that will be used entirely by the program. There will be no user interaction with this module in the final solution. As a result, there is no reason to validate user input for this module. The only reason this module has a UI is for me, the developer, to test the module. But now it's fully tested, I can get rid of the UI, which means there won't be any user input, which means I don't have to validate anything.

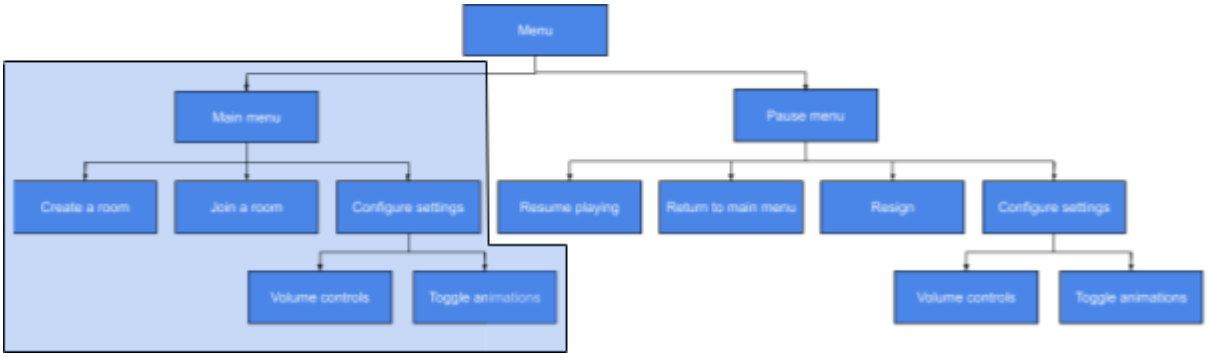
Review

So far, I've currently implemented a web server that has some functionality to send and receive messages with clients in specific rooms. I still have a lot left to do, such as developing a UI, developing the backgammon game and making it multiplayer. I also have to come back to this section in order to develop a way for the server to keep track of multiple

players, boards and rooms simultaneously. Luckily, I have a lot of time left to develop these modules and I feel like I'm ahead of schedule.

Main Menu

I need to code a main menu to allow the user to interact with the solution via a GUI on their web browser. This includes being able to create a room, join a room, enter your username and open the settings to change options like animations and volume. Unfortunately, this means I will have to completely recode everything in my index.html and game.js files as I no longer need that code to test my socket connection and sending and receiving messages to and from the server, for now. This section will tackle the decomposed modules shown below.



Iteration 1

The aim of this iteration is to code a GUI that will allow the user to create a room, join a room and open settings. I plan on using HTML and CSS to code the GUI and add styles and colours that match the design spec as closely as possible. I also plan on using Javascript to manage the code that opens the settings and makes the buttons and text fields work.

Code

index.html title + buttons div

```
<div id="title">
    <!-- Settings button -->
    <button id="settings">
        <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512">
            <!--!Font Awesome Free 6.6.0 by @fontawesome - https://fontawesome.com License - https://
            <path d="M352 320c88.4 0 160-71.6 160-160c0-15.3-2.2-30.1-6.2-44.2c-3.1-10.8-16.4-13.2-24
        </svg>
    </button>

    <!-- Title -->
    <h1 class="titleText">Online</h1>
    <h1 class="titleText">Backgammon</h1>

    <!-- Text that tells you the room code -->
    <div id="codeText" style="margin-bottom: -100px; text-align: center; white-space: nowrap;">
        <h1 style="font-size:3rem;">Your code is</h1>
        <h1>598246</h1>
        <h1 style="font-size:3rem;">Waiting for someone to join...</h1>
    </div>
</div>

<div id="buttons">
    <!-- Inputs on main menu -->
    <input class="menuButton button" id="menuCreate" type="button" value="Create a Game">
    <input class="menuButton button" id="menuJoin" type="button" value="Join a Game">

    <!-- Inputs when typing in a code  -->
    <input class="codeButton text" id="codeInput" type="text" placeholder="Enter Code">
    <input class="codeButton button" id="codeSubmit" type="button" value="Enter">

    <!-- Inputs when typing in username -->
    <input class="nameButton text" id="nameInput" type="text" placeholder="Enter Username">
    <input class="nameButton button" id="nameSubmit" type="button" value="Play">

    <!-- Back button -->
    <input class="button" type="button" value="Back" id="back">
</div>
```

index.html title + buttons div is the main part of the UI for the menu. It allows me to press buttons and type in text fields to create a menu based system where a user can create/join a game, enter a username, enter a code, wait while the other player joins and go back via a back button.

These divs are necessary as they provide a space for my UI elements. I separate the elements into different groups (title, buttons and modal) because, depending on the group, the CSS styling has to do different things. For example, everything in the title group only appears on the top half of the screen. I also add classes to each element like titleText, menuButton and codeButton so I can hide and show these classes using CSS and Javascript. As all menus occur on the same web page with the same HTML file, hiding and showing elements depending on their class is the only way to navigate through menus.

index.html modal div

```
<!-- Settings popup -->
<div id="modal">
    <div id="modalContent">
        <span id="close">&times</span>
        <h1>Settings</h1>

        <!-- SFX slider -->
        <h2>SFX Volume</h2>
        <input type="range" min="0" max="100" value="100" id="slider">

        <!-- Animation toggle button -->
        <div id="toggleAnimations">
            <h2>Toggle Animations</h2>
            <label class="switch">
                <input type="checkbox" checked>
                <span class="sliderRound"></span>
            </label>
        </div>
    </div>
</div>
```

index.html modal div is a div that contains the UI for the settings. It consists of a toggle button to toggle animations and a slider to change the volume of the sound effects.

This div is necessary as they provide a space for different settings options. I put these UI elements (the slider and toggle button) in a separate div. This is because I need it to appear on a separate layer, which ensures that the settings UI always appears over the menu. I also need it to appear on a separate layer so I can make this layer visible and invisible, allowing me to open and close settings.

menu.js updateMenu()

```
// Procedure that displays and hides elements depenging on menu
function updateMenu() {
    // Removes user input from text inputs
    document.getElementById("codeInput").value = "";
    document.getElementById("nameInput").value = "";

    // Displays/hides title depending on menu
    for (let element of document.getElementsByClassName("titleText")) {
        if (["mainMenu", "codeMenu", "nameMenu"].includes(menu)) {
            element.style.display = "block";
        } else {
            element.style.display = "none";
        }
    }

    // Displays/hides code text depending on menu
    if (["codeTextMenu"].includes(menu)) {
        document.getElementById("codeText").style.display = "block";
    } else {
        document.getElementById("codeText").style.display = "none";
    }

    // Displays/hides menu buttons depending on menu
    for (let element of document.getElementsByClassName("menuButton")) {
        if (["mainMenu"].includes(menu)) {
            element.style.display = "block";
        } else {
            element.style.display = "none";
        }
    }

    // Displays/hides code input buttons depending on menu
    for (let element of document.getElementsByClassName("codeButton")) {
        if (["codeMenu"].includes(menu)) {
            element.style.display = "block";
        } else {
            element.style.display = "none";
        }
    }

    // Displays/hides name input buttons depending on menu
    for (let element of document.getElementsByClassName("nameButton")) {
        if (["nameMenu"].includes(menu)) {
            element.style.display = "block";
        } else {
            element.style.display = "none";
        }
    }

    // Displays/hides back button depending on menu
    if (["codeMenu", "nameMenu"].includes(menu)) {
        document.getElementById("back").style.display = "block";
    } else {
        document.getElementById("back").style.display = "none";
    }
}
```

menu.js updateMenu() is a function that updates the menu by hiding and showing different elements of the same class depending on what the menu variable is set to. For example, if the menu variable is set to “mainMenu”, only the title text and menu buttons are shown.

This function is required to make the menu functional. While I could’ve implemented a menu based system across multiple web pages, which would’ve led to clearer, more concise and less repetitive code, it would’ve degraded the user experience. This is because lag is introduced between each menu button and button press as the browser loads into a new web page. Also, for cases where I have to check whether the menu variable is equal to several things, instead of writing multiple comparisons, I checked whether the menu variable was in a list. This led to more concise and more readable code.

menu.js

```
// Procedure that runs when user creates a room
document.getElementById("menuCreate").addEventListener("click", () => {
    menu = "nameMenu";
    choice = "create";
    updateMenu();
});

// Procedure that runs when user joins a room
document.getElementById("menuJoin").addEventListener("click", () => {
    menu = "codeMenu";
    choice = "join";
    updateMenu();
});

// Procedure that runs when user submits a code
document.getElementById("codeSubmit").addEventListener("click", () => {
    code = document.getElementById("codeInput").value;
    menu = "nameMenu";
    updateMenu();
});

// Procedure that runs when user submits a username
document.getElementById("nameSubmit").addEventListener("click", () => {
    name = document.getElementById("nameInput").value;
    menu = "codeTextMenu";
    updateMenu();
});

// Procedure that runs when user clicks the back button
document.getElementById("back").addEventListener("click", () => {
    if (menu == "nameMenu" && choice == "join") {
        menu = "codeMenu";
        updateMenu();
    } else {
        menu = "mainMenu";
        choice = null;
        updateMenu();
    }
    code = "";
    name = "";
});
```

menu.js contains this section which makes the buttons on the UI functional by changing the menu variable and, in some cases, the choice variable. By changing these variables and running the updateMenu() function, you create a functional menu. In some cases, where text input is required, I store the text in an appropriately named variable. Finally, the back button changes which menu it leads to depending on the menu and choice variables.

These procedures make the buttons on the menu functional. The reason I have a choice variable is because of the back button. On the enter username menu, the back button needs to go to either the enter code menu or the main menu, depending on whether the user clicked on “Create a Game” or “Join a Game”. Thus, the choice variable exists, ensuring that the back button works and doesn’t lead to any incorrect menus. And the reason the menu variable exists is so the updateMenu() function knows what menu and UI to display to the user.

menu.js

```
// Procedure that runs when user opens settings popup
document.getElementById("settings").addEventListener("click", () => {
    document.getElementById("modal").style.display = "flex";
});

// Procedure that runs when user closes settings popup
document.getElementById("close").addEventListener("click", () => {
    document.getElementById("modal").style.display = "none";
});
```

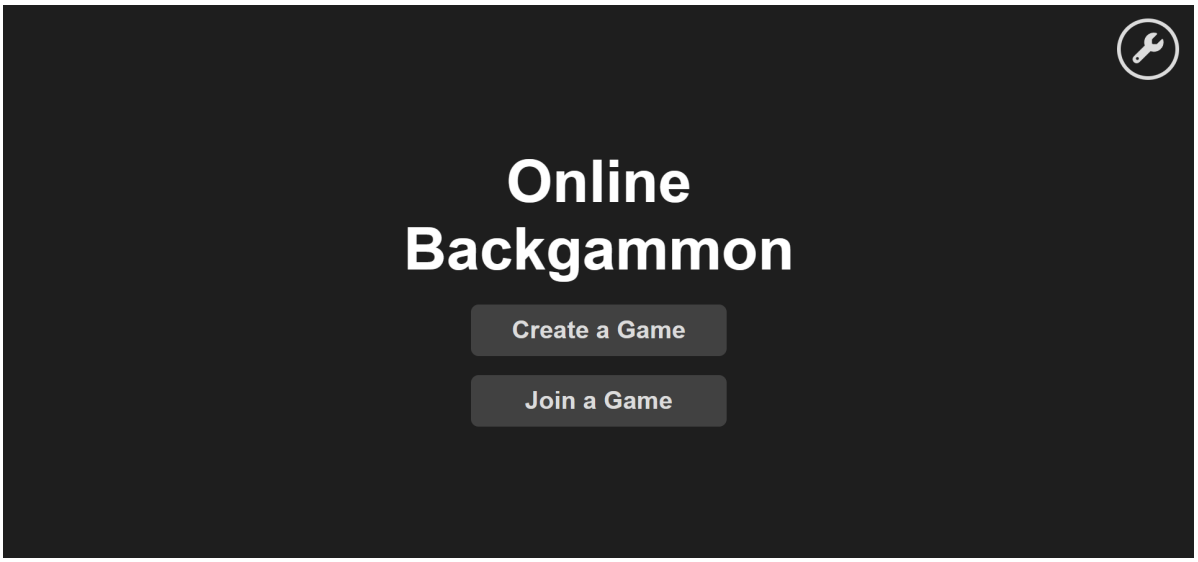
menu.js contains this section of code which allows the settings menu to be opened and closed when the user clicks on the settings and close button.

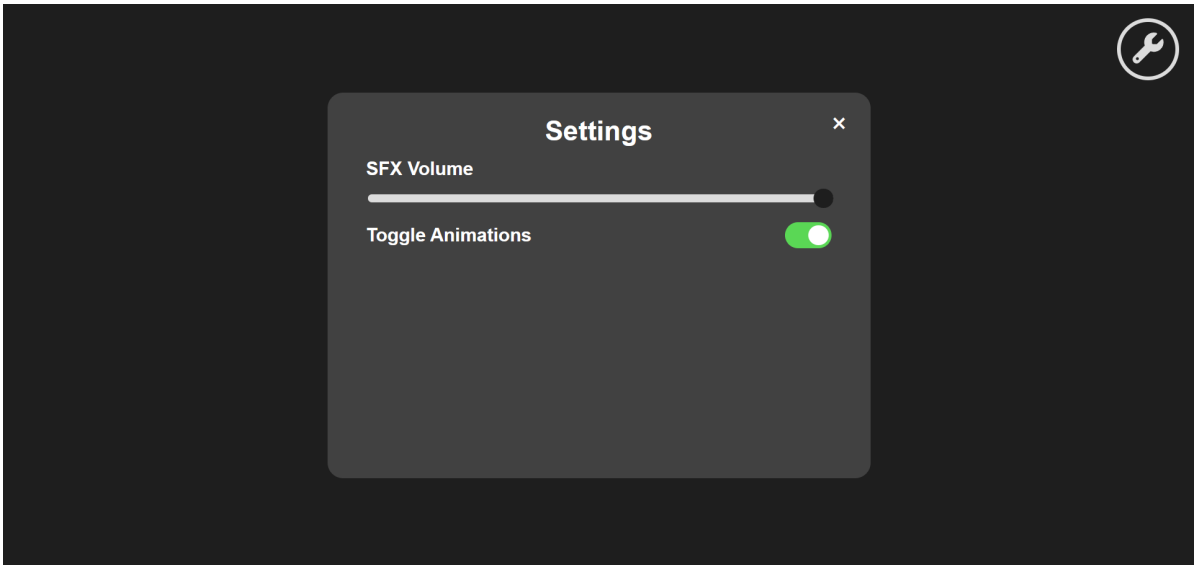
These procedures allow the user to open and close the settings menu. I find the element in the document using getElementById() and then change the style.display property to either flex or none. This is how to hide and show flexbox UI elements in Javascript. The reason why the settings element is a flexbox is because it allows me to have multiple settings options in a list format. This makes the position of each settings option flexible and allows the position to change depending on the length of the list and where it is in the list. This makes it easy to add more settings options in the future.

Notes

- The svg of the wrench icon used in my UI comes from Font Awesome’s icon library
  - https://fontawesome.com/icons/wrench?f=classic&s=solid
- Code for the popup, slider styling and toggle button is inspired by W3Schools tutorials
  - https://www.w3schools.com/howto/howto_css_modals.asp
  - https://www.w3schools.com/howto/howto_js_rangeslider.asp
  - https://www.w3schools.com/howto/howto_css_switch.asp

GUI Output





Testing

Test	Req. tested	Expected result	Result	Remedial action
Navigating between menus	1.1, 1.3	The user can navigate to the correct menus via the menu buttons		None
Entering a room code	1.2, 2.1	The user can type a room code that only consists of numbers		Letters and long numbers can be typed in - need to properly

		and is at most 6 numbers long		implement validation.
Entering a username	1.2, 2.2	The user can type a username that only consists of alphanumeric characters		Unicode characters can be typed in - need to properly implement validation
Volume control	1.3, 3.2, 3.3	The user can adjust a slider that changes the volume of sound effects		While the user can adjust a slider, this slider isn't functional - needs to change the volume of sound effects
Toggling animation	1.3, 3.3	The user can click on a toggle button that toggles animation		While the user can click on a toggle button, the button isn't functional - needs to toggle animation

Iteration 2

The aim of this iteration is to implement client-side validation such that only alphanumeric characters can be typed into the username text field and at most 6 numbers can be typed into the room code text field. Furthermore, I have to add sound effects and animations to the GUI so that I can test whether or not the volume slider and toggle animation button is functional or not. I plan on using Javascript to code the validation, the sound effects, the animation and the settings functionality.

Following input from my stakeholder, I've also been asked to implement localisation into multiple languages and multiple themes (i.e. dark mode, light mode). Specifically, I've been asked to add a pink mode. As a result, I've added these features and added new iterative tests below to test the new features.

Code

index.html langChoice div

```
<!-- Language dropdown -->
<div id="langChoice">
    <h2 data-i18n="language"></h2>
    <div class="dropdown">
        <button onclick="openDropdown('langDropdown')" class="dropbtn" id="langDropdown">English
        (United Kingdom)</button>
        <div id="langDropdownContent" class="dropdown-content">
            <a id="english" onclick="changeLanguage('en')">English (United Kingdom)</a>
            <a id="australian" onclick="changeLanguage('au')">English (Australia)</a>
            <a id="spanish" onclick="changeLanguage('es')">Español</a>
            <a id="french" onclick="changeLanguage('fr')">Français</a>
            <a id="german" onclick="changeLanguage('de')">Deutsch</a>
            <a id="arabic" onclick="changeLanguage('ar')">العربية</a>
            <a id="japanese" onclick="changeLanguage('ja')">日本語</a>
            <a id="hebrew" onclick="changeLanguage('he')">עברית</a>
        </div>
    </div>
</div>
```

index.html langChoice div is a dropdown that allows the user to change the language of the program.

Localisation allows more non-English users to use my solution. The data-i18n tag is present throughout my HTML code and allows the text content of each element to change depending on the language selected. Here, you can see data-i18n=”language”. This means that the h2 tag will have the translated word “language” in its text content. Translations are not automatic and are stored in a separate json file.

menu.js themeChoice div

```
<!-- Theme dropdown -->
<div id="themeChoice">
    <h2 data-i18n="theme"></h2>
    <div class="dropdown">
        <button onclick="openDropdown('themeDropdown')" class="dropbtn" id="themeDropdown">Dark</button>
        <div id="themeDropdownContent" class="dropdown-content">
            <a id="dark">Dark</a>
            <a id="light">Light</a>
            <a id="pink">Pink</a>
        </div>
    </div>
</div>
```

index.html themeChoice div is a dropdown that allows the user to change the theme of the program.

I have a dark and light theme because that’s standard for many web applications. The pink theme is not as standard and exists because of a request from the stakeholder.

menu.js playSFX()

```
// Procedure that plays the SFX
function playSFX() {
    const audioElement = new Audio(sfx);
    audioElement.volume = document.getElementById("slider").value / 100;
    audioElement.play()
}
```

menu.js playSFX() is a function that plays a sound effect at the volume specified by the user.

This exists because of my solution's requirements to have a way of changing the settings, such as the volume of sound effects. In order to do this, I have to implement sound effects.

menu.js getRidOfDropdowns() + openDropdown()

```
// Procedure that gets rid of dropdown menus
function getRidOfDropdowns() {
    var dropdowns = document.getElementsByClassName("dropdown-content");
    var i;
    for (i = 0; i < dropdowns.length; i++) {
        var openDropdown = dropdowns[i];
        if (openDropdown.classList.contains('show')) {
            openDropdown.classList.remove('show');
        }
    }
}

// Procedure that opens dropdown
function openDropdown(dropdown) {
    if (document.getElementById(dropdown + "Content").classList.contains("show")) {
        getRidOfDropdowns();
    } else {
        getRidOfDropdowns();
        document.getElementById(dropdown + "Content").classList.add("show");
    }
}
```

menu.js getRidOfDropdowns() + openDropdown() are functions that get rid of dropdowns and open dropdowns.

I have these functions because I need dropdown functionality so the user can choose from a selection of themes and languages. I need a function to get rid of all dropdowns because if the user clicks off a dropdown, I have no idea which dropdown the user clicked off. So, I just get rid of all of them. I also need a function to open dropdowns to make them usable and functional. I add and remove the show class as that's a class I've already defined in my CSS to hide and show elements. Finally, I get rid of all dropdowns before opening a dropdown because a user might click off a dropdown by opening a new dropdown. It also ensures only one dropdown can be open at a time.

menu.js updateContent()

```
// Procedure that updates all of the text on the website depending on language
function updateContent(langData) {
    document.querySelectorAll('[data-i18n]').forEach(element => {
        const key = element.getAttribute('data-i18n');
        // Figures out which attribute to modify depending on element type
        // i.e. for button input elements, change the value, not the innerHTML
        if (element.matches("input")) {
            if (element.classList.contains("text")) {
                element.placeholder = langData[key];
            } else {
                element.value = langData[key];
            }
        } else {
            element.innerHTML = langData[key];
        }
    });
}
```

menu.js updateContent() is a function that changes the text of every HTML element if it has the data-i18n tag. This is useful for changing languages.

This function is essential in order to add localisation features to my solution. Furthermore, HTML is annoying and doesn't have a standardised way of accessing the text content of different HTML elements. For some elements, you access the value attribute. For others, the placeholder attribute. Thus, I need several if else statements to change the text properly.

menu.js changeLanguage()

```
// Procedure that changes the language and flips horizontally if in a RTL language
// and flips vertically if on the southern hemisphere
async function changeLanguage(lang) {
    langData = await fetchLanguageData(lang);
    updateContent(langData);
    if (lang == "ar" || lang == "he") {
        document.querySelector(':root').style.setProperty("--arabic", "-1");
        document.querySelector(':root').style.setProperty("--australia", "0deg");
    } else if (lang == "au") {
        document.querySelector(':root').style.setProperty("--arabic", "1");
        document.querySelector(':root').style.setProperty("--australia", "180deg");
    } else {
        document.querySelector(':root').style.setProperty("--arabic", "1");
        document.querySelector(':root').style.setProperty("--australia", "0deg");
    }
}
```

menu.js changeLanguage() is a function that runs several language functions together, changing the language and updating the text on the website. It also sets several CSS variables depending on the language. These CSS variables can flip the website horizontally, adding RTL support for languages like Arabic and Hebrew.

While not completely essential to the functionality of the solution, it is standard on websites that, when including localisation for RTL languages like Arabic and Hebrew, the website UI

elements should change and be flipped horizontally. This is because Arabic and Hebrew speakers read from right to left and thus navigating the menu becomes easier for them.

menu.js changeTheme()

```
// Procedure that changes all of the CSS color variables depending on the theme
function changeTheme() {
    if (theme == "dark") {
        document.querySelector(':root').style.setProperty("--bgColor", "#222222");
        document.querySelector(':root').style.setProperty("--txtColor", "#DDDDDD");
        document.querySelector(':root').style.setProperty("--btnColor", "#444444");
        document.querySelector(':root').style.setProperty("--settingsHover", "#474747");
        document.querySelector(':root').style.setProperty("--btnHover", "#696969");
        document.querySelector(':root').style.setProperty("--placeholderColor", "#AAAAAA");
        document.querySelector(':root').style.setProperty("--dropdownColor", "#555555");
        document.querySelector(':root').style.setProperty("--txtInputBgColor", "#DDDDDD");
        document.querySelector(':root').style.setProperty("--txtInputColor", "#444444");
    } else if (theme == "pink") {
        document.querySelector(':root').style.setProperty("--bgColor", "#FF96C4");
        document.querySelector(':root').style.setProperty("--txtColor", "#FFFFFF");
        document.querySelector(':root').style.setProperty("--btnColor", "#FF6193");
        document.querySelector(':root').style.setProperty("--settingsHover", "#FF78BB");
        document.querySelector(':root').style.setProperty("--btnHover", "#FF78BB");
        document.querySelector(':root').style.setProperty("--placeholderColor", "#AAAAAA");
        document.querySelector(':root').style.setProperty("--dropdownColor", "#FF96C4");
        document.querySelector(':root').style.setProperty("--txtInputBgColor", "#DDDDDD");
        document.querySelector(':root').style.setProperty("--txtInputColor", "#444444");
    } else {
        document.querySelector(':root').style.setProperty("--bgColor", "#EEEEEE");
        document.querySelector(':root').style.setProperty("--txtColor", "#474747");
        document.querySelector(':root').style.setProperty("--btnColor", "#FFFFFF");
        document.querySelector(':root').style.setProperty("--settingsHover", "#C7C7C7");
        document.querySelector(':root').style.setProperty("--btnHover", "#C7C7C7");
        document.querySelector(':root').style.setProperty("--placeholderColor", "#AAAAAA");
        document.querySelector(':root').style.setProperty("--dropdownColor", "#EEEEEE");
        document.querySelector(':root').style.setProperty("--txtInputBgColor", "#FFFFFF");
        document.querySelector(':root').style.setProperty("--txtInputColor", "#474747");
    }
}
```

menu.js changeTheme() is a function that changes a bunch of CSS variables to change the colours of the elements on the web page, thus changing the theme. This exists to add more customizability to my solution.

menu.js showError()

```
// Procedure that displays an error
function showError(text) {
    document.getElementById("errorText").textContent = text;

    if (document.getElementById("errorBanner").classList.contains("errorAppear")) {
        clearTimeout(closeError);
        document.getElementById("errorBanner").classList.remove("errorAppear");;
    }

    // For whatever reason, I have to remove the errorAppear class and add it again
    // 10ms later so the animation plays
    setTimeout(function() {
        document.getElementById("errorBanner").style.transform = "translateY(0px)";
        document.getElementById("errorBanner").style.visibility = "visible";
        document.getElementById("errorBanner").classList.add("errorAppear");
        document.getElementById("errorBanner").classList.remove("errorDisappear");
    }, 10);

    closeError = setTimeout(function (){
        document.getElementById("errorBanner").style.transform = "translateY(20vh)";
        document.getElementById("errorBanner").style.visibility = "hidden";
        document.getElementById("errorBanner").classList.add("errorDisappear");
        document.getElementById("errorBanner").classList.remove("errorAppear");
    }, 3000)
}
```

menu.js showError() is a function that shows an error on the menu when you enter a non-existent room code or an invalid username. It will animate the red error banner to pop up from the bottom of the screen, stay there for 3 seconds and then go back down.

I need to show errors because I need to validate the user's input and, if it's incorrect, I have to tell the user why. I use the Javascript functions clearTimeout() and setTimeout() in order to ensure the error pops up for only 3 seconds, ensuring that the user has ample time to read the error message. Furthermore, I show and hide errors using CSS classes because I've already configured them in a separate CSS file with animations, allowing the error to pop up and go down smoothly.

en.json

```
{
    "onlineBackgammon": "Online Backgammon",
    "online": "Online",
    "backgammon": "Backgammon",
    "back": "Back",
    "createAGame": "Create a Game",
    "joinAGame": "Join a Game",
    "enterUsername": "Enter Username",
    "play": "Play",
    "enterCode": "Enter Code",
    "enter": "Enter",
    "yourCodeIs": "Your code is",
    "waitForJoin": "Waiting for someone to join...",
    "settings": "Settings",
    "language": "Language",
    "sfx": "SFX",
    "sfxVolume": "SFX Volume",
    "animations": "Animations",
    "theme": "Theme",
    "numbersError": "You need to enter numbers only",
    "sixDigitError": "You need to enter a 6-digit room code",
    "alphanumericError": "You need to enter alphanumeric characters only",
    "usernameError": "You need to enter a username"
}
```

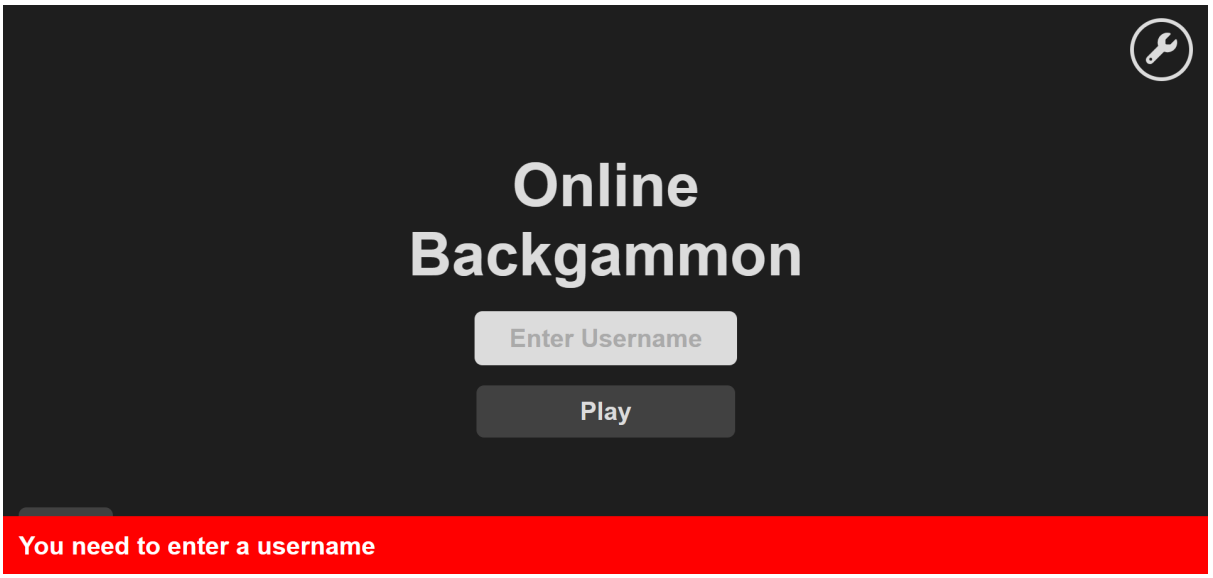
en.json is a json file that contains all of the translated text values for the English version of the solution.

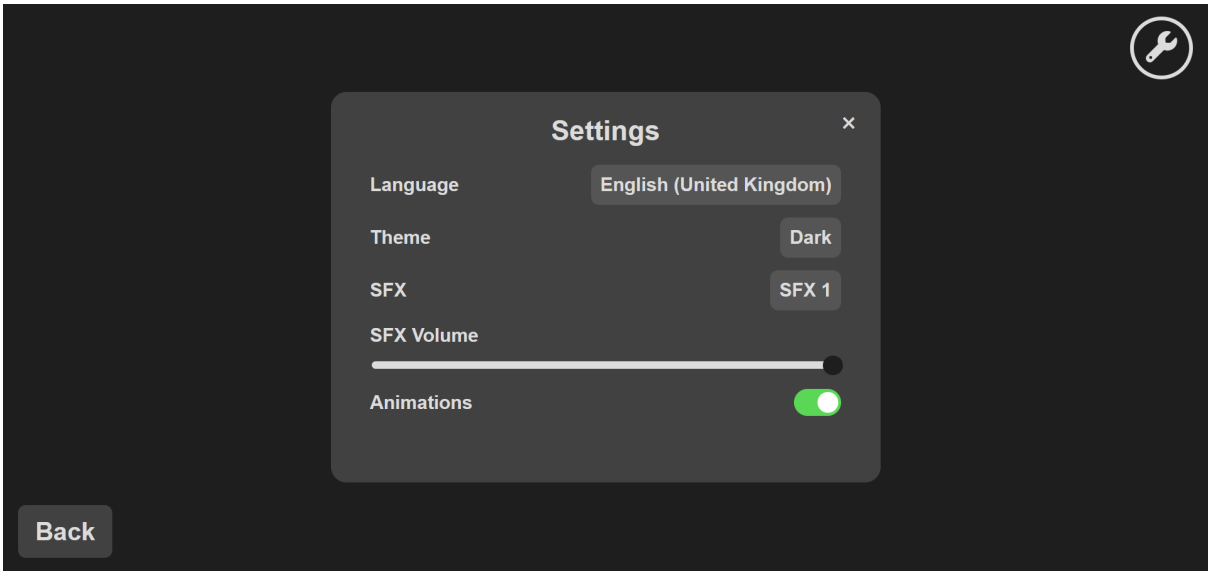
These json files are required for localisation. Putting text values in a separate json file is necessary in order to enable localisation of my solution. For every language I implement, I can just add a new json file with the same keys and different translated contents. The keys are required so the Javascript knows where to insert each translated text. This method makes adding more languages in the future easier.

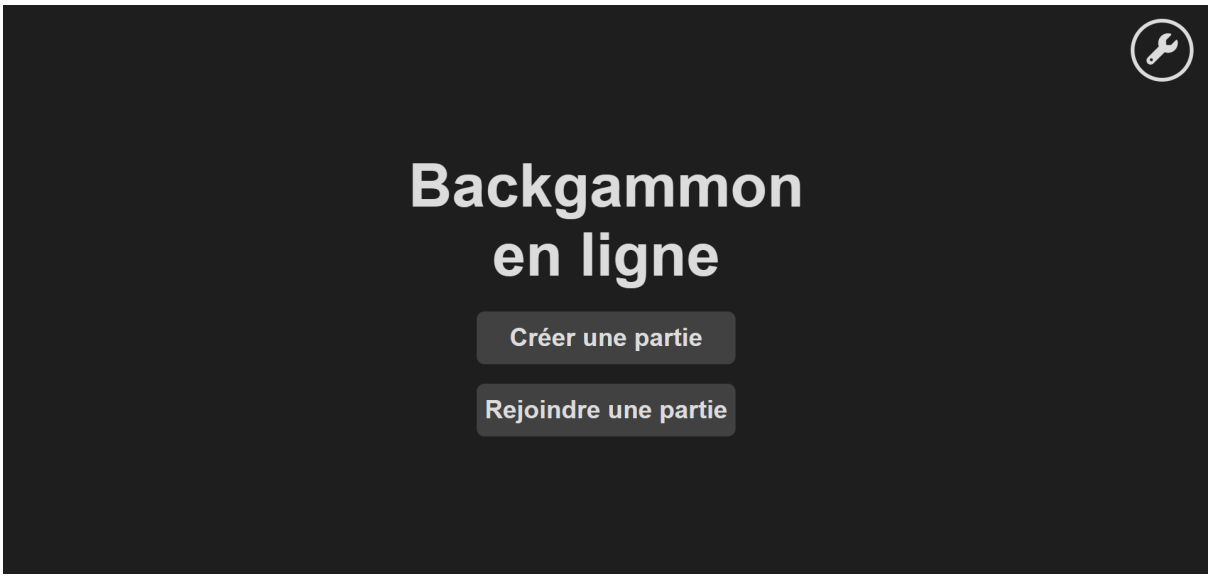
Notes

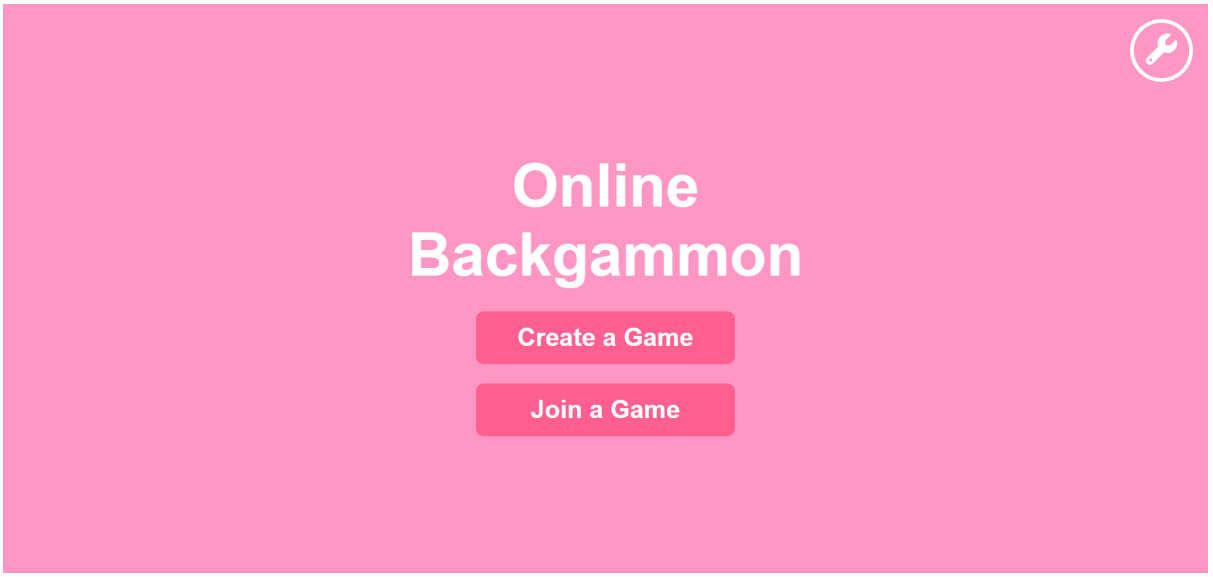
- Code for the dropdown menu is inspired by W3Schools tutorial
    - https://www.w3schools.com/howto/howto_css_dropdown.asp
- Code for localisation comes from
    - https://medium.com/@nohanabil/building-a-multilingual-static-website-a-step-by-step-guide-7af238cc8505
- I am not fluent in 7 languages. I used AI to translate text to different languages for localisation and then double checked the AI translations for languages that my friends and I knew. Out of the 7 different languages, 5 of them are accurately localised.

GUI Output


Online
Backgammon
Enter Username
Play
You need to enter a username


Settings
Language
English (United Kingdom)
Theme
Dark
SFX
SFX 1
SFX Volume
Animations
Back





Testing

Test	Req. tested	Expected result	Result	Remedial action
Entering a room code	1.2, 2.1	The user can type a room code that only consists of numbers and is at most 6 numbers long		None
Entering a username	1.2, 2.2	The user can type a username that only consists of		None

		alphanumeric characters		
Volume control	1.3, 3.2, 3.3	The user can adjust a slider that changes the volume of sound effects		None
Toggling animation	1.3, 3.3	The user can click on a toggle button that toggles animation		None
Displaying an error	2.1, 2.2, 3.4	An error message pops up smoothly when validating user input, telling what went wrong		None
Toggling languages	3.3	The user can change the language of the solution		None
Toggling themes	3.3	The user can change the theme of the solution		None

Validation

Most of my solution is self-validating as the user clicks on predefined buttons that I have coded. However, there is some text input for entering code and usernames. Thus I have implemented 4 validation checks - a presence check for the username, an alphanumeric check for the username, a numbers check for the code and a length check for the code (also acts as a presence check). The validation code will be pasted again for better clarity.

menu.js

```
// Checks the code is made up of numbers only
for (let letter of code) {
    if (!"0123456789".includes(letter)) {
        code = "";
        showError(langData["numbersError"]);
        return;
    }
}

// Checks the code has a length of 6
if (code.length != 6) {
    code = "";
    showError(langData["sixDigitError"]);
    return;
}
```

```
// Checks the username is made up of alphanumeric characters
for (let letter of name) {
    if (!"abcdefghijklmnopqrstuvwxyz0123456789 ".includes(letter.toLowerCase())) {
        name = "";
        showError(langData["alphanumericError"]);
        return;
    }
}

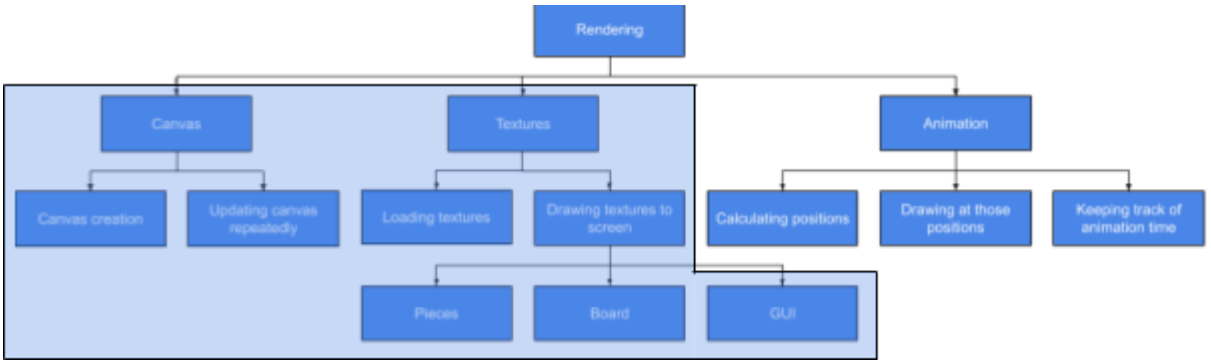
// Checks for the presence of a username
if (name.length == 0) {
    showError(langData["usernameError"]);
    return;
}
```

Review

So far, I've currently implemented a web server and a fully functional menu with a lot of features, options and animations. I still have a lot left to do, such as developing the backgammon game itself and making it multiplayer. But I won't have to come back to this module for a while, not until I need to add resignation and rematch settings. Unfortunately, I don't have much time left because I spent a lot of time on themes and languages, which was probably not the most efficient use of time. I am quite behind schedule and will need to catch up a bit to finish the development section on time.

Rendering the Game

I need to set up a way to render the game so that the user can view the game and thus play it. I will implement this as a separate web page as my singular web page for menu actions is quite cluttered with multiple lines of code and I think keeping game and menu code separate is useful and helps organise my project. I will have to create the board texture, create the piece texture and load and render them at the right positions on a Javascript canvas that redraws itself 30 times a second. I will also have to create a GUI in line with the current theme of the website that has a pause/settings button and a way to view the opponents username. This section will tackle the decomposed modules shown below.



Iteration 1

The aim for this iteration is to implement a GUI on a separate page that shows the canvas which will render the board and displays the name of your opponent. I plan on modifying the Python code to create a separate web page, using HTML to create the layout of the web page, using a large part of my existing CSS code to style it and using Javascript to load and render the board texture on the canvas.

Code

main.py game()

```
# Serves the game page
@main.route("/game")
def game():
    return render_template("game.html")
```

main.py game() is just a procedure that serves the game.html web page when the user navigates to the /game page.

game.html gameArea div

```
<!-- Game area div, contains everything under the header -->
<div id="gameArea">
    <!-- Game div, contains board and player usernames -->
    <div id="game">
        <p>Player 1</p>
        <canvas id="canvas" width="852" height="500" style="background: #FFFFFF;"></canvas>
        <p>Player 2</p>
    </div>

    <!-- Game info box, contains game info and buttons to resign and return to main menu -->
    <div id="gameInfo">
        <h1 data-i18n="onlineGame"></h1>
        <h2 id="gameInfoCode"></h2>
        <h2 id="gameInfoWhite"></h2>
        <h2 id="gameInfoBlack"></h2>
        <h2 id="gameInfoTurn" data-i18n="yourTurn"></h2>
        <button data-i18n="resign"></button>
        <button data-i18n="mainMenu"></button>
    </div>
</div>
```

game.html gameArea div is a section of HTML code responsible for the GUI of the actual game. It has 2 divs. 1 div contains the game itself (as seen by the canvas tag) and the usernames of the players. The other div contains information about the game, such as the code, who's playing white and black and buttons to resign and go back to the main menu.

The gameArea div is required as it provides a space to put UI elements such as the game and any game information. I split it into 2 sections because, upon observation of other game UIs similar to this one, most of them contain 2 sections placed side by side. One section has the game and the other section has information about the game and buttons such as the

resign button. I followed this approach because it was aesthetically pleasing and highly functional. I also needed a place to put the resign and main menu buttons (part of the decomposition of the menu module). Originally, I was going to put them within the settings menu however there wasn't enough space and coding a scrolling settings menu seemed tedious. So, I opted for creating a second section called gameInfo and putting the buttons there.

game.js Board

```
// Board class, contains board data and functions for rendering
class Board {
    constructor() {
        this.img = new Image();
        this.img.src = "static/assets/board.png";
    }

    // Renders the board
    renderBoard() {
        ctx.drawImage(this.img, 0, 0, 852, 500);
    }
}
```

game.js Board is a Javascript class that has one attribute: the image of the board. It also has one procedure, which draws the board onto the Javascript canvas.

This board class is necessary as all Backgammon games need a board to play on. I chose to implement the board as a class as, in the future, I anticipate the board will have more attributes, such as board data (where the pieces are stored), and more procedures, such as rendering the pieces, rendering highlighted points and pieces, updating the board state when a move is made etc. To help with organising my code, I believe that, when I implement these functions, grouping them under one class will make my code a lot cleaner.

game.js Game

```
// Game class, currently it only contains the board
// In the future, it will contain players, selected pieces, valid move calculations etc.
class Game {
    constructor() {
        this.board = new Board();
    }
}
```

game.js Game is a Javascript class that has one attribute: the board object. I implemented it as a class for similar reasons that I mentioned above. I know that the game will have a lot more attributes to store the pieces, the dice, the turn, the valid moves, the players username and their colors etc. I also know that the game will have more procedures, to switch the turn, to move a piece etc. To help with organising my code, grouping future procedures and attributes under one class will make my code a lot cleaner.

game.js variables and draw()

```
// Initialise JS canvas and game object
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");
let game = new Game();

// Function that renders the game on the canvas
function draw() {
    game.board.renderBoard();
}

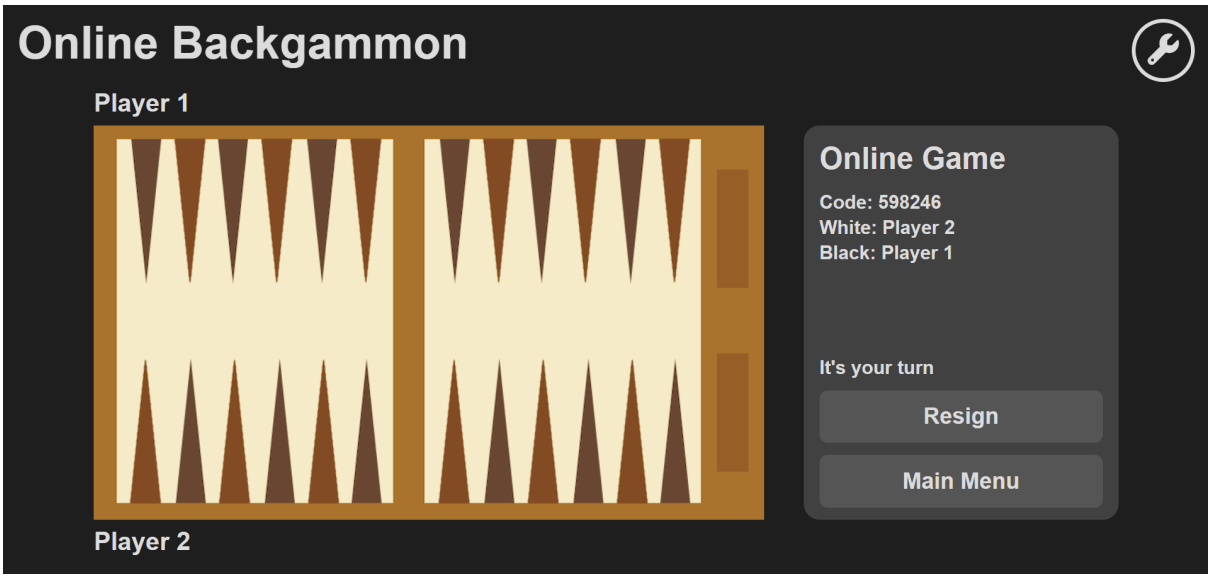
// Calls the draw function every 33ms (to run at roughly 30fps)
const interval = setInterval(draw, 33);
```

game.js variables include the canvas, the context (which provides functions for drawing on the canvas) and the game. It also has an interval variable, which just runs the draw function every 33ms.

game.js draw() is a function that renders the Backgammon game.

This function is necessary to render the Backgammon game to the screen. In order to use the Game class, I have to create an object and store it in a variable. Furthermore, to draw anything in Javascript, I have to initialize the canvas and context in variables to use its functions to draw shapes and images. I implemented a draw function as a function because, in the future, I will need to draw pieces, dice, text etc. Placing all of those functions in a draw function is useful because I can then run a single function to render everything. I can then run that function 30 times a second by using the Javascript function setInterval().

GUI Output



Testing

Test	Req. tested	Expected result	Result	Remedial action
Displaying a GUI	1.1, 3.4	The game and its GUI elements should be displayed with no visual bugs		None
Rendering the board	3.1, 3.4	The board should be rendered at 30fps		None
Rendering the pieces	3.1, 3.4	The pieces should be rendered in the correct places at 30fps		Pieces are not rendered - need to properly implement piece rendering

Iteration 2

The aim for this iteration is to implement a method of storing pieces, figuring out the correct locations of the pieces and rendering them. Furthermore, I noticed a bug that wasn't in my iterative test plan but I discovered through developing this program. When scaling the browser window, thanks to the dynamic nature of flexboxes, my website tends to usually scale quite well. However, the canvas element does not scale and is often unusable on small windows. Thus, I'd like to implement a method to dynamically resize the canvas when the window resizes.

Code

game.js resize()

```
// Scale variable, value that determines how much to scale the canvas by
let scale = 1;

// Pixel function, recalculates every coordinate value (n) depending on size of canvas
function pixel(n) {
    return Math.floor(n * scale);
}

// Resize function, runs when the window is resized, recalculates scale variable
// changes the canvas dimensions and changes the height of the game info box
function resize() {
    scale = Math.min(window.innerWidth / 1536, window.innerHeight / 730);
    canvas.width = pixel(852);
    canvas.height = pixel(500);
    document.getElementById("gameInfo").style.height = pixel(500).toString() + "px";
}
```

game.js resize() is a function that recalculates the scale variable, changes the canvas dimensions and changes the height of the gameInfo box. As you can see, it uses the pixel function, which uses the scale value to return a different value of n.

I need to resize the game window to make the website functional on many devices, regardless of their screen size. There is no easy way to resize the elements within the canvas. Thus, I have opted for calculating every coordinate, length, radius value etc. using the pixel function. This takes the default coordinates and scales them depending on the size of the window. While this requires recalculating the same coordinates every frame, I deemed it to be too complex to optimise this situation by storing previously calculated values in some sort of cache. Furthermore, I have not noticed any considerable performance hits when testing on my computer. The scale function changes based on the window size compared to the default window size that I have been using to test and design my program (1536 x 730). Finally, I continuously resize the canvas and the gameInfo box at the same time because, design-wise, it looks weird if they're not the same height.

game.js Board constructor()

```
constructor() {
    // Board data is an array with 24 elements (for 24 points on a Backgammon board)
    // Each point stores 2 values: the number of pieces and the color of pieces
    this.board = [
        {count: 2, color: "black"},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 5, color: "white"},
        {count: 0, color: null},
        {count: 3, color: "white"},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 5, color: "black"},
        {count: 5, color: "white"},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 3, color: "black"},
        {count: 0, color: null},
        {count: 5, color: "black"},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 0, color: null},
        {count: 2, color: "white"}
    ];

    this.img = new Image();
    this.img.src = "static/assets/board.png";
}
```

game.js Board constructor() now has the board attribute, which stores the data of all of the pieces on the board. It is an array with 24 elements, with each element representing the 24 points on a Backgammon board. The count attributes represent the number of pieces on each point and the color represents the color of those pieces.

I need the board to store piece data so the program knows where to draw the pieces and which moves are valid. I can define the color for all of the pieces on a point instead of defining the color of each piece individually. This is because, in a Backgammon game, it is guaranteed that all of the pieces on a point will be the same color. But, the main thing I have to justify is the method in which I store the data for the pieces. I could have implemented storing 24 arrays, each with Piece objects. But the reason I opted for this approach is because it doesn't use as much memory as a 2d array, it is much easier to send data in this format to and from the server and I cannot think of any problems that this method will cause when implementing more functions such as moving pieces and capturing.

game.js Board renderPiece()

```
// Renders a piece
renderPiece(x, y, color) {
    if (color == "black") {
        ctx.fillStyle = "#000000";
    } else {
        ctx.fillStyle = "#ffffff";
    }

    // Draws a circle with centre x, y and radius 20px
    ctx.beginPath();
    ctx.arc(pixel(x), pixel(y), pixel(20), 0, 2 * Math.PI);
    ctx.fill();

    // Draws an outline for the circle
    ctx.lineWidth = pixel(1);
    ctx.strokeStyle = "#000000";

    ctx.stroke();
    ctx.closePath();
}
```

game.js Board renderPiece() is a function that renders a single piece at specified coordinates. It doesn't do any calculations as to where the piece should go. It renders the piece by simply drawing a circle and an outline.

I need a way to render each piece to the screen so the user can see the pieces. While I did plan for each piece to have its own texture, I thought that rendering 30 separate images at the same time would cause performance issues, especially for browsers running on lower spec computers. So, I made the decision to just draw circles with the canvas context interface. I also figured that using circles wouldn't impact the experience too much as plain circles and the piece texture are very similar. As you can further see, I use the pixel function to scale measurements based on canvas/window size to ensure resizability.

game.js Board renderPieces()

```
// Calculates where each piece should be and renders them
renderPieces() {
    this.board.forEach((point, index) => {
        if (point.count > 0) {
            // The gaps between points at the top and bottom because
            // for some reason, they aren't perfectly equidistant
            let intervalsTop = [0, 56, 111, 167, 224, 280];
            let intervalsBottom = [0, 57, 113, 171, 226, 281];
            let x, y;

            // Calculates x axis depending on which point it is
            if (index >= 12 && index <= 17) {
                x = 66 + intervalsTop[index - 12];
            } else if (index >= 18 && index <= 23){
                x = 458 + intervalsTop[index - 18];
            } else if (index >= 6 && index <= 11) {
                x = 66 + intervalsBottom[11 - index];
            } else {
                x = 458 + intervalsBottom[5 - index];
            }

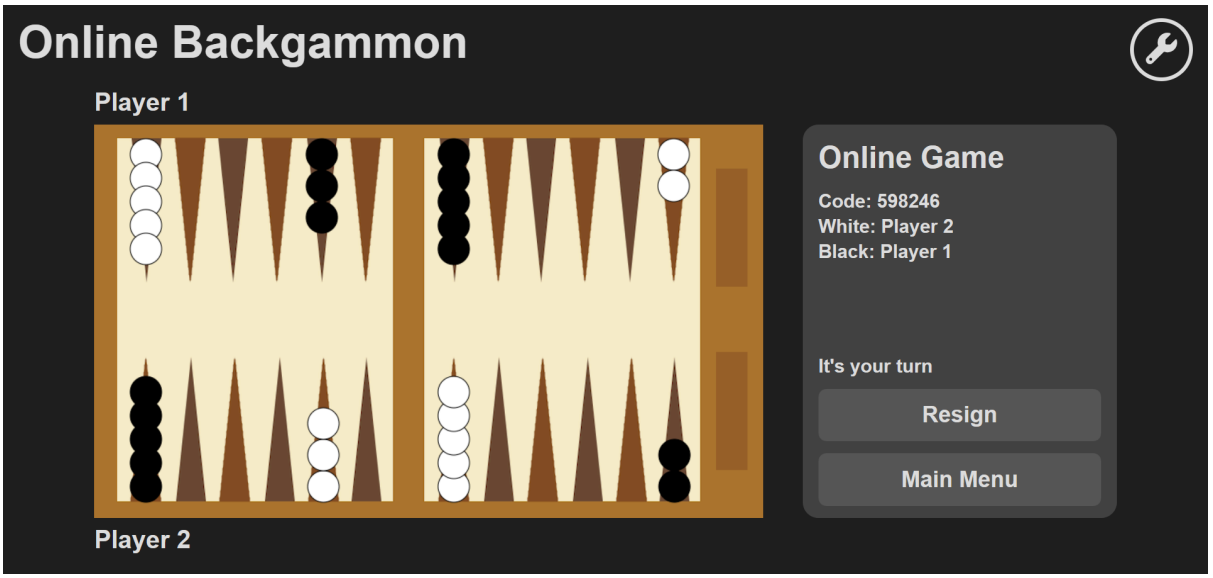
            // Calculates y axis depending on where the piece is within the point
            for (let i = 0; i < point.count; i++) {
                if (index >= 12) {
                    y = 38 + i * Math.min(40, 120 / (point.count - 1));
                } else {
                    y = 460 - i * Math.min(40, 120 / (point.count - 1));
                }

                this.renderPiece(x, y, point.color);
            }
        }
    });
}
```

game.js Board renderPieces() is a function that reads the board data, calculates the coordinates of each piece using some math and renders each piece using the renderPiece() function written above.

Unfortunately, my skills for designing the board were so unbelievably inept that I made a lot of mistakes, including somehow not spacing the spikes equidistantly. So, like any competent programmer, instead of redesigning the board, I just coded more complicated code. As you can see from the variables intervalsTop and intervalsBottom, those are the distances in pixels between the 1st and nth point. Then, based on the index of the point, I figure out the x coordinate. I also figure out the y coordinate based on the position of the piece on the point. I came across a bug where pieces could theoretically stack infinitely and off the board. As a result, I must code more math (Math.min(40, 120 / (point.count - 1)) to ensure that all of the pieces fit on the point.

GUI Output



Testing

Test	Req. tested	Expected result	Result	Remedial action
Rendering the pieces	3.1, 3.4	The pieces should be rendered in the correct places at 30fps		None

Validation

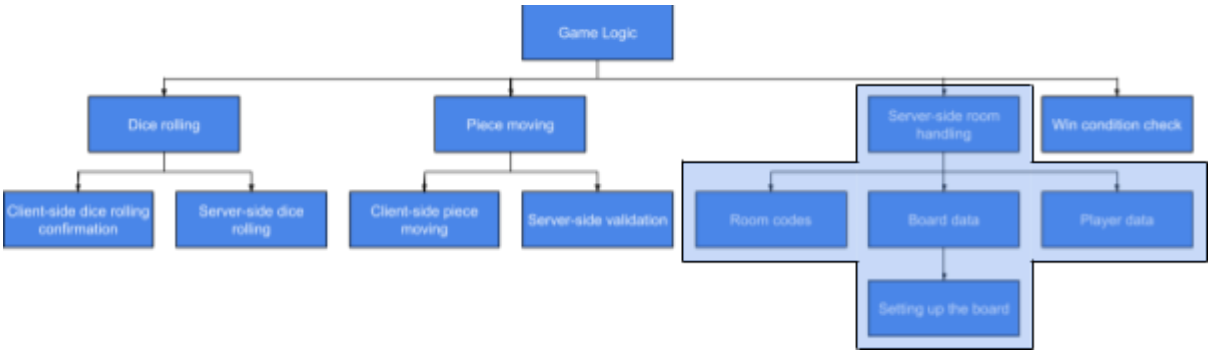
While there will be user interaction with this module in the final solution, every user interaction involves them interacting with predefined buttons that I have coded. As a result, this module is self-validating.

Review

So far, I've currently implemented a web server, a fully functional main menu and a game web page that displays the backgammon board with pieces. I still have a lot left to do, such as developing the backgammon game itself and making it multiplayer. But, I suspect I won't ever come back to this module because the other part of the rendering module includes piece animations. This comes last in my list of things to develop and, given time constraints, I will not be able to implement everything. I am very behind schedule and will need to do as much as I can before the development section deadline.

Server-side Room Handling

I need to set up a way for the server to store multiple rooms, boards and users and handle the logic for them simultaneously. This module is nearly entirely server-side and I will need to set up multiple procedures on the server to check things like whether a room exists or not, creating a waiting room, destroying a waiting room, creating a room (and setting up a board for that room), joining a room and sending room codes, board data and player data to the client. I also need procedures to update the client UI when they join the room (or fail to join a room, if the room doesn't exist). This section will primarily deal with the client creating, joining and leaving a room and will have nothing to do with the server actions during a backgammon game. The section will tackle the decomposed modules shown below.



Iteration 1

The aim of this iteration is to create server procedures to create a waiting room and send the code back to the client. If the waiting client disconnects, I need a procedure to destroy the waiting room. Then, I need to create a procedure that checks whether the waiting room exists. If it exists, the client will send a create room request, which is a procedure that attempts to create the room and redirect both the joining and waiting client to the game room. When this happens, I need some sort of verification function to check whether the clients are allowed in the game room. If they are allowed, game data should be sent to both clients and then they can update their respective UIs.

Code

main.py handle_create_waiting_room()

```
# Listens for when a client creates a waiting room
@socketio.on("create_waiting_room")
def handle_create_waiting_room(username):
    # Generates a unique random 6 digit code
    code = random.randint(100000, 999999)
    while code in waiting_rooms:
        code = random.randint(100000, 999999)

    # Creates a waiting room and user
    waiting_rooms[code] = request.sid
    users[request.sid] = {"sid": request.sid, "username": username}
    print(f"{request.remote_addr} has created a room with code {code}")

    # Sends the room code back to the client
    emit("receive_room_code", f"{code}")
```

main.py handle_create_waiting_room() is a procedure that runs when the client requests to create a waiting room, which happens when they click on the create room button. A random 6 digit code is generated, the SID (socket id, which is a unique string associated with each socket connection) is stored, the users dictionary is updated with the sid and username and the server sends the code back to the client.

I need a function to create a waiting room so the user that's trying to create a room has a way to wait for the opponent to join. I generate a random 6 digit code because that allows 1,000,000 possible simultaneous games running at the same time, which will never happen. But it allows multiple rooms to exist at the same time. It creates a waiting room with the key value as the code. As a result, it will be very easy to check whether a code is in the waiting room dictionary or not and, being a dictionary, checking whether an element exists is very fast and runs in linear time.

main.py handle_disconnect()

```
# Listens for when a client disconnects
@socketio.on("disconnect")
def handle_disconnect():
    print(f"{request.remote_addr} has disconnected!")

    # Removes the waiting room the client created, if they created one
    for room in waiting_rooms:
        if users[waiting_rooms[room]]["sid"] == request.sid:
            waiting_rooms.pop(room, None)
            break
```

main.py handle_disconnect() is a procedure that runs when the disconnect event, a built-in flask event that runs when a client disconnects, occurs. When the client disconnects, the procedure iterates through every waiting room, which has an associated SID. If the waiting room's associated SID is equal to the SID of the client that's disconnecting, the waiting room will be destroyed.

The main reason why I have to destroy waiting rooms is because if a client connects to a waiting room where the waiting client has disconnected, a game will load with one opponent only. This will certainly cause some errors.

main.py handle_create_room()

```python
# Listens for when a client creates a room
@socketio.on("create_room")
def handle_create_room(code, username):
    # Creates a user
    users[request.sid] = {"sid": request.sid, "username": username}

    # Creates a game room
    code = int(code)
    game_rooms[code] = {}

    # Randomly assigns white and black to the connected users
    if random.randint(0, 1):
        game_rooms[code]["white"] = request.sid
        game_rooms[code]["black"] = waiting_rooms[code]
    else:
        game_rooms[code]["white"] = waiting_rooms[code]
        game_rooms[code]["black"] = request.sid

    # Board data
    game_rooms[code]["board"] = [
        {"count": 2, "color": "black"},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 5, "color": "white"},
        {"count": 0, "color": None},
        {"count": 3, "color": "white"},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 5, "color": "black"},
        {"count": 5, "color": "white"},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 3, "color": "black"},
        {"count": 0, "color": None},
        {"count": 5, "color": "black"},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 0, "color": None},
        {"count": 2, "color": "white"}
    ]

    # Tells the clients to join the room the server set up
    emit("join_room", request.sid)
    emit("join_room", waiting_rooms[code], to=users[waiting_rooms[code]]["sid"])
```

main.py handle_create_room() is a procedure that runs when the client sends a request to the server to create a room. First of all, the client is added to the users dictionary. Then, a

game room is created, which is a dictionary with attributes white, black and board. White and black are randomly assigned the two players in the room and the board is the starting position of a backgammon game. Finally, the server sends a command to both clients in the room to join the game.

This function is necessary as it allows a way for two people to play online Backgammon against each other. I make the game room object a dictionary because it's structurally quite similar to the JSON object format, consisting of key-value pairs. This is also the reason for the structure of the board data. I feel like it would be easier to send this format of data to and from the client since the client (the Javascript) can more easily decode and comprehend the data. I also make use of the random module in Python to randomly assign the white and black pieces to the clients. This is because, if it was not random, it would consistently create an unfair advantage gameplay wise for one of the players.

main.py handle_receive_game_info()

```
# Receives game information
@socketio.on("receive_game_info")
def handle_receive_game_info(code, userid):
    # Checks if the code is a number
    try:
        code = int(code)
    except:
        emit("redirect_home")

    # Checks the code is valid and verifies the user can view the web page
    # Only the users who created and joined the room should be able to view the web page
    if code in game_rooms:
        if game_rooms[code]["white"] == userid or game_rooms[code]["black"] == userid:
            # Sets up game data
            game_data = {
                "color": "white",
                "white": users[game_rooms[code]["white"]]["username"],
                "black": users[game_rooms[code]["black"]]["username"],
                "board": game_rooms[code]["board"]
            }

        if game_rooms[code]["white"] == userid:
            # Updates the user sid and emits game data to the user
            users[userid]["sid"] = request.sid
            emit("game_info", game_data)

            # Emits to the opponent that the opponent has reconnected
            emit("opponent_reconnected", users[game_rooms[code]["white"]]["username"], to=users[game_room
        elif game_rooms[code]["black"] == userid:
            # Updates the user sid and emits game data to the user
            users[userid]["sid"] = request.sid
            game_data["color"] = "black"
            emit("game_info", game_data)

            # Emits to the opponent that the opponent has reconnected
            emit("opponent_reconnected", users[game_rooms[code]["black"]]["username"], to=users[game_room
        else:
            emit("redirect_home")
    else:
        emit("redirect_home")
```

main.py handle_receive_game_info() is a procedure that runs when a client requests game info. This happens as soon as a client connects to the game web page. By receiving the correct game info, the client can correctly display the proper UI and the state of the board. As it runs immediately when connecting to the web page, it can also be used for validation to check whether the client is even allowed to access the web page. Initially, it checks if the code is an integer, if the code exists and if the client (identified by a unique user id) is part of the room. If any of these are false, the server sends a redirect home command, which redirects the client back to the main menu. Otherwise, it will create game data that includes the client's color that they are, the username of white, the username of black and the board data, before sending it.

This function is necessary as I need a way to send data to the clients so they can update their UI. I have to do validation checks because, unfortunately, the user can type any url in their browser and might stumble upon a game room which they're not supposed to be part of. As a result, I have to assign a unique user id to each client to get their username and check which game, if any, they have access to. Game data is stored as a dictionary for ease of sending it to the client. I also emit an opponent reconnected signal to the other client in the room simply to update the other client's UI. I have code that adds "(disconnected)" next to the opponent's username if they disconnect. If they reconnect, this emitted signal removes this from the opponent's username.

main.py handle_disconnect()

```
# Emits an opponent_disconnected event to the opponent
for user_id in users:
    if users[user_id]["sid"] == request.sid:
        for game in game_rooms:
            if game_rooms[game]["white"] == user_id:
                emit("opponent_disconnected", to=users[game_rooms[game]["black"]]["sid"])
                break
            elif game_rooms[game]["black"] == user_id:
                emit("opponent_disconnected", to=users[game_rooms[game]["white"]]["sid"])
                break
        break
```

main.py handle_disconnect() is a procedure that runs when the disconnect event, a built-in flask event that runs when a client disconnects, occurs. This is a section of the procedure that handles sending an opponent disconnected signal to the other client. It will iterate through the users dictionary until it finds the client trying to disconnect. If the client is in a game room, it will do 2 things. If the client is white, it sends the opponent disconnected signal to black and vice versa.

This part of the code really isn't too important. This ensures the other client's UI gets updated and adds "(disconnected)" next to the opponent's username. This is just an aesthetic design choice and adds little functionality to the solution.

main.py exists()

```python
# Endpoint that checks whether a room, where an opponent is waiting, exists
@main.route("/exists/<string:room_code>")
def exists(room_code):
    if int(room_code) in waiting_rooms:
        return jsonify({"exists": True})
    else:
        return jsonify({"exists": False})
```

main.py exists() is a function that checks whether a waiting room with that room code exists. It simply checks whether the code is in the waiting room dictionary and returns either True or False to the client.

This function is required for the validation check that ensures that the room code a user has entered exists. The reason I check whether a waiting room exists and not a game room is because this endpoint is only used by the client when joining a room. Before joining a room, it must ensure that the room exists and that a client is waiting in a waiting room. At this point in the program, the game room hasn't been created yet, so you must check for waiting rooms.

menu.js

```javascript
// Creates a waiting room
socket.connect();

socket.emit("create_waiting_room", name);

// Updates UI upon receiving a room code
socket.on("receive_room_code", (room_code) => {
    code = room_code;
    document.getElementById("code").textContent = room_code;
});

// Leaves the waiting room and joins a room when other opponent joins
socket.on("join_room", (userid) => {
    sessionStorage.setItem("userid", userid);
    window.location.replace("/" + code);
});
```

menu.js is a Javascript file that adds functionality to the menu. This section of code runs when clicking the "create a room" button in the menu. It connects the socket. Then, it emits a create waiting room request to the server and sends its username. When this happens, it expects to receive a room code from the server once the server makes a waiting room. The procedure that deals with receiving a room code just updates the UI with the correct code. When the client receives a request to join the game room from the server, it will store its unique user id allocated by the server in session storage and redirect to the game web page.

This is required so that a code is displayed on the UI when creating a room, allowing another person to join the room. The reason the client creates a waiting room is to allow the opponent to join the room before starting the game. I update the UI with the correct code to

make the room code shareable with the opponent. Finally, I use session storage to store the unique user id. As a result, even if the wifi goes out or the browser temporarily disconnects, as long as the tab remains open, the user id is stored and the client can reconnect to the correct room at any time.

menu.js

```
// Checks if the room exists first
fetch(`exists/${code}`)
.then((response) => response.json())
.then((response_json) => {
    if (response_json.exists) {
        // Creates a game room
        socket.connect();

        socket.emit("create_room", code, name);

        // When the server sets up the game room, the client joins the game room
        socket.on("join_room", (userid) => {
            sessionStorage.setItem("userid", userid);
            window.location.replace("/" + code);
        });
    } else {
        showError("Room does not exist");
    }
})
```

menu.js is a Javascript file that adds functionality to the menu. This section of code runs when clicking the “join a room” button on the menu. It will send a post request to the server and check if the room code exists. If it doesn’t, an error message will appear stating that the room code does not exist. Otherwise, it will connect the socket and emit a create room request to the server with the code of the room and the client’s username. When the client receives a request to join the game room from the server, it will store its unique user id allocated by the server in session storage and redirect to the game web page.

This is required so that the program can check whether a room exists before connecting the user to a non-existent room. Also, this code allows the user to join a room, making online Backgammon functional. The reason it sends a post request to the server instead of establishing a socket and sending a request is simple. I thought that connecting a socket, sending one request, receiving one response and possibly being disconnected all in a few moments would be too time consuming to implement server-side. Furthermore, I’d have no idea how to handle rapid connections and disconnections and differentiate them from waiting room disconnections and game room disconnections. A post request was the much simpler approach. Again, I use session storage to store the unique user id. As a result, even if the wifi goes out or the browser temporarily disconnects, as long as the tab remains open, the user id is stored and the client can reconnect to the correct room at any time.

network.js

```
// Creates a socket
const socket = io();

// Requests information about the game from the server
// (what pieces are on the board, who's white, who's black)
socket.emit("receive_game_info", code, sessionStorage.getItem("userid"));

// Redirects to main menu if the server sends a redirect request
socket.on("redirect_home", () => {
    window.location.replace("/");
});
```

network.js is a Javascript file that runs only on the game web page. It creates a socket, receives game information from the server and has a procedure that redirects the client back to the main menu if the server requests the client to do so.

This code is necessary to let the server know that someone has connected to the room. When receiving game information, I have to send the code to the server so the server knows which game room data to look at and send back. I also have to send my user id to the server, to verify that the client is supposed to be able to access the web page.

network.js

```
// Processes game information from the server, updates UI and changes data in Board class
socket.on("game_info", (data) => {
    color = data["color"];
    white = data["white"];
    black = data["black"];

    // Changes player names in gameInfo section
    if (lang == "ar" || lang == "he") {
        document.getElementById("gameInfoWhite").textContent = white + " :" + langData["white"];
        document.getElementById("gameInfoBlack").textContent = black + " :" + langData["black"];
    } else {
        document.getElementById("gameInfoWhite").textContent = langData["white"] + ": " + white;
        document.getElementById("gameInfoBlack").textContent = langData["black"] + ": " + black;
    }

    // Changes player names around board
    if (color == "white") {
        document.getElementById("player1").textContent = black;
        document.getElementById("player2").textContent = white;
    } else {
        document.getElementById("player2").textContent = black;
        document.getElementById("player1").textContent = white;

        // Flips board vertically
        game.board.perspective = "black";
    }

    // Changes data in Board class
    game.board.board = data["board"];
});
```

network.js is a Javascript file that runs only on the game web page. This section deals with updating the client UI once game information has been received. It will first update variables black, white and color with usernames and what color the client is. It will then update the game info section with usernames, changing how text is concatenated depending on the language. It will then update the usernames around the board and, if the player is playing black, it will flip the board vertically. Finally, it loads the board data received from the server.

I have to update the client UI in order to make the game playable. Without this section, pieces would never be loaded onto a board and the player would be left staring at an empty board, unable to play. I also have to flip the board vertically if the player is playing black in order to make it easier to play and make moves.

network.js

```
// Procedure that updates UI when opponent disconnects and reconnects
socket.on("opponent_disconnected", () => {
    document.getElementById("player1").innerHTML = document.getElementById("player1").innerHTML + " <span
    style='color: red;'>(disconnected)</span>";
});
socket.on("opponent_reconnected", (username) => {
    document.getElementById("player1").textContent = username;
});
```

network.js is a Javascript file that runs only on the game web page. This section deals with updating the UI when the opponent disconnects and reconnects. If they disconnect, a red disconnected word appears next to the opponent's name. If they reconnect, the disconnected word disappears.

This isn't strictly necessary to the function of my solution and exists as a quality of life feature for the connected client. If the opponent hasn't played a move in a while, the connected client can check whether the opponent has disconnected or not.

Testing

Test	Req. tested	Expected result	Result	Remedial action
Creating a room	2.3	A joinable room should be created on the web server		None
Joining a room	2.4	A room that already exists should be joined, causing both players to load in the game		None
Sending game info	2.7	Game information should be sent to both clients once they've		None

		joined the room		
Updating client UI	3.1, 3.4	The client UI should update once game information has been received from the server		None
Disconnecting and reconnecting	2.4, 2.7	The client should be able to disconnect and reconnect to the same room without the room being disbanded immediately		None

Validation

Most of my solution is self-validating as the user clicks on predefined buttons that I have coded. However, while technically not part of my solution, the client can interact with the text input provided by their browser to navigate to any URL. This means they can navigate to any room they want, even rooms with non-numeric codes. As a result, I have implemented several validation checks - a numbers check to ensure that the code is only numbers, a check to ensure that a game with that code exists and a check to ensure that the user id of the client is in that game which verifies whether the client is allowed to play that specific game or not. The validation code, with unrelated details being omitted, will be pasted again for better clarity.

main.py

```
# Checks if the code is a number
try:
    code = int(code)
except:
    emit("redirect_home")

# Checks the code is valid and verifies the user can view the web page
# Only the users who created and joined the room should be able to view the web page
if code in game_rooms:
    if game_rooms[code]["white"] == userid or game_rooms[code]["black"] == userid: ···

    if game_rooms[code]["white"] == userid: ···
    elif game_rooms[code]["black"] == userid: ···
    else:
        emit("redirect_home")
else:
    emit("redirect_home")
```

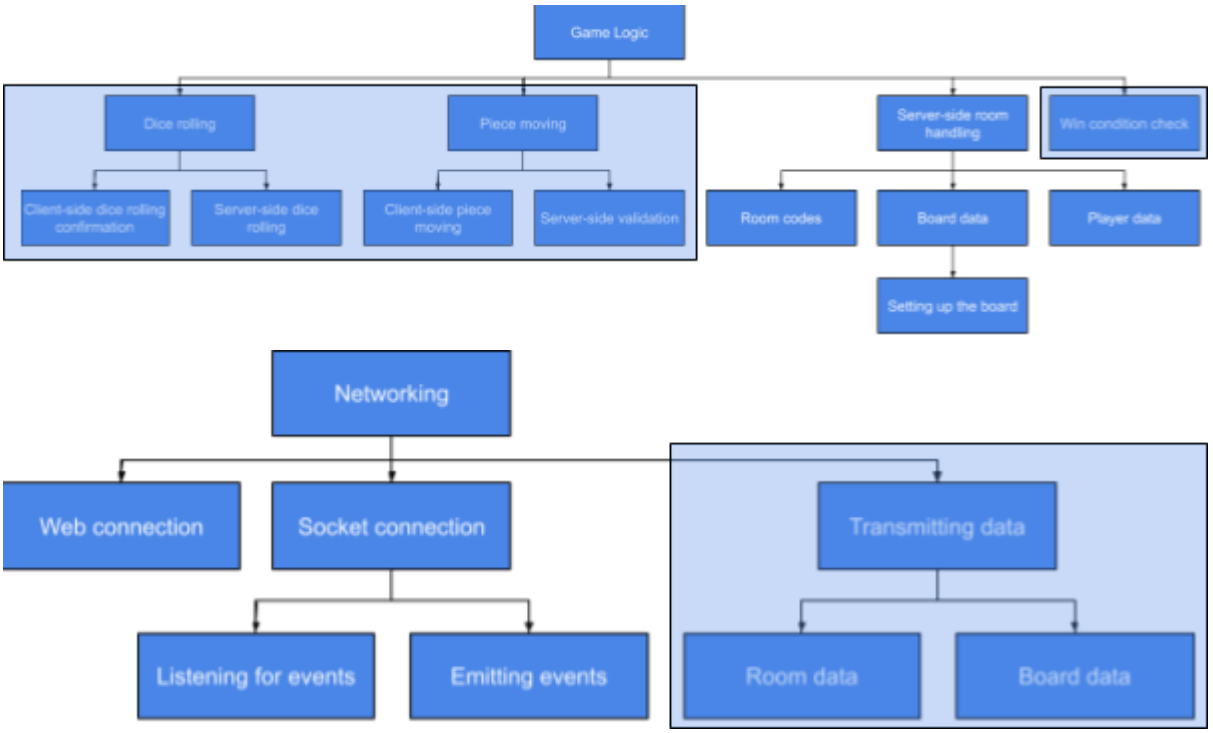
Review

So far, I've currently implemented a web server, a fully functional main menu, game web page that displays the backgammon board with pieces and a functional multiplayer

room-based system. I still have a lot left to do, such as developing the backgammon game itself. Funnily, this is the last module I can develop before the first development deadline and I haven't implemented backgammon yet, which will be my next course of action. But I feel like I am only 2-3 sizable modules away from finishing a functional project and, with any luck, I will be able to finish everything by the resubmission deadline. I am quite behind schedule and will need to do as much as I can before the development section resubmission deadline.

Backgammon Gameplay

I need to set up a way for the user to play Backgammon once they join a room. This module is a combination of client-side and server-side logic to get board moves and ensure that they are legal. I will need to set up a procedure to roll the dice, move the pieces, ensure that the piece movement is valid and check when someone has won. The section will tackle the decomposed modules shown below.



The reason why I decomposed the game like this is because I planned for the game logic modules to be client-side and the networking modules to be server-side. This is why similar modules are decomposed under different branches. However, I plan to tackle both at the same time because they are relatively similar and both modules work in tandem to create the piece moving and dice rolling functionality.

Iteration 1

The aim of this iteration is to create client procedures to figure out what the mouse clicks and what moves/actions the client makes. The aim is also to create server procedures that allow

the server to respond to these actions i.e. generate a random number when they roll the dice to check whether a move the client makes is valid or not. I also need to create these procedures in such a way that they use sockets to allow the client and server to communicate. For this iteration, I will only concern myself with dice rolling and piece moving only.

Code

game.js Dice constructor()

```
constructor() {
    //  dice roll start and stop are set to frame numbers that control when the animation plays
    this.dice1 = NaN;
    this.dice2 = NaN;
    this.activeDice = [];
    this.diceRollStart = NaN;
    this.diceRollStop = NaN;
    this.hideDice = true;
    this.hideRollDiceButton = true;

    this.rollDiceButtonImg = new Image();
    this.rollDiceButtonImg.src = "static/assets/rollDiceButton.png";
    this.diceImg = new Image();
    this.diceImg.src = "static/assets/dice.png";
}
```

game.js Dice constructor() is a function that sets the attributes for the dice object. It has values for the activeDice (actual dice values used for calculating valid moves), values for determining which dice faces are rendered, values for determining when animation plays and variables that determine the file locations of the assets.

I need to know the actual dice values and store it in a variable to determine the possible valid moves and ensure that the moves the client makes are legal. I need to know when the dice animation should start and stop to ensure the animation is consistent and that the animation plays for the same amount of time and the dice rolls at the same rate. I need to know whether to hide the dice or the button to control when, during a game, the player can roll the dice. Finally, file locations of assets are required to render them.

game.js Dice rollDice()

```
// Rolls the dice
// (visually changes the dice if final is false otherwise it actually changes)
rollDice(final=true) {
    if (final) {
        this.dice1 = this.activeDice[0];
        this.dice2 = this.activeDice[1];
    } else {
        let newDice1 = Math.floor(Math.random() * 6);
        let newDice2 = Math.floor(Math.random() * 6);

        while (newDice1 == this.dice1) {
            newDice1 = Math.floor(Math.random() * 6);
        }
        while (newDice2 == this.dice2) {
            newDice2 = Math.floor(Math.random() * 6);
        }

        this.dice1 = newDice1;
        this.dice2 = newDice2;
    }
}
```

game.js Dice rollDice() is a function that visually rolls the dice in two ways depending on whether the final attribute is set to true or false. If the final attribute is set to true, at this point the client will have received the dice roll from the server and the dice1 and dice2 are updated accordingly. Otherwise, it will set dice1 and dice2 to different random values. This allows the dice rolling animation to work.

This is a purely visual function because actual dice rolling happens on the server-side code. If dice rolling happened client-side, the client could cheat and set the dice variable to whatever they desired. I need to set both dice values to random values in order to create a dice rolling animation, which works by changing the value of the dice multiple times in a second. I also need to set both dice values equal to active dice (the dice received from the server) in order to let the client know what exactly they rolled.

game.js Dice animateDice()

```
// Animates the dice
animateDice() {
    if ((game.frame - this.diceRollStart) % 3 == 0 && game.frame >= this.diceRollStart && game.frame <= this.diceRollStop) {
        this.rollDice(false);
    }
    if (game.frame == this.diceRollStop + 1) {
        this.rollDice();
        console.log(game.dice.activeDice);
        console.log(game.validMoves);
    }
}
```

game.js Dice animateDice() is a function that visually rolls and animates the dice. It works simply by calling the rollDice function every few frames, creating a rolling effect. This is

unless the animation ends, where it instead calls the rollDice function for a final time with the final attribute set to true, updating the dice to the server-side dice values.

It needs to access the diceRollStart, diceRollStop and game.frame variables in order to figure out when it needs to animate the dice. This is because this function is called every frame. It also checks whether the current frame subtracted by the starting frame is a multiple of 3 to ensure the rollDice function is run every 3 frames. This is because, after some testing, a gap of 3 frames ensures the animation feels like a dice roll. However, it does not need to log values to the console. I added that however as it is immensely useful when developing and testing code.

game.js Dice displayDice()

```
// Renders the dice and the roll dice button
displayDice() {
    this.animateDice();

    if (!this.hideRollDiceButton) {
        ctx.drawImage(this.rollDiceButtonImg, pixel(126), pixel(217), pixel(169), pixel(59));
    }

    if (!this.hideDice) {
        ctx.drawImage(this.diceImg, 64 * this.dice1, 0, 64, 64, pixel(525), pixel(215), pixel(64), pixel(64));
        ctx.drawImage(this.diceImg, 64 * this.dice2, 0, 64, 64, pixel(605), pixel(215), pixel(64), pixel(64));
    }
}
```

game.js Dice displayDice() is a function that displays the dice. It calls the animateDice function and checks what to display. If the hideRollDiceButton variable is false, it displays the rollDiceButton. If the hideDice variable is false, It displays the dice.

This is a function that needs to be run every frame because it's drawing images to the screen and, if it's not run every frame, the images won't be visible every frame. It runs the animateDice function because that's a function that needs to be run every frame. It also needs to check whether to display the button and the dice. This is because, depending on the turn, the color of the client and whether the button has been pressed, the dice and the button may or may not be visible. An example of this is when it's the opponent's turn. In this case, the roll dice button should never be visible.

game.js Dice startRoll()

```
// Changes attributes and starts the roll when the roll dice button is clicked
// doServerStuff determines whether a roll_dice request should be sent to the server
startRoll(doServerStuff=true) {
    this.hideRollDiceButton = true;
    this.hideDice = false;
    this.diceRollStart = game.frame;
    if (doServerStuff) {
        this.diceRollStop = Number.POSITIVE_INFINITY;
        socket.emit("roll_dice", sessionStorage.getItem("userid"));
    } else {
        this.diceRollStop = game.frame + 29;
    }

}
```

game.js Dice startRoll() is a function that actually rolls the dice and doesn't do anything visually. It changes variables to hide the roll dice button, show the dice, start the animation and determine when the dice stops. It also has the ability to request a dice roll from the server.

This function is run when the roll dice button is pressed in order to make the button work. It hides the button because the client should not have the ability to roll the dice twice in one turn. It shows the dice so the client can see what dice he rolled. And it sends a request to the server because actual dice rolling happens server-side, to prevent cheating. The reason the doServerStuff attribute exists is because this function must be run by both clients in the game, so both clients can see the dice. However, only one server request needs to be sent. Sending two requests may cause problems. Finally, the reason I set the stopping frame to infinity is because , in some cases, the server may take longer than 30 frames to respond. If this were to happen, errors would occur. Thus, I set the stopping frame to infinity and later change the stopping frame when a response is received.

main.py handle_roll_dice()

```
# Rolls dice
@socketio.on("roll_dice")
def handle_roll_dice(userid):
    # Checks the userid is valid and retrieves the code of the game the user is in
    try:
        code = users[userid]["code"]
    except:
        emit("redirect_home")

    # Generates random dice
    active_dice = [random.randint(0, 5), random.randint(0, 5)]
    if active_dice[0] == active_dice[1]:
        active_dice += [active_dice[0], active_dice[0]]

    # Updates game dice values
    game_rooms[code]["activeDice"] = active_dice

    # Calculates valid moves
    calculateValidMoves(code)

    # Sends a response to the client
    emit("receive_roll_dice", active_dice, to=everyone_in_room(int(code)))
```

main.py handle_roll_dice() is a server-side function that rolls the dice. It checks that the userID of the request is valid (to check whether the client exists and is currently in a backgammon game) and retrieves the code of the game the client wants to roll the dice in. It then rolls the dice and stores it in an active_dice variable. If both dice rolls are the same, 2 more of the same dice rolls get appended to active_dice i.e. [2,2] becomes [2,2,2,2]. The dice rolls are stored, moves are calculated and a response is sent to everyone in the game room.

I need to check that the client exists and is currently in a backgammon game. This is because in order to roll the dice, the client must be part of a backgammon game. Otherwise, things might break. The reason the active_dice “doubles” when rolling a double (2 rolls of the same value) is because of Backgammon rules. If you roll a double, you should be able to move 4 times. Thus, by “doubling” the active_dice variable, the game thinks you’ve rolled 4 dice, allowing you to move 4 times. The dice values are stored because, if they weren’t, rolling dice would be useless. Also, a response is sent to all clients in the room because, if not, the clients would never know what the server rolled.

main.py calculateValidMoves() (first half)

```
def calculateValidMoves(code):
    # Get variables from the game
    game = game_rooms[code]
    active_dice = game["activeDice"]
    compound_move_distances = []
    game["validMoves"] = []

    # If there are no dice rolls remaining, there are no valid moves
    if len(active_dice) == 0:
        return

    #  Check if compound moves need to be checked (i.e. moving 7 when rolling 3 and 4)
    if len(active_dice) >= 2:
        if active_dice[0] != active_dice[1]:
            compound_move_distances.append(sum(active_dice) + 1)
        else:
            # If you roll [2, 2], compounds moves should be [4,6,8]
            # Since doubled dice gives you 4 moves
            for i in range(2, len(active_dice) + 1):
                compound_move_distances.append((active_dice[0] + 1) * i - 1)
```

main.py calculateValidMoves() is a function that calculates the valid moves. It gets variables such as the dice in play and the board data. If the length of the dice in play is 0, it returns no valid moves. Otherwise, if there are 2 or more dice, it calculates the compound move distances (i.e. it needs to check moving 7 spaces if you roll a 3 and a 4). Then, it checks valid moves for single dice rolls and valid moves for compound dice rolls.

The reason I need the first part of calculateValidMoves() is to calculate compound move distances. Originally, I did not want to implement compound moves because every compound move can be made with 2 simple moves. (instead of moving 7 spaces, just move 3 spaces then 4 spaces). Furthermore, as I found out during development, they are complex and frustrating to implement. Unfortunately, my stakeholder threatened to cut funding for this project if this feature was not implemented. He argued that being able to visualise two moves in one turn is incredibly useful for beginners just starting out in Backgammon. Defeatedly, I implemented this feature.

main.py calculateValidMoves() (second half)

```
    # Calculates whether non compound moves are legal
    for i, point in enumerate(game["board"]):
        # Goes through every point of the same color of whoever's turn it is
        if point["color"] == game["turn"]:
            # Goes through every dice roll
            for dice in game["activeDice"]:
                endpoint = i - (dice + 1) if game["turn"] == "white" else i + (dice + 1)

                # If piece move does not go off board and the piece moves to a legal spot
                # add move to validMoves
                if endpoint in range(24):
                    if game["board"][endpoint]["count"] < 2 or game["board"][endpoint]["color"] == game["turn"]:
                        move = {"from": i, "to": endpoint, "dice": [dice]}

                        if move not in game["validMoves"]:
                            game["validMoves"].append(move)

    # Checks whether compound moves are legal
    if len(active_dice) >= 2:
        # Goes through every movable point of whoever's turn it is
        for i in [move["from"] for move in game["validMoves"]]:
            # Goes through every compound dice roll
            for distance in compound_move_distances:
                endpoint = i - (distance + 1) if game["turn"] == "white" else i + (distance + 1)

                # If piece move does not go off board and the piece moves to a legal spot
                # add move to validMoves
                tmp = len(game["validMoves"])
                if endpoint in range(24):
                    if game["board"][endpoint]["count"] < 2 or game["board"][endpoint]["color"] == game["turn"]:
                        # stupidly long line just figures out dice required
                        # made complex because of doubled dice
                        move = {"from": i, "to": endpoint, "dice": active_dice if active_dice[0] != active_dice[1]
                                else [active_dice[0] for j in range((distance + 1) // (active_dice[0] + 1))]}

                        if move not in game["validMoves"]:
                            game["validMoves"].append(move)

                if tmp == len(game["validMoves"]):
                    break
```

I need to check each point of the same color of whoever's turn it is because any valid moves will only start on that set of points. The endpoint variable (determines the final point that the piece moves to) is important because I need to check whether the count is less than 2 or the point is of the same color as the moving piece. These are required checks to adhere to the rules of backgammon. I ensure the validMove variable contains unique elements only. If I didn't, rendering issues might occur because, when it comes to visualising validMoves (highlighting the points the player can move to), it might highlight the same point or piece multiple times. I perform similar checks and validation on compound moves once simple moves have been calculated and validated, in order to allow both simple and compound moves to be played.

main.py everyone_in_room()

```
# Gets session ids of everyone in a room
def everyone_in_room(code):
    return [users[game_rooms[code]["white"]]["sid"], users[game_rooms[code]["black"]]["sid"]]
```

main.py everyone_in_room() is a simple helper function that returns the session ids of all users in the specific room.

Flask uses the session id to communicate with a specific client only. Thus, in order for the server to communicate information like a move or a dice_roll to everyone in the room, it needs both session ids of both users in the room.

network.js

```
// Procedure that runs when server responds with a dice roll
socket.on("receive_roll_dice", (activeDice) => {
    game.dice.activeDice = activeDice;
    if (game.frame < game.dice.diceRollStop) {
        game.dice.diceRollStop = game.dice.diceRollStart + Math.max(29, Math.ceil((game.frame - game.dice.diceRollStart) / 3 + 0.1) * 3 - 1)
    } else {
        game.dice.startRoll(false);
    }

    socket.emit("receive_game_info", sessionStorage.getItem("userid"));
});
```

network.js is a Javascript file that runs only on the game web page. This section deals with when the server responds with a dice roll. It checks whether you rolled the dice. If you rolled the dice, your diceRollStop is currently set to infinity (as explained earlier in the startRoll() function). Once a response is received, it calculates what frame to stop the dice roll. If you never rolled the dice, it rolls the dice. That way, when the opponent rolls the dice, you can see the dice rolling animation once a response is received. It then requests game info from the server to update the board, the dice, the possible valid moves etc.

The reason why this function exists is mainly to let the non-rolling player know that the opponent is rolling. In a real backgammon game, both players are able to see the rolling dice and I wanted to imitate this in my online version. I check whether the frame is less than the stopping animation frame. This allows me to check whether a dice rolling animation is currently playing, allowing me to figure out who rolled the dice. Finally, I send a receive_game_info request to receive the possible valid moves that the roller can make.

network.js

```
game.dice.activeDice = data["activeDice"];
game.validMoves = data["validMoves"];

if (game.dice.activeDice.length == 0) {
    game.dice.dice1 = NaN;
    game.dice.dice2 = NaN;
}

if (color != game.turn && game.dice.activeDice.length == 0) {
    game.dice.hideRollDiceButton = true;
} else if (game.dice.activeDice.length > 0) {
    try {
        game.dice.dice1 = game.dice.activeDice[0];
    } catch {}
    try {
        game.dice.dice2 = game.dice.activeDice[1];
    } catch {}
    game.dice.hideRollDiceButton = true;
    game.dice.hideDice = false;
}
```

Note that the handle_receive_game_info() function in main.py is changed ever so slightly as it now responds with the dice in play, the turn and the valid moves. This was only 3 extra lines of code and thus I didn't think it was worth rewriting an explanation and justification.

network.js is a Javascript file that runs only on the game web page. This section only deals with changing variables to change how the dice is rendered to the screen. It follows a set of rules and checks whose turn it is and whether there are any dice left in play to determine whether the dice are displayed or the button is displayed.

This is required in order to prevent actions like rolling the dice twice in the same turn and correctly displaying the dice to allow both players to see it when appropriate. Furthermore, it prevents dumb bugs like displaying dice when there are no dice to display by using try catch statements.

game.js Board renderPointHighlight()

```
// Renders point highlights
renderPointHighlight(point) {
    let intervalsTop = [0, 56, 111, 167, 224, 280];
    let intervalsBottom = [0, 57, 113, 171, 226, 281];
    let x, y;

    // Calculates x and y values
    if (point >= 12 && point <= 17) {
        x = 66 + intervalsTop[point - 12];
    } else if (point >= 18 && point <= 23){
        x = 458 + intervalsTop[point - 18];
    } else if (point >= 6 && point <= 11) {
        x = 66 + intervalsBottom[11 - point];
    } else {
        x = 458 + intervalsBottom[5 - point];
    }

    if (point >= 12) {
        y = 199;
    } else {
        y = 297;
    }

    // Draws a yellow triangle outline
    ctx.beginPath();
    ctx.moveTo(pixel(x), pixel(y));
    ctx.lineTo(pixel(x - 18), pixel(y - 180 * (point >= 12 ? 1 : -1)));
    ctx.lineTo(pixel(x + 18), pixel(y - 180 * (point >= 12 ? 1 : -1)));
    ctx.closePath();
    ctx.lineWidth = 3;
    ctx.strokeStyle = "#FFFF00";
    ctx.stroke();
}
```

game.js Board renderPointHighlight() is a function that renders the highlighting of points. It calculates the x coordinate of the point, the y coordinate of the point, checks whether the point is upright or upside down and then uses the canvas rendering context to draw a yellow triangle outline.

This function is necessary because points are highlighted to show the user the valid points they can move to. This can improve the experience of beginners getting into backgammon as they often get confused which way to move the pieces or move the pieces an incorrect amount of spaces. Unfortunately, unlike chess or checkers, backgammon is not a grid. As a result, calculating the positions of pieces is needlessly complicated and requires a lot of if statements and arbitrary pixel values. This is why I use many if and else statements.

game.js handleClick()

```
// Handles what actions are performed depending on what the user clicked
if (selectedPoint == null) {
    if (point != null) {
        if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(point)) {
            // Selects piece only if the piece has a valid move
            selectedPoint = point;
        }
    }
} else {
    if (point != null && point != selectedPoint && game.validMoves.reduce((acc, move) => acc = acc ||
    (move.from == (["whiteBar", "blackBar"].includes(selectedPoint) ? null : selectedPoint) && move.to == point), false))
        // Moves only if a point was selected and move is legal
        socket.emit("make_move", sessionStorage.getItem("userid"), {from: selectedPoint, to: point});
        selectedPoint = null;
        return;
    }

    // Unselects the piece
    selectedPoint = null;

    // Reselects another piece if the user unselected the piece by selecting another valid piece
    if (point != null) {
        if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(point)) {
            selectedPoint = point;
        }
    }
}
```

game.js handleClick() is a function that runs every time the user clicks. This is just a small part of the function. It is important to know that previously, point is either null, an integer between 0 and 23 or “whiteBar” or “blackBar”. This variable refers to which point or what the user is clicking. I omitted the previous part of the code because it was just a lot of if statements to figure out what the user was clicking and, in my opinion, the code is straightforward and self-explanatory. Regardless, this function selects pieces with a valid move if they aren’t currently selecting anything. If they are currently selecting something, it checks if the move is valid (the really long reduce statement is a one liner that just iterates through all validMoves and returns true if the move you’re trying to make is in validMoves). If the move is valid, it sends a move request to the server.

This function is necessary as it is needed to figure out when the user makes a move. Without this function, when the user clicks on a point, we don’t know if the user is selecting that point or moving to that point. Thus, we need two variables: selectedPoint (what’s currently selected) and point (what the user is going to select). All the checks made to ensure the client makes a valid move are required. This is because, even though all of these checks also happen server-side, it ensures that the server doesn’t waste processing time checking obviously illegal moves.

main.py handle_make_move()

```python
# Makes a move
@socketio.on("make_move")
def handle_make_move(userid, move):
    # Checks the userid is valid and retrieves the code of the game the user is in
    try:
        code = users[userid]["code"]
    except:
        emit("redirect_home")

    # Checks whether move is in valid moves
    for valid_move in game_rooms[code]["validMoves"]:
        if valid_move["from"] == move["from"] and valid_move["to"] == move["to"]:
            board = game_rooms[code]["board"]
            turn = game_rooms[code]["turn"]

            # Moves a piece from one point to another
            if board[move["from"]]["count"] > 0 and board[move["from"]]["color"] == turn:
                board[move["from"]]["count"] -= 1

                if board[move["from"]]["count"] == 0:
                    board[move["from"]]["color"] = None

                board[move["to"]]["count"] += 1
                if board[move["to"]]["color"] == None:
                    board[move["to"]]["color"] = turn

                # Captures piece
                if board[move["to"]]["count"] == 2 and board[move["to"]]["color"] != turn:
                    board[move["to"]]["count"] = 1
                    board[move["to"]]["color"] = turn

            # Removes dice rolls required to make the move
            for n in valid_move["dice"].copy():
                game_rooms[code]["activeDice"].remove(n)

            # Calculates valid moves
            calculateValidMoves(code)

            # If no valid moves, remove all dice rolls to make the game think your turn is over
            if len(game_rooms[code]["validMoves"]) == 0:
                game_rooms[code]["activeDice"] = []

            # Switch turn
            if len(game_rooms[code]["activeDice"]) == 0:
                game_rooms[code]["turn"] = "white" if turn == "black" else "black"

            break

    # Sends a response to the client
    emit("receive_make_move", to=everyone_in_room(int(code)))
```

main.py handle_make_move() is a server-side function that makes a move. It checks that the userID of the request is valid (to check whether the client exists and is currently in a

backgammon game) and retrieves the code of the game the client wants to roll the dice in. It then checks if the move is in validMoves. If so, it changes the board variable to reflect this move (and possible capture). It then calculates valid moves. If there are no more valid moves, it wipes the dice rolls left in play. Thus, when the game checks whether to switch turns by checking the number of dice left in play, it switches turns.

This function is necessary because a fundamental aspect of playing Backgammon is moving the pieces. I need to check that the client exists and is currently in a backgammon game. This is because in order to roll the dice, the client must be part of a backgammon game. Otherwise, things might break. I have to check whether the move is in validMoves. Otherwise, the client can send any move via Javascript code, illegal or not, and it would be played. I remove dice rolls from dice in play because, if I didn't, the user could play an infinite amount of moves. I wipe the dice rolls left in play if there are no valid moves because, if I didn't, the turn wouldn't switch and zero valid moves results in a softlock. Finally, I check the amount of dice rolls left in play before deciding whether to switch. This is how the rules of Backgammon work.

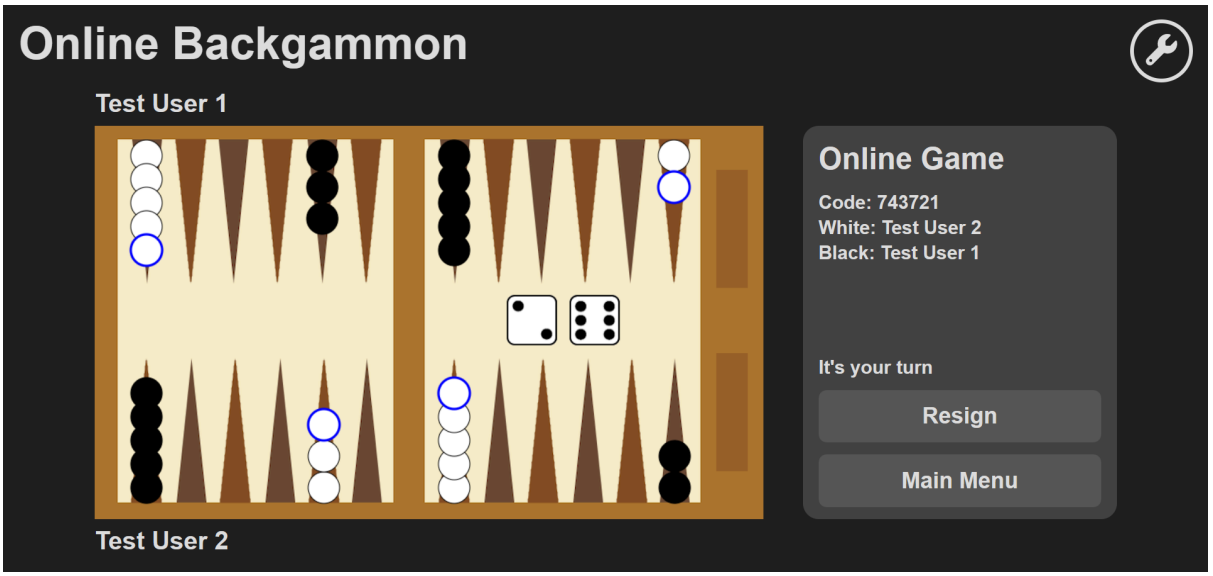
network.js

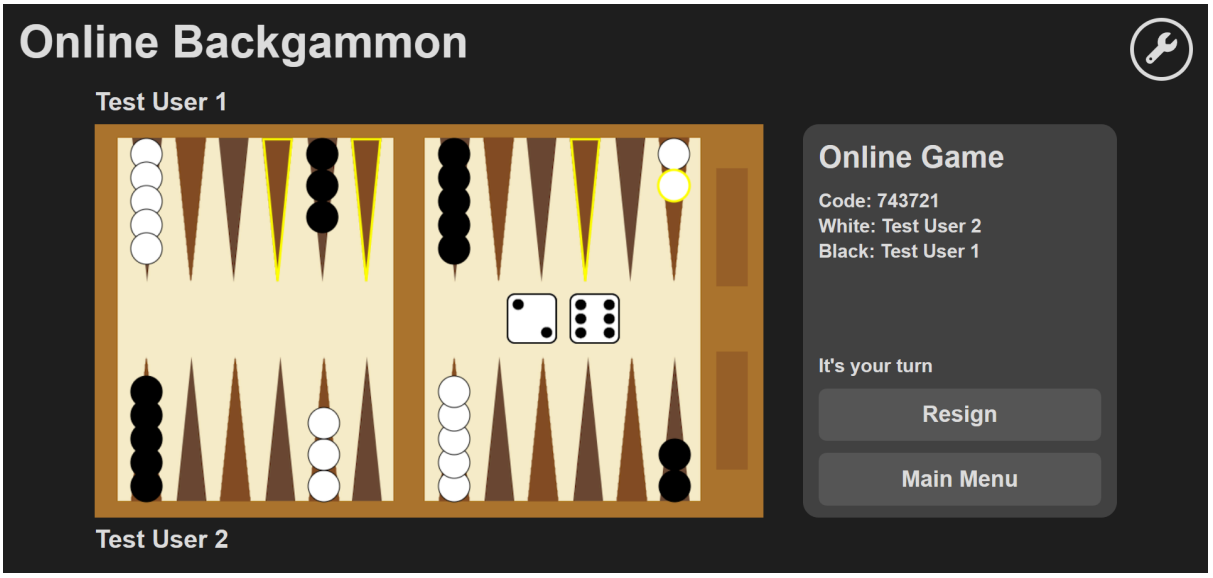
```
// Procedure that runs when server responds with a piece move
socket.on("receive_make_move", () => {
    socket.emit("receive_game_info", sessionStorage.getItem("userid"));
});
```

network.js is a Javascript file that runs only on the game web page. This section of code deals with receiving a make_move response from the server. In this case, it simply sends a request to the server to update the board.

The reason I update the board like this is because I already have server-side and client-side functions to receive board info and sync the client board to the server board. As a result, I thought it was unnecessary to write new code and functions when I can use pre existing functions. While I save development time, unfortunately, making a move and syncing the board now requires two requests to the server instead of one, which ultimately slows down the time taken for the board to update when making a move. Luckily, since server requests are already incredibly quick (maybe tens of milliseconds), I felt like this was a decent compromise.

GUI Output





Testing

Test	Req. tested	Expected result	Result	Remedial action
Dice Rolling	1.3, 2.5, 2.7	Making sure the user can roll the dice in order to play the game.		None
Piece moving	1.3, 2.6, 2.7	Making sure the user can move a piece in order to play the game.		While piece moving does work, not all of the possible moves in

				Backgammon have been implemented, such as properly capturing moves and bearing off pieces - need to implement all possible moves
Win condition checking	2.7, 2.8	Making sure the server is capable of recognising when a player has won or not in order to end the game.		Needs to be implemented

Iteration 2

The aim of this iteration is to create even more procedures, both client-side and server-side, in order to implement all possible types of moves that can be made in Backgammon. So far, I have implemented simple moves that work when pieces stay on the board. However, I have not implemented moves that happen when a piece moves off the board, either through capture or bearing off, or when a piece moves onto the board. In order to implement this, I need to expand on my move selection and allow the user to select pieces off the board. I also need to change my calculateValidMoves() function as well as the handle_make_move() function, both in main.py, to allow these new types of moves to be calculated as legal and played on the board.

Code

game.js Board renderPieces() (first half)

```
// Calculates where each piece should be and renders them
renderPieces() {
    // Renders black bar pieces
    for (let i = 0; i < game.blackData.onBar; i++) {
        let x = 400;
        let y = 158 + i * Math.min(40, 40 / (game.blackData.onBar - 1));
        if (i == game.blackData.onBar - 1 && selectedPoint == "blackBar") {
            this.renderPiece(x, y, "black", 1);
            for (let move of game.validMoves) {
                if (move.from == null) {
                    this.renderPointHighlight(move.to);
                }
            }
        // Long if statement just checks if a valid move exists that starts from the black bar
        } else if (i == game.blackData.onBar - 1 && selectedPoint == null &&
            Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) &&
            game.turn == "black") {
            this.renderPiece(x, y, "black", 2);
        } else {
            this.renderPiece(x, y, "black");
        }
    }

    // Renders white bar pieces
    for (let i = 0; i < game.whiteData.onBar; i++) {
        let x = 400;
        let y = 340 - i * Math.min(40, 40 / (game.whiteData.onBar - 1));
        if (i == game.whiteData.onBar - 1 && selectedPoint == "whiteBar") {
            this.renderPiece(x, y, "white", 1);
            for (let move of game.validMoves) {
                if (move.from == null) {
                    this.renderPointHighlight(move.to);
                }
            }
        // Long if statement just checks if a valid move exists that starts from the black bar
        } else if (i == game.whiteData.onBar - 1 && selectedPoint == null &&
            Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) &&
            game.turn == "white") {
            this.renderPiece(x, y, "white", 2);
        } else {
            this.renderPiece(x, y, "white");
        }
    }
```

game.js Board renderPieces() is a function that reads the board data, calculates the coordinates of each piece using some math and renders each piece using the renderPiece() function. This part of the function deals with rendering new parts of the board required for other backgammon moves, which are the bar (where pieces go when they're captured) and the borne off area (where pieces go when they're taken off). Depending on the number of pieces in the bar and the index of the piece, the code calculates where to render the piece. It then checks whether it's selected or whether it's a selectable move and highlights the piece accordingly. If it's a selected piece, it highlights the points where the piece can move to.

Rendering bar pieces is essential because the player needs the ability to see whether he has pieces on the bar. This is part of the rules of Backgammon. If the player makes a regular

move while not realising he has pieces on the bar, the player is making an illegal move. While I have two very similar pieces of code (dealing with the black and white bar), I couldn't put the code in a function. While using functions make some code reusable and shortens the length of the program, both the black and white bar require different coordinate values and different color values ("white" instead of "black", "whiteBar" instead of "blackBar"). Overall, I determined that it would make the code less readable and wouldn't really shorten the length of the program.

game.js Board renderPieces (second half)

```
// Renders white borne off pieces
if (game.whiteData.borneOff > 0) {
    ctx.fillStyle = "#ffffff";
    ctx.beginPath();
    ctx.rect(pixel(792), pixel(439 - 10 * game.whiteData.borneOff), pixel(40), pixel(10 * game.whiteData.borneOff));
    ctx.closePath();
    ctx.fill();
}
// Renders black borne off pieces
if (game.blackData.borneOff > 0) {
    ctx.fillStyle = "#000000";
    ctx.beginPath();
    ctx.rect(pixel(792), pixel(56), pixel(40), pixel(10 * game.blackData.borneOff));
    ctx.closePath();
    ctx.fill();
}
```

This part of the code calculates the size of the rectangle to render, depending on the amount of pieces borne off. The rectangle represents a top down view of placing pieces into the area off the board and makes more sense when looking at the GUI output of this iteration. This rendering is necessary to give an idea of how close either player is to winning. If the rectangle is at the max size, it means that all pieces have been taken off and someone has won.

game.js handleClick()

```
let onWhiteBar = x >= pixel(380) && y >= pixel(280) && x <= pixel(420) && y <= pixel(360);
let onBlackBar = x >= pixel(380) && y >= pixel(138) && x <= pixel(420) && y <= pixel(218);

let onWhiteHole = x >= pixel(792) && y >= pixel(289) && x <= pixel(832) && y <= pixel(439);
let onBlackHole = x >= pixel(792) && y >= pixel(56) && x <= pixel(832) && y <= pixel(206);

// Handles what actions are performed depending on what the user clicked
if (selectedPoint == null) {
    if (point != null) {···
    } else if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) && onWhiteBar &&
        selectedPoint = "whiteBar";
    } else if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) && onBlackBar &&
        selectedPoint = "blackBar";
    }
} else {
    if (point != null && point != selectedPoint && game.validMoves.reduce((acc, move) => acc = acc || (move.from == (["whiteBar
    } else if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].to).includes(null) && (onWhiteHole &&
        socket.emit("make_move", sessionStorage.getItem("userid"), {from: selectedPoint, to: null});
        selectedPoint = null;
        return;
    }
    if (point != null && point == selectedPoint || point == null && ["whiteBar", "blackBar"].includes(selectedPoint)) {
        selectedPoint = null;
        return;
    }

    // Unselects the piece
    selectedPoint = null;

    // Reselects another piece if the user unselected the piece by selecting another valid piece
    if (point != null) {···
    } else if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) && onWhiteBar &&
        selectedPoint = "whiteBar";
    } else if (Array.from({length: game.validMoves.length}, (_, i) => game.validMoves[i].from).includes(null) && onBlackBar &&
        selectedPoint = "blackBar";
    }
}
```

game.js handleClick() is a function that runs every time the user clicks. This is just a small part of the function. The screenshot includes previously shown repeated code so some parts have been omitted for better clarity. Variables have been added to check whether the mouse is hovering over the hole or the bar. When the user selects something, the code checks whether they're selecting the bar and whether the bar has legal moves. When the user plays a move, the code also checks whether they're taking a piece off. When reselecting pieces, the same thing occurs.

The reason this function needs to be expanded is because, previously, the function only looked at clicks on a point. However, valid moves can happen where the user doesn't click on points i.e. the user clicks on the holes or the bar. As a result, I need extra if statements to check whether this is the case. I also create variables onWhiteHole, onBlackHole, onWhiteBar and onBlackBar because the boolean comparisons are quite long and I use them frequently throughout this code.

main.py calculateValidMoves() (first part)

```
# Checks whether the player can bear off pieces
low = None
high = None
for i, point in enumerate(game["board"]):
    if point["color"] == turn:
        if low == None:
            low = i
        high = i
if (high <= 5 and turn == "white") or (low >= 18 and turn == "black"):
    canBearOff = True

# Handles weird rule in backgammon
# i.e. when white can bear off and he rolls a 6 but all pieces are on positions 0-4, he must bear off a piece from 4
# https://www.gnu.org/software/gnubg/manual/html_node/Bearing-Off.html
for dice in active_dice:
    if turn == "white":
        if dice > high:
            move = {"from": high, "to": None, "dice": [dice]}
            if move not in game["validMoves"]:
                game["validMoves"].append(move)
    else:
        if 23 - dice < low:
            move = {"from": low, "to": None, "dice": [dice]}
            if move not in game["validMoves"]:
                game["validMoves"].append(move)
```

main.py calculateValidMoves() is a function that calculates the valid moves. This part of the function deals with figuring out if the player can bear off and handles a weird rule that's a little difficult to explain. Essentially, if you can bear off and there are no checkers on the same or higher-numbered points, you must bear off the checker on the highest-numbered point.

This part of the code is necessary in order to follow the rules of backgammon. Furthermore, without working out whether a player is allowed to bear off, the player will never be able to bear off pieces and finish the game. I set up two variables (high and low) representing the highest and lowest point a player's piece is on. Comparing these variables to the dice, as well as knowing whose turn it is, is necessary as it is used to figure out whether a player can bear off pieces or not.

main.py calculateValidMoves() (second part)

```
# Checks if there are no pieces on bar
if (turn == "black" and users[game["black"]]["onBar"] == 0) or (turn == "white" and users[game["white"]]["onBar"] == 0):
    for i, point in enumerate(game["board"]):
        # Goes through every point of the same color of whoever's turn it is
        if point["color"] == game["turn"]:
            # Goes through every dice roll
            for dice in game["activeDice"]:
                endpoint = i - (dice + 1) if game["turn"] == "white" else i + (dice + 1)

                # Checks if piece goes to a legal spot
                if (endpoint in range(24) and canBearOff == False) or (endpoint in range(-1, 25) and canBearOff == True):
                    # Checks if piece isn't taken off and adds it to validMoves
                    if endpoint not in [-1, 24]:
                        if game["board"][endpoint]["count"] < 2 or game["board"][endpoint]["color"] == game["turn"]:
                            move = {"from": i, "to": endpoint, "dice": [dice]}

                            if move not in game["validMoves"]:
                                game["validMoves"].append(move)
                    # Checks if piece is taken off and adds it to validMoves
                    # Sets "to" attribute to None to indicate the piece is being taken off
                    else:
                        move = {"from": i, "to": None, "dice": [dice]}

                        if move not in game["validMoves"]:
                            game["validMoves"].append(move)
else:
    # If pieces are on active bar, you only need to moves from the bar
    for dice in game["activeDice"]:
        endpoint = 24 - (dice + 1) if game["turn"] == "white" else dice

        # If piece move does not go off board and the piece moves to a legal spot
        # add move to validMoves
        if endpoint in range(24):
            if game["board"][endpoint]["count"] < 2 or game["board"][endpoint]["color"] == game["turn"]:
                move = {"from": None, "to": endpoint, "dice": [dice]}

                if move not in game["validMoves"]:
                    game["validMoves"].append(move)
```

This part of the function deals with calculating simple valid moves. It's very similar to the first iteration but now implements edge cases for when moving from the bar or taking off pieces. If there are no pieces on the bar, it acts very similarly to the first iteration. Otherwise, the function only needs to move at valid moves coming from the bar.

This function needed to be expanded in order to calculate other valid moves that didn't just take place entirely on the board, such as taking off pieces and moving pieces onto the board. This is required as it's part of the rules of Backgammon. I have to check whether there are pieces on the bar initially. This is because whether pieces are on the bar heavily affects which moves are legal. For example, if there are pieces on the bar, a vast majority of moves become illegal.

main.py calculateValidMoves() (third part)

```
# Checks whether compound moves are legal by checking length of active dice and number of pieces on bar
if len(active_dice) >= 2 and (turn == "black" and users[game["black"]]["onBar"] <= 1) or (turn == "white" and users[game["white"]]["onBar"] <= 1):
    # Goes through every movable point of whoever's turn it is
    for i in [move["from"] for move in game["validMoves"]]:
        # Goes through every compound dice roll
        for distance in compound_move_distances:
            i = 24 if i == None and turn == "white" else (-1 if i == None and turn == "black" else i)
            endpoint = i - (distance + 1) if game["turn"] == "white" else i + (distance + 1)

            # Checks if piece goes to a legal spot
            tmp = len(game["validMoves"])
            if (endpoint in range(24) and canBearOff == False) or (endpoint in range(-1, 25) and canBearOff == True):
                # Checks if piece isn't taken off and adds it to valid moves
                if endpoint not in [-1, 24]:
                    if game["board"][endpoint]["count"] < 2 or game["board"][endpoint]["color"] == game["turn"]:
                        # Stupidly long line just figures out dice required
                        # Made complex because of doubled dice
                        move = {"from": None if i == 24 or i == -1 else i, "to": endpoint, "dice": active_dice if active_dice[0] != active_dice[1]
                                else [active_dice[0] for j in range((distance + 1) // (active_dice[0] + 1))]}

                        if move not in game["validMoves"]:
                            game["validMoves"].append(move)
                # Checks if piece is taken off and adds it to validMoves
                # Sets "to" attribute to None to indicate the piece is being taken off
                else:
                    move = {"from": None if i == 24 or i == -1 else i, "to": None, "dice": active_dice if active_dice[0] != active_dice[1]
                            else [active_dice[0] for j in range((distance + 1) // (active_dice[0] + 1))]}

                    if move not in game["validMoves"]:
                        game["validMoves"].append(move)

            if tmp == len(game["validMoves"]):
                break
```

This part of the function deals with compound moves. It goes through every movable point, goes through every compound dice roll at that point and checks whether it's valid. There are extra checks due to the extra moves, such as a compound move into taking off a piece. But overall, the code is very similar to previous valid piece calculating code.

This function needs to be expanded because compound moves can now start off the board or end the board, as that's part of the rules of backgammon. The reason I check the amount of pieces on the bar is less than or equal to one is because I know that a compound move cannot occur if there are two or more pieces on the bar.

main.py handle_make_move()

```
# Moves a piece from one point to another
if move["to"] != None:
    board[move["to"]]["count"] += 1
    if board[move["to"]]["color"] == None:
        board[move["to"]]["color"] = turn

    # Captures piece
    if board[move["to"]]["count"] == 2 and board[move["to"]]["color"] != turn:
        board[move["to"]]["count"] = 1
        board[move["to"]]["color"] = turn
        users[game_rooms[code]["black" if turn == "white" else "white"]]["onBar"] += 1
else:
    users[game_rooms[code][turn]]["borneOff"] += 1

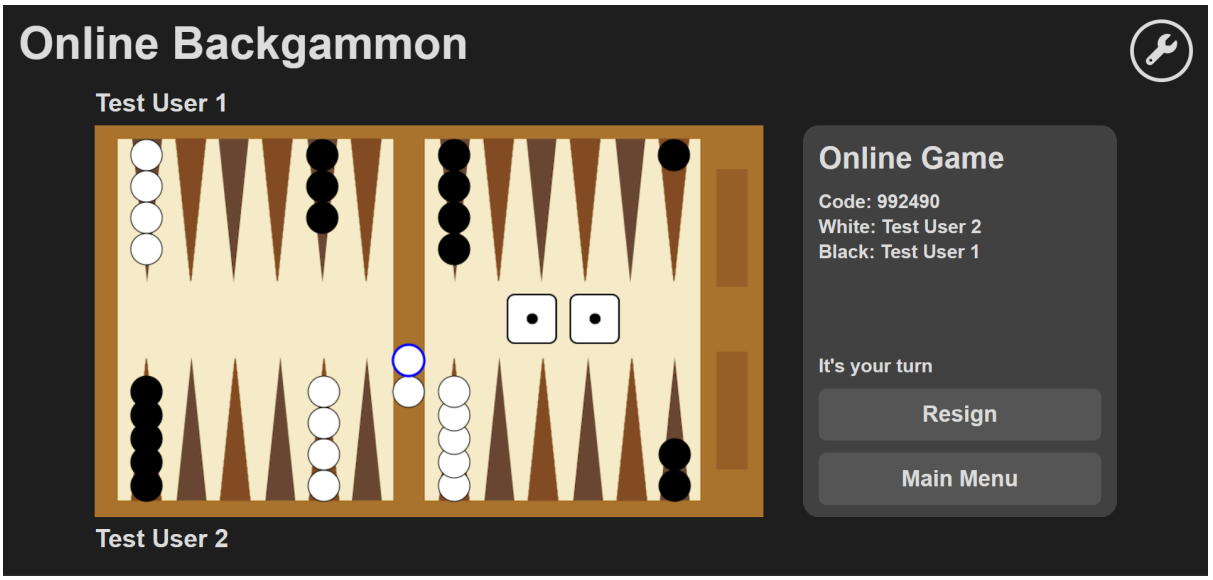
if move["from"] == None:
    if turn == "white":
        users[game_rooms[code]["white"]]["onBar"] -= 1
    else:
        users[game_rooms[code]["black"]]["onBar"] -= 1
# Moves a piece from one point to another
elif board[move["from"]]["count"] > 0 and board[move["from"]]["color"] == turn:
    board[move["from"]]["count"] -= 1

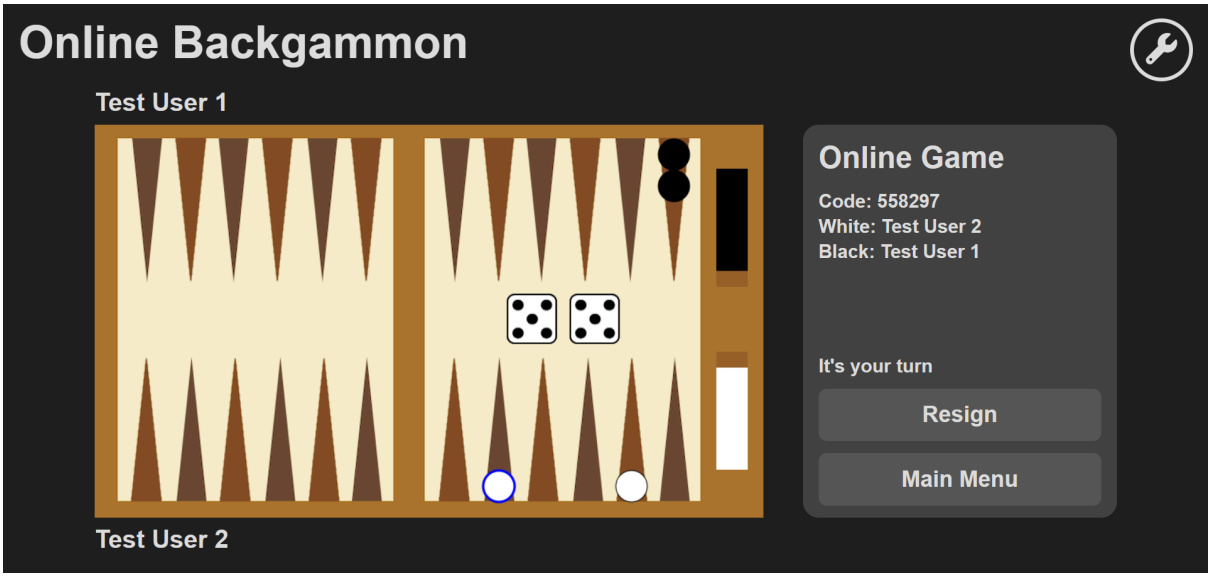
    if board[move["from"]]["count"] == 0:
        board[move["from"]]["color"] = None
```

main.py handle_make_move() is a server-side function that makes a move. This part changes the board variable to reflect this move and has been updated to include moves from the bar and moves that bear off pieces. If move[“to”] is None (i.e. bearing off) it will add one to the borneOff variable, otherwise the same previously implemented logic will apply. If move[“from”] is None (i.e. taking off from the bar), the number of pieces on the bar is subtracted by one.

This function needs to be improved to allow extra backgammon moves that I’ve added in this iteration to take place. Since the bar and bearing off pieces work slightly differently to regular backgammon points, special logic and rules must apply in order for the game to properly function. For example, the bar doesn’t have a color. Therefore, you can’t set the bar’s color to None when all pieces have left the bar. This would cause an error.

GUI Output





Testing

Test	Req. tested	Expected result	Result	Remedial action
Piece moving	1.3, 2.6, 2.7	Making sure the user can move a piece in order to play the game.		None
Win condition checking	2.7, 2.8	Making sure the server is capable of recognising when a		Needs to be implemented

		player has won or not in order to end the game.		

Iteration 3

The aim of this iteration is to figure out when someone's won and end the game. When the game ends, no more moves should be able to be played and a win message should be displayed. While I contemplated creating a new popup modal from scratch that tells the player they won, I chose not to because it would take too much time and code. Instead, I just displayed the words winner and loser next to the usernames when the game ended. Thus, to implement these features, I need functions to check when the game ends and a function to update the username text.

Code

main.py handle_receive_game_info()

```
# Checks if someone has won and sets the winner variable accordingly
for color in ["white", "black"]:
    if users[game_rooms[code][color]]["borneOff"] >= 15:
        game_data["winner"] = color
        game_data["turn"] = color
        break
```

main.py handle_receive_game_info() is a procedure that runs when a client requests game info. This is a small part of the function that checks whether to respond with a winner. It checks the amount of borne off pieces. If the amount is greater than or equal to 15, that means that all pieces have been taken off, the game has ended and the player who took off all their pieces has won.

The reason this part of the function is necessary is the game needs to end. According to the rules of backgammon, the game must end when a player has taken off all of their pieces. Since I'm recreating online Backgammon, I must follow this rule. Interestingly, I set the game's turn to the winner's color. This is because I was experimenting with several ways to stop moves from being able to be played and setting the game's turn to the winner's color is the easiest way to achieve this. This is because the winner has no valid moves (all his pieces are gone) yet, since the game thinks it's the winner's turn, the game waits for the winner to play a valid move. Thus, the game waits indefinitely, essentially softlocking the game and preventing further moves from being played.

network.js resetUsernames()

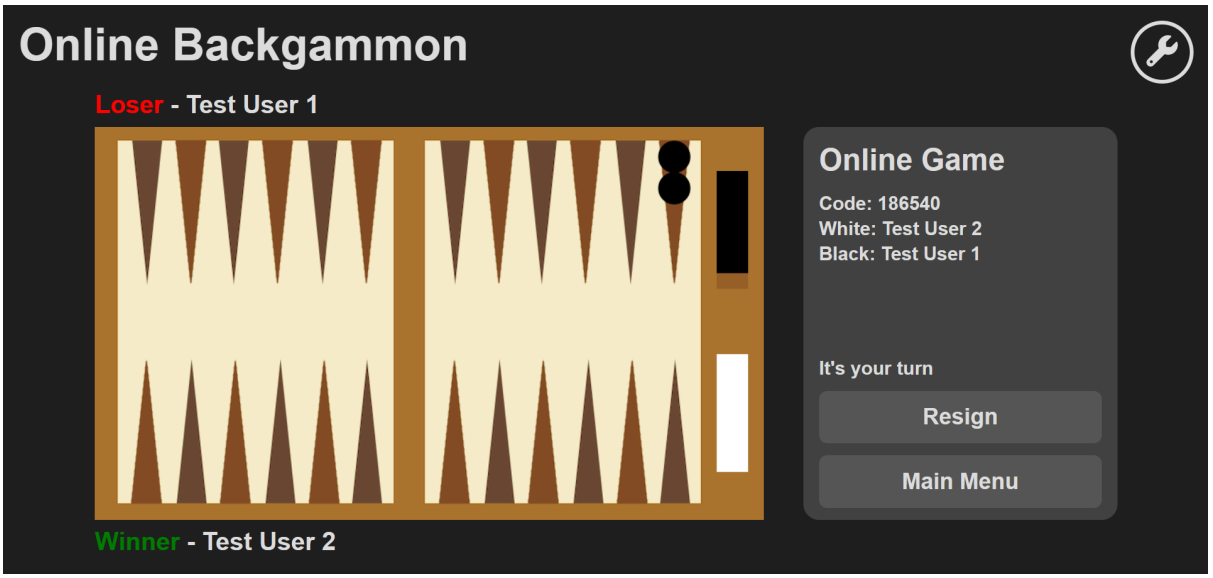
```
// Function that resets the usernames and adds winner and loser prefixes
function resetUsernames() {
    let winPrefix = "<span style='color: green;'>Winner</span> - ";
    let losePrefix = "<span style='color: red;'>Loser</span> - ";

    // Checks if there is no winner
    if (winner == null) {
        if (color == "white") {
            document.getElementById("player1").textContent = black;
            document.getElementById("player2").textContent = white;
        } else {
            document.getElementById("player2").textContent = black;
            document.getElementById("player1").textContent = white;
        }
    // Checks if the winner is white
    } else if (winner == "white") {
        if (color == "white") {
            document.getElementById("player1").innerHTML = losePrefix + black;
            document.getElementById("player2").innerHTML = winPrefix + white;
        } else {
            document.getElementById("player2").innerHTML = losePrefix + black;
            document.getElementById("player1").innerHTML = winPrefix + white;
        }
    // Checks if the winner is black
    } else {
        if (color == "white") {
            document.getElementById("player1").innerHTML = winPrefix + black;
            document.getElementById("player2").innerHTML = losePrefix + white;
        } else {
            document.getElementById("player2").innerHTML = winPrefix + black;
            document.getElementById("player1").innerHTML = losePrefix + white;
        }
    }
}
```

network.js resetUsernames() is a function that resets usernames. Whatever is currently being displayed as usernames is removed and replaced with plain usernames stored on the server. It also adds the winner and loser prefix, depending on whether someone has won and who won.

This function is essential to letting the players know the game is over. Without adding a winner and loser prefix before each username, the players have no indication that the game is over. The reason I use if statements and repetitive code is for readability and, also, trying to make my code more concise and less repetitive proved a challenge. This is particularly because of the amount of cases to consider (whether someone won, who won, what your colour is etc.)

GUI Output



Testing

Test	Req. tested	Expected result	Result	Remedial action
Win condition checking	2.7, 2.8	Making sure the server is capable of recognising when a player has won or not in order to end the game.		None

Validation

While there will be user interaction with this module in the final solution, every user interaction involves them interacting with predefined buttons that I have coded. As a result, this module is self-validating.

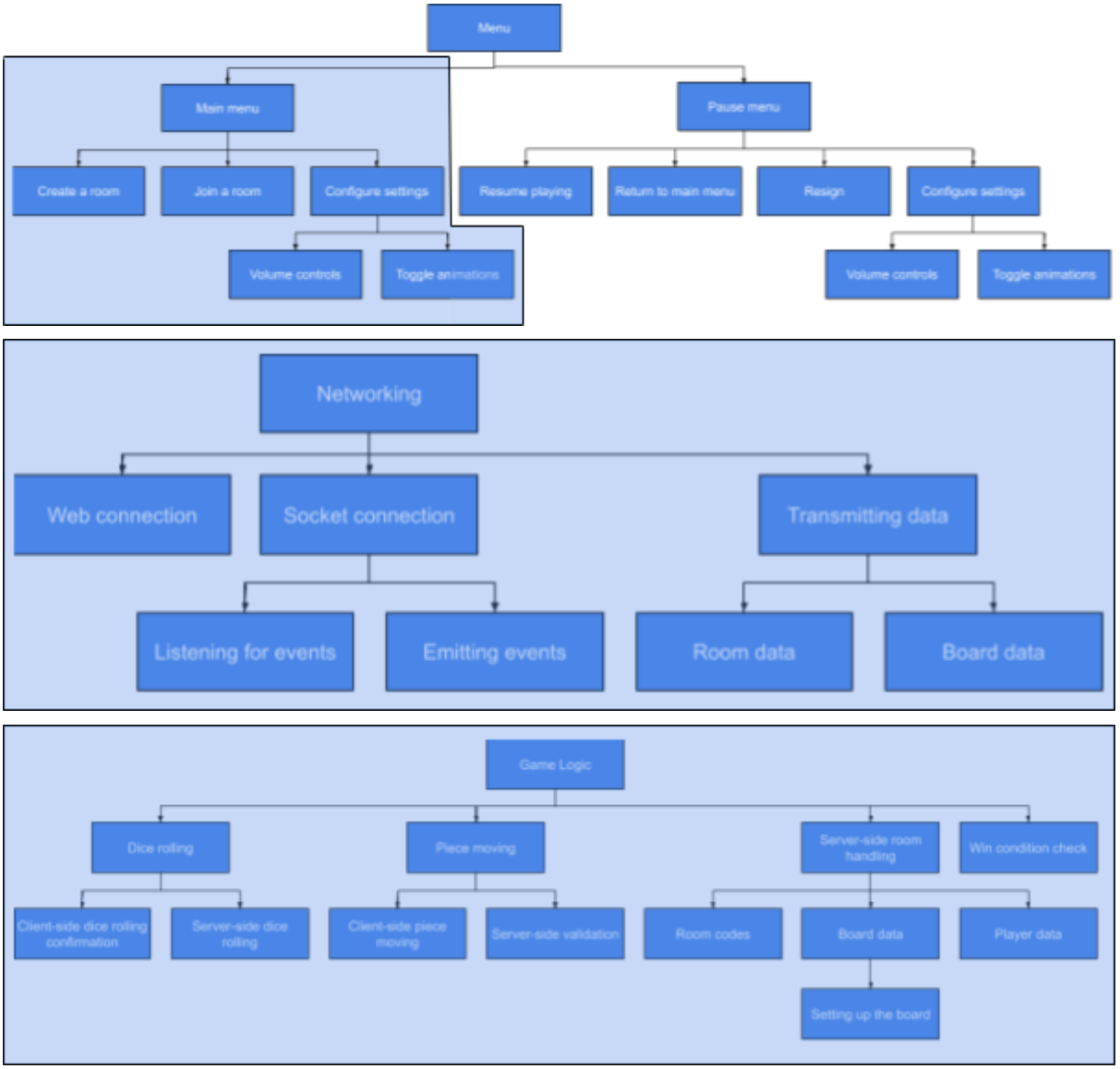
Review

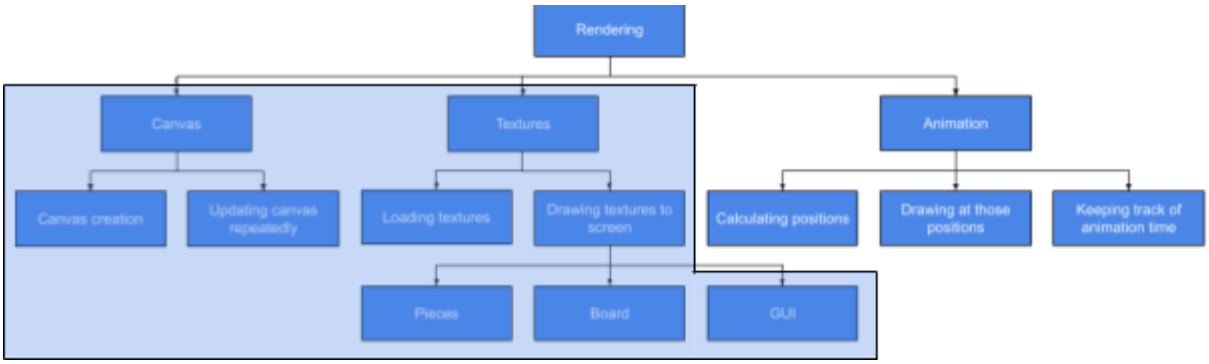
So far, I've currently implemented a web server, a fully functional main menu, a game web page that displays the backgammon board with pieces, a functional multiplayer room-based system and a functional multiplayer backgammon game. These were the largest modules of the project and I'm incredibly relieved that I was able to complete them before the resubmission deadline.

I've shown my stakeholder the project and he is very happy with how it turned out, especially with the compound moves that he strongly urged me to implement.

Currently, I have a fully functional project and all key modules are fully developed. Unfortunately, there are some modules which I never managed to complete. For example, I was meant to implement a feature that allowed the player to resign and return to the main menu. In my solution, the player cannot resign and returning to the main menu involves closing the tab and opening the website in another tab. Also, I was meant to add piece animations, allowing pieces to not instantly teleport from one point to another. However, I don't consider these modules to be essential to the final solution.

For better clarity, I will show all of the decomposed modules and highlight the modules that are developed.





Maintainability

The clear commenting, the modular approach, the way I separate functions into specific files depending on what the function does and the naming conventions I employ make my solution maintainable. Clear commenting allows a programmer to understand what my code does and why it does that, allowing them to debug or potentially improve the code. The modular approach is useful for event-driven languages, such as Javascript, allowing me to write functions that run when the user submits input or clicks somewhere. It's also useful for sockets and networking code, where I need functions that run when a response is received to interpret the response. It's also useful for game development as games require a lot of reusable code (for rendering many pieces or for getting input). Separating functions into specific files is useful to prevent a single file from being absurdly long. I have several Javascript files (appearance.js, game.js, menu.js, network.js and settings.js) and their names are self-explanatory. If a programmer needs to add more Javascript functions or find a specific function, these files make the task trivial. Finally, I have attempted to make the naming conventions as consistent as possible. Python files use snake case for everything except dictionary keys, which use camel case. Javascript typically uses camel case for everything except network events, which use snake case (since it's interacting with a Python server). These features make my solution maintainable.

Maintainability feature	Example

Clear commenting	`// Board class, contains board data and functions for rendering`
	`class Board {`
	`  constructor() {`
	`  // Board data is an array with 24 elements (for 24 points on a Backgammon board)`
	`  // Each point stores 2 values: the number of pieces and the color of pieces`
	`  this.board = [];`
	`  this.img = new Image();`
	`  this.img.src = "static/assets/board.png";`
	`  }`

Modular approach	`def handle_roll_dice(userid):` `def handle_make_move(userid, move):`

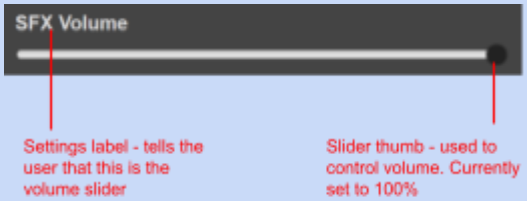
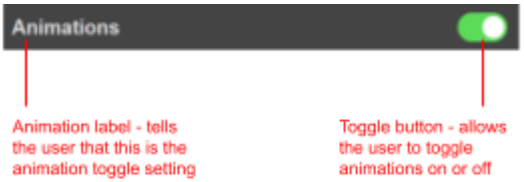
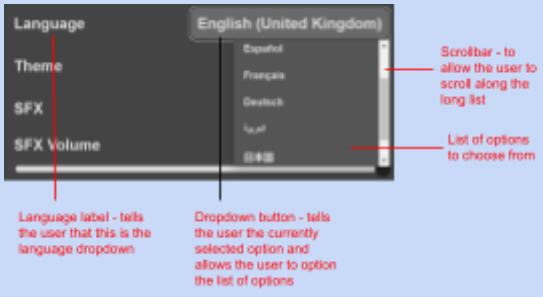
	`function handleClick(event) {` `function draw() {`
Organising functions into files	JS appearance.js JS game.js JS menu.js JS network.js JS settings.js

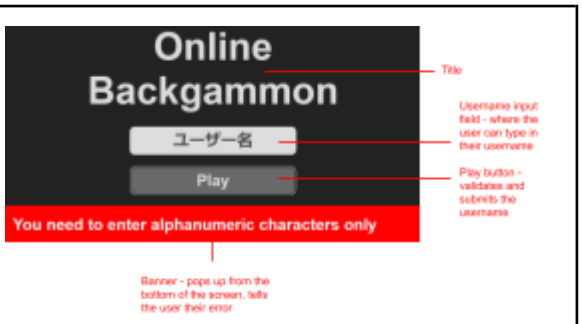
Naming conventions	For Javascript
	`this.dice1 = NaN;`
	`this.dice2 = NaN;`
	`this.activeDice = [];`
	`this.diceRollStart = NaN;`
	`this.diceRollStop = NaN;`
	`this.hideDice = true;`
	`this.hideRollDiceButton = true;`
	
	`this.rollDiceButtonImg = new Image();`
	`this.rollDiceButtonImg.src = "static/assets/rollDiceButton.png";`
	`this.diceImg = new Image();`
	`this.diceImg.src = "static/assets/dice.png";`
	
	For Python
	`waiting_rooms = {}`
	`game_rooms = {}`
	`users = {}`

Post Development Testing

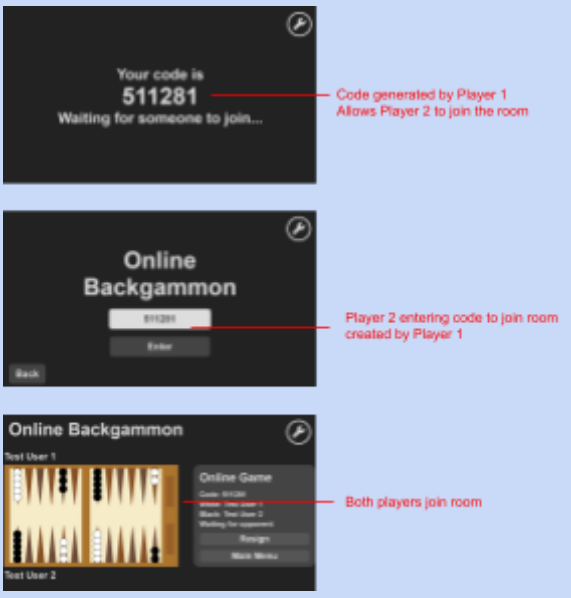
Function and Robustness Testing

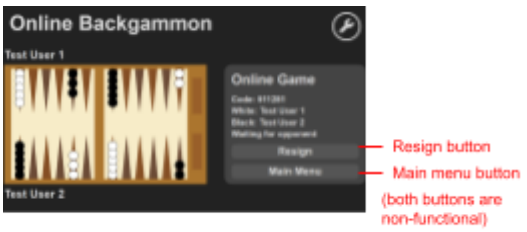
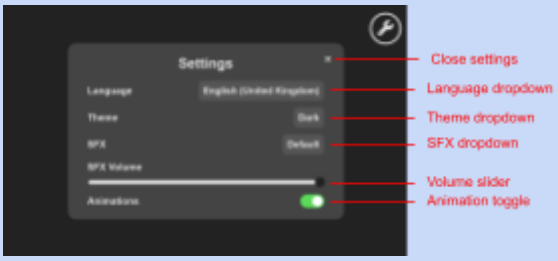
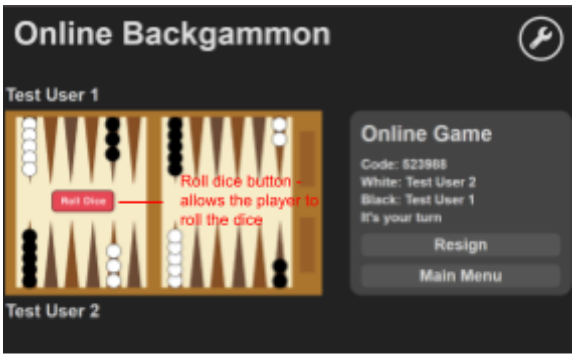
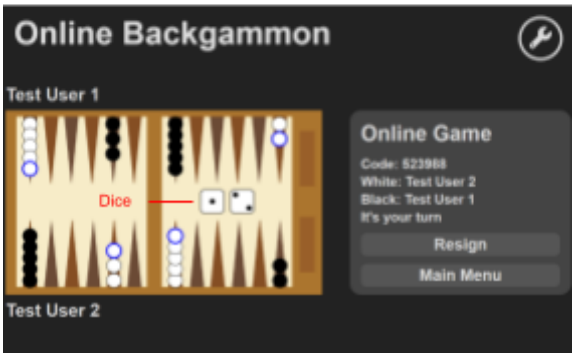
The following tests can be found at the bottom of the design section. Note that I have added several tests to accompany any features which I did not plan to develop in my design section. These tests have been carried out by me and my stakeholder and the collective consensus and results are documented here. Green represents a successful test, amber represents a partially successful test and red represents a failed test.

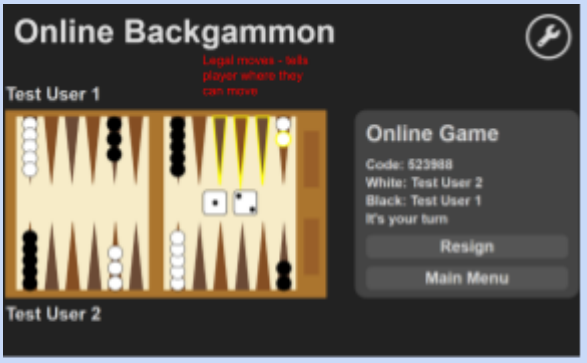
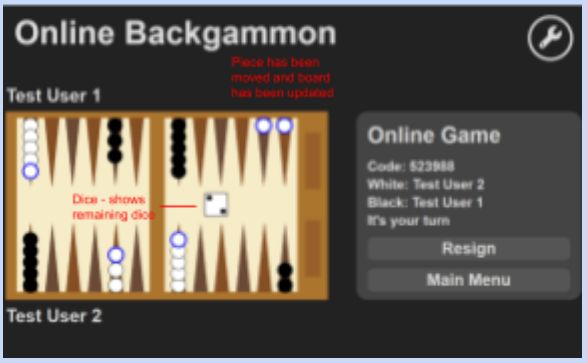
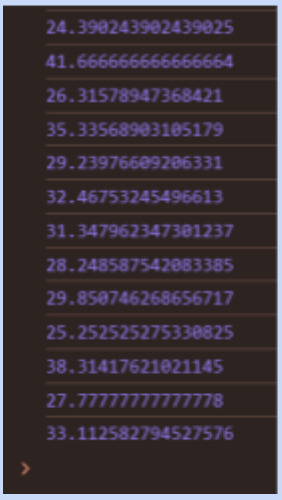
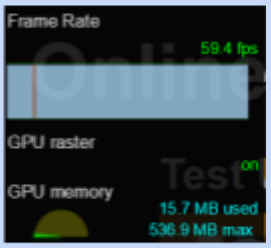
Test	Result	Pass/ Fail	Evidence
Volume control	The player can change the volume of the sound effects to anything between 0 and 100%		
Toggle animations	The player can toggle animations on and off. This works for UI elements but not for game elements, like pieces		
Selecting an option from dropdowns	The player can click on a dropdown to open a list of choices. They can then select one of those choices		

Entering a username	The player can enter any string of alphanumeric characters only		
Entering a code	The player can type any 6 digit integer and join the game if the code is valid		

Usability Testing

Test	Result	Pass/ Fail	Evidence
Menu (creating and joining a game)	The player can create and join a game		

Menu (pause)	The player cannot pause, resign, rematch, resume play or return back to the main menu. Due to time constraints, I was unable to implement these features		
Menu (settings)	The player can change some settings		
Game logic (rolling a dice)	The player can roll a dice		 

Game logic (moving a piece)	The player can move a piece to any legal spot on their turn		 
Game logic (win condition)	The player can win when they have no pieces left		
Rendering (running game smoothly)	The game runs smoothly at 30fps all the time		FPS of canvas printed to console calculated in Javascript using maths and performance.now()  FPS of website calculated by Chrome DevTools 

Stakeholder Feedback

I have decided to ask my stakeholder how usable my solution is. In order to do this, I have given my stakeholder ample time to thoroughly go through all of the functions of my solution, judge how well my solution fits the usability requirements and comment on his judgement. Annotated evidence of the testing has been shown above in usability testing. The results are shown below.

Requirements	Pass/Fail	Comments
Creating and joining a game		
Pausing		The game is turn-based; pausing is trivial (just don't take your move), but the 'pause menu' buttons don't work
Settings		
Dice rolling		
Piece moving		If a roll cannot be moved, it is impossible to progress the game
Winning (correctly declaring a winner and loser)		
Rendering (game running smoothly)		

Looking at the results, I am surprised by the fact that my stakeholder marked pausing as partially met. He argues that, since backgammon is a turn-based game, pausing is functional since all you need to do is to stop playing during your turn. He also marked piece moving as partially met because, during a test game, he came across a bug that occurs if, on your turn, you have no legal moves. This bug is rare but severe as it makes the game impossible to continue as the game expects you to make a move when none are possible. In order to fix this bug in the future, I should check if there are any legal moves before any moves are made and, if not, the game should switch turns.

Evaluation

Success Criteria

The following table contains my comments on how well the solution meets the success criteria and reasons why I think the solution fully meets, partially meets or fails to meet the criteria, backed by evidence in coding and iterative testing during the development section. In cases where I partially meet or fail to meet the criteria, I have written about what I can do in the future to meet the criteria, reasons why I should try to meet the criteria and how I would go about further developing the solution.

#	Result	Comments	Evidence
1.1		The stakeholder can join the website via their browser and load into and navigate a menu. The menu contains many features and settings options that prevent it from being “barebones”, thus it fully meets the criteria.	Main Menu Iteration 1 Page 56
1.2		While keyboard input isn’t logged (and, in retrospect, there is no reason to log all keyboard inputs), all inputs are registered and perform appropriate actions. As a result, keyboard input fully meets the criteria.	Main Menu Iteration 1 Page 56
1.3		Mouse inputs are logged (purely for debugging purposes when in the game), mouse inputs are registered and perform appropriate actions. Particularly as the mouse is the primary input device of this solution, it is essential that the experience of using a mouse is bug-free and fluid. As a result, I have coded a number of special cases to ensure the mouse functions exactly as intended without any problems and I’m very happy with how it turned out. I’m quite confident to say that mouse input fully meets the criteria.	Backgammon Gameplay Iteration 1 Page 95
2.1		The stakeholder can input a code and the code is displayed as the stakeholder types. Validation has been implemented to check that the code is 6 digits long and consists of only numbers. If either of these criteria fail, an error banner pops up from the bottom of the screen. Otherwise, the stakeholder proceeds to the next menu. This fully meets the criteria.	Main Menu Iteration 2 Page 65
2.2		The stakeholder can input a username and the username is displayed as the stakeholder types.	Main Menu Iteration 2

		Validation has been implemented to check that a username has been typed and that it only consists of alphanumeric characters. If either of these criteria fail, an error banner pops up from the bottom of the screen. Otherwise, the stakeholder proceeds to the next menu. This fully meets the criteria.	Page 65
2.3		The stakeholder can click on a button to create a room. Upon doing so, a valid room code is displayed. This room code can be shared with others and, upon someone else joining, both players load into a game. Therefore, this fully meets the criteria.	Server-side Room Handling Iteration 1 Page 84
2.4		The stakeholder can click on a button to join a valid room. This is logged and the user remotely connects, which leads to both players loading into a game. Therefore, this fully meets the criteria.	Server-side Room Handling Iteration 1 Page 84
2.5		The stakeholder can, on their turn, click on the roll dice button. The dice are rolled with a smooth animation and so the criteria is fully met.	Backgammon Gameplay Iteration 1 Page 95
2.6		The stakeholder can move their pieces once they have rolled the dice. The piece selecting and moving works and the stakeholder can only move their pieces to valid spots. I am particularly happy with successfully implementing compound moves, which allows the stakeholder to move the sum of both dice rolls in one move, while preventing bugs and dealing with all the edge cases. However, despite this, the criteria is only partially met because drag and drop has not been implemented and neither has piece moving animations, due to time constraints.	Backgammon Gameplay Iteration 1 Page 95
2.7		Whenever an update happens to the server-side board, either a roll dice request or a move piece request, the update is always validated. The server always checks whether it's the mover's turn to move the piece and roll the dice and whether the move is legal or not. If valid, the update is sent back to both players in the room, so both players can see whatever move the stakeholder makes. As a result, this criteria has been fully met.	Backgammon Gameplay Iteration 1 Page 95
2.8		The stakeholder has no abilities to rematch an opponent when the game ends. If they want to	N/A

		rematch, they must return to the main menu, create a new room code and join the new room code. This can become inconvenient, especially when two players rematch consecutively. Unfortunately, this criteria has not been met because, due to time constraints, I was unable to implement a resign/rematch system.	
3.1		I have thoroughly tested the solution during gameplay and no visual bugs have occurred and the board is properly displayed regardless of the board state. As a result, this criteria has been fully met.	Rendering the Game Iteration 2 Page 79
3.2		I have thoroughly tested the solution during gameplay and no audio bugs have occurred. Whenever a UI element is clicked on, a sound effect plays and the volume slider works as expected. Therefore, the criteria has been fully met.	Main Menu Iteration 2 Page 65
3.3		The stakeholder can use the settings menu to customise the theme, language, audio and animations on the website. This makes my solution very customisable and tailored to whatever my stakeholder's needs are. Language support is a nice feature, if I were to deploy my app to an international market, which I probably won't. Regardless, this fully meets the criteria.	Main Menu Iteration 2 Page 65
3.4		After using Chrome DevTools and their inbuilt fps counter as well as adding a few lines of Javascript to calculate the time between rendering each frame and subsequently calculating the fps, I can say that the canvas draws at around 30fps while the web page renders at 60fps. This fully meets the criteria.	Rendering the Game Iteration 2 Page 79

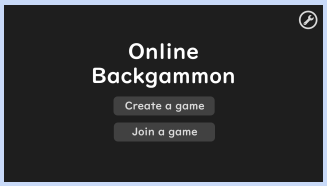
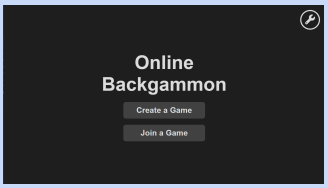
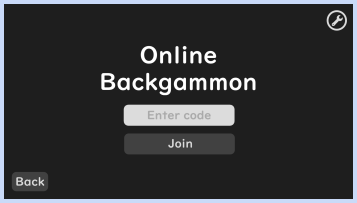
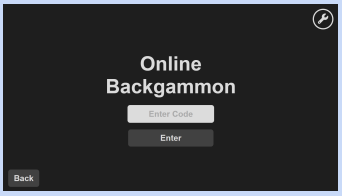
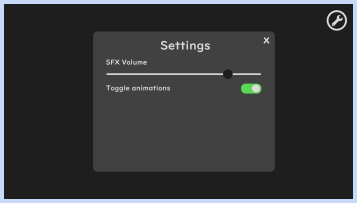
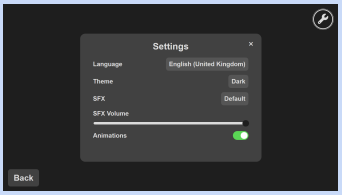
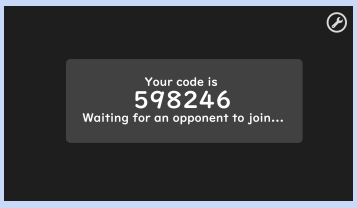
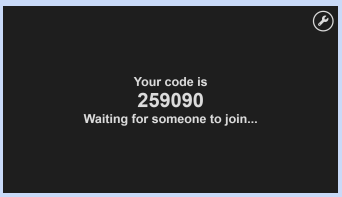
Unmet criteria

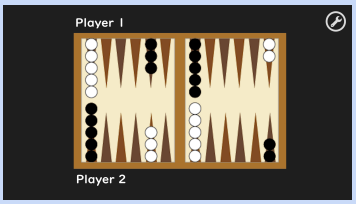
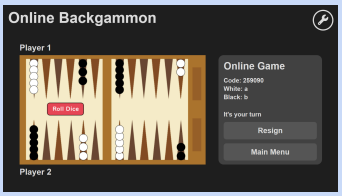
Piece moving has not been fully implemented because I did not implement drag and drop functionality and I did not implement piece animations, due to time constraints. In the future, I would add both of these features to make the user experience smoother and better. This is because some people prefer animations and some people prefer drag and drop over clicking. To implement animations, I would have to figure out a piece's starting position, ending position, animation length, frame number and calculate a piece's position on a given frame. To implement drag and drop, instead of listening to clicks only, I'd have to listen for mouse down and up events. Then, when a user is selecting a piece, I would have to make the piece follow the mouse cursor continuously. These implementations would be possible if I had more development time.

Resignation and rematching opponents has not been fully implemented, due to time constraints. In the future, I would like to add this feature to ensure that rematches and consecutive games between two players is a better experience and that it becomes easier and hassle-free. Implementation of this feature would be relatively straightforward in my mind. I already have code to set up the board, the game and determine who goes first and second. I just need to reuse that code whenever a player clicks on the rematch button. This implementation would be possible if I had more development time.

Usability features

I have not specified success criterias for my usability section in my design section. As a result, in this section, my stakeholder and I will simply compare my design specifications with my final solution and attempt to evaluate and justify how well my final solution follows the design specifications, in terms of appearance and functionality.

GUI Design	Final GUI	Result	Justification
			Looks identical and functions exactly as planned. Therefore, it fully succeeds in being an effective usability feature.
			Looks identical and functions exactly as planned. Therefore, it fully succeeds in being an effective usability feature.
			Looks identical and functions even better than planned due to extra options added during development. Therefore, it fully succeeds in being an effective usability feature.
			Does not look identical unfortunately but it still functions exactly as planned. Therefore, it fully succeeds in being an effective usability

			feature.
			Visually, the final solution does not match the design spec. A title has been added, the roll dice button (an important gameplay feature) was overlooked during design and the resign and main menu buttons are in an info box instead of in the settings menu. Personally, I prefer the new look compared to the design spec. Functionally, the usability features are nearly identical. You can play backgammon and open settings. You could even argue that the resign and main menu features are even more accessible in the final solution as they aren't hidden in settings. Unfortunately, the resign and main menu buttons aren't functional. As a result, it fails in being an effective usability feature.

Unmet Criteria

The usability features on the game screen are not fully effective. This is primarily because of the resign and main menu buttons not working at all, due to time constraints. In the future, I could implement these features in the final solution. I already have code that sets a winner and a loser. I just need to reuse that code whenever a player clicks on the resign button. I also just need to perform a simple redirect in Javascript to return the player to the main menu upon clicking on the main menu button. This implementation would be possible if I had more development time.

Maintenance

To repeat what I mentioned at the end of the development section, my solution is maintainable thanks to its clear commenting, modular approach, straightforward file/folder structure and naming conventions. However, in this section, I will talk about maintenance immediately following deployment, including corrective maintenance, adaptive maintenance and perfective maintenance.

Corrective Maintenance

This occurs after noticing a bug or error, typically during post development or when the end-user encounters and reports it. The programmer will then figure out the underlying cause of the error and fix it. For example, maybe compound moves won't behave as expected in certain situations because of an obscure edge case. The programmer would have to debug the code (probably the calculateValidMoves function in this case) and fix and update the code.

Adaptive Maintenance

This occurs after certain technology changes. The programmer will then have to update the code to ensure compatibility with this technology. For example, if Javascript suddenly updated and decided to deprecate a bunch of functions that are integral to the solution, the programmer would have to figure out what's being deprecated and change the relevant code to ensure compatibility with modern browsers.

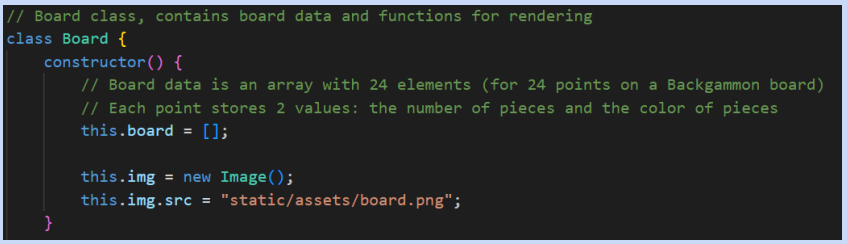
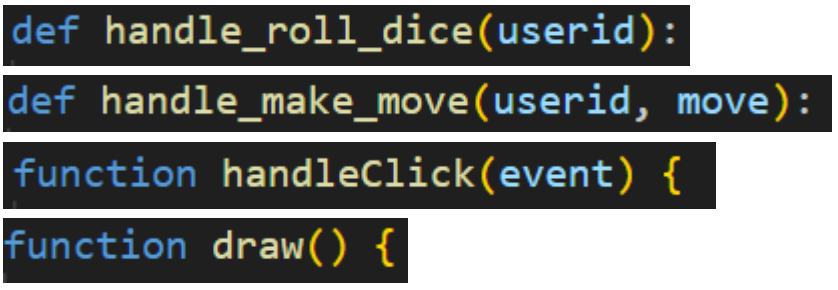
Perfective Maintenance

This occurs when requirements change. The programmer will have to enhance performance or functionality to meet these changing requirements and improve the user experience. For example, if the stakeholder wanted to implement a timer when playing online backgammon to prevent the opponent from taking too long to make their move, this would be an example of a changing requirement. The programmer would then have to add new functions and code to meet this requirement and further develop and improve the solution.

Personally, I feel like I can perform corrective maintenance following deployment. This is because having multiple people use my solution will reveal many underlying bugs and issues which I can then fix. However, I will not perform adaptive maintenance. This is because I don't plan on maintaining my code forever and I believe it will be well into the future before a major update to a language or module occurs. Finally, I will not perform perfective maintenance. This is because the solution requirements are relatively fixed and rigid, especially since the needs of the stakeholder probably won't change and the number of people using the solution will be few, if any.

Maintenance Features

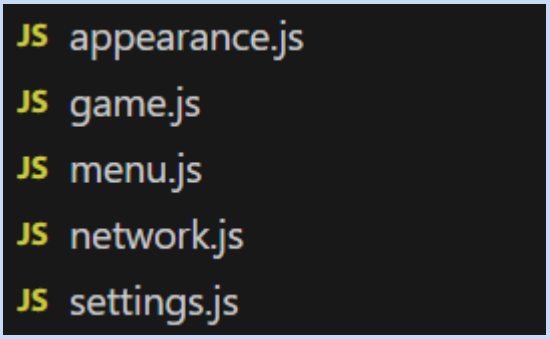
In order to aid 3rd-party programmers in maintaining and developing my solution in the future, I have decided to make my solution self-documenting and maintainable. I have done this by adopting clear commenting, using a modular approach, having a clear naming convention and separating functions into different files. Clear commenting allows a programmer to understand what my code does and why it does that, allowing them to debug or potentially improve the code. The modular approach is useful for event-driven languages, such as Javascript, allowing me to write functions that run when the user submits input or clicks somewhere. It's also useful for sockets and networking code, where I need functions that run when a response is received to interpret the response. It's also useful for game development as games require a lot of reusable code (for rendering many pieces or for getting input). Separating functions into specific files is useful to prevent a single file from being absurdly long. I have several Javascript files (appearance.js, game.js, menu.js, network.js and settings.js) and their names are self-explanatory. If a programmer needs to add more Javascript functions or find a specific function, these files make the task trivial. Finally, I have attempted to make the naming conventions as consistent as possible. Python files use snake case for everything except dictionary keys, which use camel case. Javascript typically uses camel case for everything except network events, which use snake case (since it's interacting with a Python server). The table below shows examples of maintainability features in my solution.

Maintainability feature	Example
Clear commenting	
Modular approach	

```
// Board class, contains board data and functions for rendering
class Board {
    constructor() {
        // Board data is an array with 24 elements (for 24 points on a Backgammon board)
        // Each point stores 2 values: the number of pieces and the color of pieces
        this.board = [];

        this.img = new Image();
        this.img.src = "static/assets/board.png";
    }
```

```
def handle_roll_dice(userid):
def handle_make_move(userid, move):
function handleClick(event) {
function draw() {
```

Organising functions into files	

Naming conventions	For Javascript
	`this.dice1 = NaN;`
	`this.dice2 = NaN;`
	`this.activeDice = [];`
	`this.diceRollStart = NaN;`
	`this.diceRollStop = NaN;`
	`this.hideDice = true;`
	`this.hideRollDiceButton = true;`
	
	`this.rollDiceButtonImg = new Image();`
	`this.rollDiceButtonImg.src = "static/assets/rollDiceButton.png";`
	`this.diceImg = new Image();`
	`this.diceImg.src = "static/assets/dice.png";`
	
	For Python
	`waiting_rooms = {}`
	`game_rooms = {}`
	`users = {}`

Limitations

The limitations below are ideas that would improve the overall experience but are incredibly difficult to implement without recoding large portions of the code. As a result, I won't be able to develop these aspects further, limiting the development of the solution.

Limitation	Description
Computer opponents	In order to use the solution, you must play against someone else. This is a problem if you have no friends. The game offers no single-player option and this becomes a limitation if the user wants to play backgammon but no one is available to play with them. To fix this, I would need to implement computer opponents. I could even add difficulty levels to the bots, where the user can manually adjust the strength of the bot, allowing players of all skill levels to enjoy this feature.

Account system	There is no way to track the amount of games you've won or lost over a long period of time. This could be a limitation if a user wants to track their progress and see whether they've improved or see their score against a certain opponent. To fix this, I would need to implement an account system as individual stats can be stored for each account. This would involve databases, storing usernames and passwords and separate UIs for logging in and registering.
Chat system	There is no way to chat or trash-talk your opponents during a game. This is a limitation as, if you are playing remotely and aren't on a call, the experience can be boring and lonely. Adding a chat system could fix these problems as the option to chat to your opponent could add elements of communication or psychological warfare, making the experience much more fun.
Doubling cube	There is no way to utilise the doubling cube, a backgammon mechanic, in my solution. This is a limitation as, in real backgammon, you have the option to double the stakes and force your opponent to resign or accept the doubling. This means that my solution does not implement all of the rules of backgammon and adding this feature could make consecutive backgammon games more fun and strategic.
Drag and drop	There is no way to drag and drop pieces. This is a limitation as some people will prefer to drag and drop pieces from one point to another and my solution offers no such functionality. To fix this, I would need to implement this functionality to improve the user experience for some people.

Improvements

Improvements are ideas that would improve the overall experience and, unlike limitations, are not incredibly difficult to implement as they don't involve recoding large portions of the code. They don't limit the development of the solution and, in the future, I might implement these features.

Improvement	Description
Animations	Animations would make the solution feel more polished. It would also be more visually pleasing and make it easier to follow certain moves as you can easily tell what moves where.
More settings	As I added more settings in the development section, I noticed that I was running out of space to place additional options. Adding a scrollbar would allow me to add even more settings and more settings make the solution more customisable, making the user experience better or allowing the solution to run on lower-end devices.

SFX	Sound effects, like animations, would make the solution feel more polished. It would provide audio feedback for every move or action the user takes, improving the user experience and making it feel more responsive.
Board flipping	Board flipping, in this case, is when the board flips to make it easier for black to play. This would be convenient as, currently, black plays from white's perspective. While playable, it is not realistic to the real-life backgammon experience.
Tutorial	A tutorial would be highly beneficial to players who are new to backgammon. Backgammon is quite a complex game and a tutorial would make it easier for players to learn and play straight as soon as possible, making it more fun for the player.
Profanity filter	When testing, I noticed that players can enter anything as their username. This can lead to many problems. A profanity filter could detect swear words and prevent players from entering inappropriate usernames. This would improve the experience significantly, especially for children or those easily offended.

Programming Techniques

Similar to how I developed the solution, to further develop and improve the solution, I'd use a combination of abstraction and decomposition to figure out what the problem is and the best approach to tackle it. I would probably use a lot of procedures as reusable code is immensely useful for future development. For example, I already have procedures to play sound effects since sound effects are played when UI elements are interacted with. As a result, adding more sound effects to the actual game will be simple.

To implement animations, I could calculate a piece's position depending on the starting position, ending position, animation length and current frame. Then I could render the piece at that specific position using reusable functions (such as the renderPiece function that I coded).

To implement settings, I already have code for sliders, dropdowns and toggle buttons. To make the settings modal scrollable, I can set the CSS property "overflow-y" to "scroll".

To implement board flipping, I modify my rendering functions. If the user is black, it needs to flip the board vertically and, for all mouse clicks, it needs to flip the mouse's y-coordinates vertically.

To implement a tutorial, I can implement a modal similar to settings. I already have the code for such a modal thanks to reusable functions. On the modal, I just need to add text and images with HTML's <p> tags and <img> tags.

To implement a profanity filter, I just need to go to where my code validates the username and performs an alphanumeric check and a presence check. I just need to add another check to ensure that the username contains no inappropriate words. I can do this by iterating over a list of inappropriate words and checking if the word is in the username or not.

Bibliography

- 247 Backgammon
  https://www.247backgammon.org/

- Backgammon js
  https://github.com/quasoft/backgammonjs

- Kahoot
  https://kahoot.com/

- Flask documentation
  https://flask.palletsprojects.com/en/stable/

- Flask SocketIO documentation
  https://flask-socketio.readthedocs.io/en/latest/

- Wrench icon SVG
  https://fontawesome.com/icons/wrench?f=classic&s=solid

- Popup, slider, toggle button and dropdown code
  https://www.w3schools.com/howto/howto_css_modals.asp
  https://www.w3schools.com/howto/howto_js_rangeslider.asp
  https://www.w3schools.com/howto/howto_css_switch.asp
  https://www.w3schools.com/howto/howto_css_dropdown.asp

- Localisation code
  https://medium.com/@nohanabil/building-a-multilingual-static-website-a-step-by-step-guide-7af238cc8505